

Post-Quantum Shielded-Pool zk-Rollup (WHIR stack)

Specification extracted from the implementation at commit a54c197

zkcrypto-audit engagement toy-zk-rollup-2026-10-09

9 October 2026

Contents

1	System overview and actors	2
1.1	What the system is	3
1.2	Components	3
1.3	Pipeline	3
1.4	Node HTTP surface	5
1.5	Actors and trust boundaries	5
2	Cryptographic primitives and encodings	7
2.1	Base field	7
2.2	Quartic extension	7
2.3	Montgomery encoding on the wire	8
2.4	The Poseidon2 permutation	8
2.5	Byte, limb and digest encodings	9
2.6	Shielded-layer hashes	9
2.7	Keccak Fiat-Shamir transcript	10
3	Shielded state: keys, notes, trees	10
3.1	Hash instantiation in the shielded layer	11
3.2	Keys	11
3.3	Notes, commitments and nullifiers	12
3.4	Note-commitment tree	13
3.5	Nullifier sparse Merkle tree	14
3.6	Constants	16
4	The transfer relation	16
4.1	Shape parameters and domain tags	17
4.2	In-circuit gadgets	17
4.3	Instance, circuit constants and witness	18
4.4	The relation	19
4.5	Statement layout	20
4.6	Value conservation over the integers	20
4.7	Scope of the relation	21
4.8	Spec deltas	21
5	The block relation and statement fold	22
5.1	Transfer shapes and statement projections	22
5.2	Child objects and in-circuit verification	22
5.3	The block relation	23
5.4	The statement fold (D-089)	24
5.5	Block statement layout	25
5.6	L1 decoding: <code>LimbCodec</code> and <code>BlockStatement</code>	25
5.7	Construction order	26
5.8	Scope of the relation	26

6	Proof system: WHIR PCS, batch-STARK, recursion	27
6.1	Instantiations and parameters	28
6.2	WHIR univariate PCS: encoding	28
6.3	WHIR univariate PCS: commit	29
6.4	WHIR univariate PCS: open and verify	30
6.5	Batch-STARK transcript	31
6.6	Recursion relation	32
6.7	Circuit-prover table AIRs	33
6.8	Spec deltas	33
7	The deployed Solidity settlement verifier	34
7.1	Wire format	35
7.2	Statement binding and terminal pre-check	36
7.3	Batch transcript	36
7.4	WHIR opening rounds	37
7.5	Terminal identity	39
7.6	Constraint identity	39
7.7	The satellite and its pin	40
7.8	Spec deltas	41
8	The settlement contract	41
8.1	Contract state	41
8.2	Construction	42
8.3	<code>applyBlock</code> : the state transition	42
8.4	The <code>BlockApplied</code> event	43
8.5	<code>withdrawFees</code>	44
8.6	Deployment	44
8.7	Settlement calldata	44
8.8	Submission	45
9	The application layer: node, spend authorization, custody	45
9.1	The spend-authorization message	46
9.2	SLH-DSA instantiation	46
9.3	The transfer wire format and <code>/v1/transfer</code> admission	47
9.4	In-process mempool admission	49
9.5	Bearer tokens and access control	49
9.6	Custody: the password vault	50
10	Specification deltas (documentation vs enforced code)	51
10.1	Summary	51
10.2	Hash instantiation of the shielded and commitment layers	52
10.3	Who may settle	53
10.4	Genesis nullifier root	53
10.5	Vendored recursion tree	53
10.6	Security level and configuration knobs	54
10.7	Settlement-verifier header	55
10.8	Fees	56
10.9	Pointer to the findings	56

1 System overview and actors

This section fixes the vocabulary used throughout the specification: the components that make up the system, the pipeline that carries a value transfer from a wallet to an on-chain state update, and the actors together with the trust boundaries that separate them. Everything stated here is what the code *enforces*; where README prose or a source comment states an intent that the code does not back, a spec delta records the gap.

1.1 What the system is

The repository implements a shielded value pool that proves its own state transitions with hash-based STARKs and settles them on an EVM chain by replaying the proof under the native `keccak256` opcode [README.md:3-6]. The settlement contract stores a block counter and two 32-byte roots, the commitment-tree root `currentRoot` and the nullifier-map root `currentNullifierRoot` [contracts/src/ShieldedPool.sol:70-77]. Both roots are taken from the verified statement and only checked for continuity; the contract never re-derives or opens either tree [contracts/src/ShieldedPool.sol:151-166].

In the code paths that the client prover, the node and the wallet actually use, every shielded-layer digest is the Poseidon2 sponge H_{P2} (Section 2) applied to the 16-bit little-endian limb encoding of a byte preimage [crates/pq-hash/src/poseidon.rs:100-111][crates/pq-hash/src/poseidon.rs:225-238]:

$$cm = H_{P2}(\text{DOMAIN_NOTE} \parallel \text{value}_{LE,8} \parallel \rho \parallel \psi \parallel pk_d), \quad (1)$$

$$nf = H_{P2}(\text{DOMAIN_NULLIFIER} \parallel sk_d \parallel \rho). \quad (2)$$

The preimage orders are fixed by `Note::commit` and `Note::nullifier` [crates/shielded/src/note.rs:108-127]; the Poseidon2 instantiation is selected by the callers, `Poseidon2Commitment` for (1) and `Poseidon2Shielded` for (2) [crates/prover/src/client.rs:147-155][crates/prover/src/client.rs:204][crates/node/src/main.rs:276-279][crates/pq-hash/src/shielded.rs:98-105]. The commitment tree and the nullifier map are both Poseidon2-hashed [crates/prover/src/client.rs:119-124]; the empty nullifier map is the depth-96 Poseidon2 empty-subtree root, pinned in the contract as `EMPTY_NULLIFIER_ROOT` [contracts/src/ShieldedPool.sol:79-86]. Spend authorization is an SLH-DSA (SPHINCS+, parameter set `Sha2.128f`) signature over a canonical statement encoding [crates/pq-sign/src/spend_auth.rs:89-94][README.md:37-38]. No elliptic-curve primitive is used by the proof, hash or signature crates [README.md:3-5]; signing of the L1 settlement transaction is delegated to the RPC endpoint, the node holding no L1 key [crates/node/src/settlement.rs:10-17]. End to end the demonstration covers per-transfer STARKs, a block proof that recursively verifies its children and folds their statements into one digest, and SNARK-free EVM settlement [README.md:29-54].

Spec delta 1.1. README describes the note commitment as Keccak-256 and the nullifier as SHA3-256 [README.md:31-37], and its layout table assigns “SHA3-256 note/nullifier hashing” to `crates/pq-hash` [README.md:60]. The Keccak-256 and SHA3-256 hashers still exist [crates/pq-hash/src/commitment.rs:30][crates/pq-hash/src/shielded.rs:13], but the client, node and wallet paths select the Poseidon2 hashers of (1)–(2), a switch recorded as D-092 batch 82 [crates/pq-hash/src/shielded.rs:73-85][crates/wallet-wasm/src/lib.rs:533-536] (cross-reference [I-02]).

1.2 Components

Table 1 lists the workspace components and the role each plays, in the wording of the repository layout table [README.md:56-72] (the `crates/pq-hash` row is README’s wording; see the spec delta above).

1.3 Pipeline

Two WHIR configurations are used. The client and recursion layer (`InnerWhirConfig`) runs a Poseidon2 duplex challenger over the width-16 KoalaBear Poseidon2 permutation (rate 8) [crates/prover/src/whir_recursion.rs:83-84][crates/prover/src/whir_recursion.rs:103]; the settlement layer (`whir::Config`) runs a Keccak-256 `SerializingChallenger32` over `HashChallenger` with `Keccak256Hash` [crates/prover/src/whir.rs:72-77]. Only the Keccak layer is replayed on the EVM [.scratch/pq-shielded-rollup/map.md:90]. The path from a transfer to a settled state update is a four-stage composition.

- 1. Client transfer proof.** The spender proves one transfer locally as a `p3_circuit_prover::BatchStarkProof` under `InnerWhirConfig` and retains the matching `CircuitVerifier` [crates/prover/src/client.rs:76-81][crates/prover/src/client.rs:119-124] (README names the proving crate `p3-multi-stark` [README.md:39-40]). This layer is asserted at compile time to run with HVZK blinding [crates/prover/src/lib.rs:109-113]. Each spend draws a nullifier non-membership witness against the nullifier map before the nullifier is inserted [crates/prover/src/client.rs:197-212], and the claimed post-append commitment root is computed from the pre-append frontier that the circuit re-derives [crates/prover/src/client.rs:143-163][contracts/src/ShieldedPool.sol:15-18]; the binding of the append position is examined in [H-02].
- 2. Block / recursion circuit.** The block circuit re-verifies each child proof in-circuit through the vendored `WhirRecursionBackend` [crates/prover/src/block.rs:103][crates/prover/src/block.rs:425]

Path	Role
crates/pq-hash	SHA3-256 note/nullifier hashing; Poseidon2 trees and commitments [README.md:60]
crates/pq-sign	SPHINCS+ (slh-dsa) key/sign/verify wrappers; canonical statement encoding [README.md:61]
crates/vault	password-derived SPHINCS+ vault format, shared by wallet and node [README.md:62]
crates/shielded	transfer statement, witness, admission rules (PoolState) [README.md:63]
crates/prover	transfer and block circuits; WHIR configs (Poseidon2 and Keccak); statement fold; vector generators [README.md:64]
crates/node	sequencer actor; HTTP API; tokens/ACL/rate-limit; settlement submitter [README.md:65]
crates/wallet-wasm	the wasm the extension ships (same crates as the node) [README.md:66]
crates/recursion-test	recursion harnesses against upstream p3-recursion [README.md:67]
contracts/	ShieldedPool, BlockStatement decoder, LimbCodec, generated WhirVerifier and tests [README.md:68]
extension/	MV3 wallet (plain JS + wasm) [README.md:69]
vendor/p3-recursion	the recursion tree (upstream repo, own workspace) [README.md:70]
scripts/	quality gate and the two end-to-end drivers [README.md:71]
.scratch/pq-shielded-rollup/	wayfinder map, decision log, per-task observations [README.md:72]

Table 1: Workspace components and their roles.

[README.md:41-45], constrains child i 's `root_after` to equal child $i+1$'s `root` and chains the nullifier roots likewise [crates/prover/src/block.rs:60-85], and folds the child statements [crates/prover/src/block.rs:273-290]:

$$r_0 = 0^8, \quad r_i = H_{P2}(r_{i-1} \parallel \text{stmt}_i) \quad (1 \leq i \leq n), \quad \text{statementRoot} = r_n, \quad (3)$$

a digest of eight base-field elements exported as 16 little-endian 16-bit limbs (D-089) [crates/prover/src/block.rs:292-305]. A block has $n \geq 1$ children [crates/prover/src/block.rs:331-333][contracts/src/BlockStatement.sol:82]. The child verifier object is supplied alongside each child proof; its pinning is examined in [V-06].

- Keccak re-expression and on-chain replay.** The block proof is produced under the Keccak settlement configuration [crates/prover/src/whir_recursion.rs:615-627], which is asserted at compile time to be non-ZK [crates/prover/src/lib.rs:102-107] (cross-reference [M-10]). The generated Solidity `WhirVerifier` replays `p3_batch_stark::verify_batch` on one Keccak sponge that runs the batch phases and then every WHIR opening round without reseeding [contracts/src/verifier/WhirVerifier.sol:10-11][contracts/src/verifier/WhirVerifier.sol:39-46][contracts/src/verifier/WhirVerifier.sol:282-303].
- Settlement.** `ShieldedPool.applyBlock` reverts unless the injected verifier returns `true`, decodes the statement, requires `rootBefore = currentRoot` and `nfRootBefore = currentNullifierRoot`, overwrites both stored roots with `rootAfter` and `nfRootAfter`, increments `blockNumber`, and emits `BlockApplied` carrying the four roots and the decoded total fee [contracts/src/ShieldedPool.sol:144-178]. The fee is only emitted, and the decoded `statementRoot` is neither stored nor emitted (cross-reference [L-04]).

The block statement carried across stages 2-4 has the fixed layout and length

$$\text{stmt}_{\text{blk}} = [n, (\text{in}_i, \text{out}_i)_{i < n}, \text{statementRoot}, \text{rootBefore}, \text{rootAfter}, \text{nfRootBefore}, \text{nfRootAfter}, (\text{fee}_i)_{i < n}], \quad (4)$$

$$|\text{stmt}_{\text{blk}}| = 1 + 2n + (1 + 4) \cdot 16 + 4n = 81 + 6n, \quad (5)$$

where every digest occupies 16 limbs (limb j holds digest bytes $2j$ and $2j+1$, low byte first) and every fee 4 limbs read as a little-endian `uint64`; the decoder requires $1 \leq n \leq 2^{16} - 1$, an exact length match, and every decoded limb $\leq 0xffff$, and sums the fees into `totalFee` [contracts/src/BlockStatement.sol:11-15][contracts/src/BlockStatement.sol:63-111][contracts/src/LimbCodec.sol:16-64][crates/prover/src/block.rs:265-271]. The length is independent of how many nullifiers and outputs each transfer carries [contracts/src/BlockStatement.sol:65-68].

1.4 Node HTTP surface

The router registers the eight routes of Table 2, attaches the authentication and rate-limit guard as a route layer (it runs only for matched routes), a CORS layer allowing GET/POST with the **Authorization** and **Content-Type** headers, and a request-body limit of 1048576 bytes [crates/node/src/main.rs:717-741] [crates/node/src/main.rs:90]. The guard answers 404 for a path absent from the ACL, 401 for a missing, malformed, badly MACed or expired token, 403 for an insufficient role, and 429 when the limiter bucket keyed by (role, path) is exhausted [crates/node/src/main.rs:543-581] (cross-reference [M-11]). A token is

$$\text{tok} = \text{b64url}(m) \parallel . \parallel \text{b64url}(\text{HMAC-SHA256}(k, m)), \quad (6)$$

$$m = \text{role (1 byte)} \parallel \text{expiry (8 bytes, LE)}, \quad (7)$$

verified in the order format, MAC (constant-time), payload length 9, role byte, expiry [crates/node/src/auth.rs:13-15] [crates/node/src/auth.rs:190-211] [crates/node/src/auth.rs:236-253]. The ACL is deny-by-default and maps each exact path to the minimum role [crates/node/src/auth.rs:256-302]. The three roles are totally ordered, **ReadOnly** = 0 < **Submitter** = 1 < **Admin** = 2, with the access test role $\geq$ required [crates/node/src/auth.rs:56-65] [crates/node/src/auth.rs:219-234].

Endpoint	Min. role	Behavior (handler bound by the router)
GET /health	ReadOnly	health : node snapshot JSON (block number, note and spent counts, both roots, pending count) [crates/node/src/main.rs:727] [crates/node/src/main.rs:583-589] [crates/node/src/main.rs:205-214] [crates/node/src/auth.rs:277]
GET /metrics	ReadOnly	Prometheus text exposition [crates/node/src/main.rs:728] [crates/node/src/main.rs:602-608] [crates/node/src/auth.rs:278]
GET /v1/state	ReadOnly	bound to the same health handler; returns the same snapshot JSON [crates/node/src/main.rs:729] [crates/node/src/auth.rs:280]
GET /v1/roots	ReadOnly	the snapshot's commitment and nullifier roots [crates/node/src/main.rs:730] [crates/node/src/main.rs:591-600] [crates/node/src/auth.rs:281]
POST /v1/transfer	Submitter	parse the envelope and verify its SPHINCS+ signature against the envelope's own verifying key (cross-reference [M-02]), then an actor admission check; envelope or state rejection (double spend, stale root) $\rightarrow$ 422; valid and admissible $\rightarrow$ 501 (remote proof admission deferred, D-079) [crates/node/src/main.rs:731] [crates/node/src/main.rs:619-659] [crates/node/src/wire.rs:104-121] [crates/node/src/auth.rs:283]
POST /v1/demo/transfer	Admin	scripted demo prover path over fixture notes [crates/node/src/main.rs:732] [crates/node/src/main.rs:669-687] [crates/node/src/auth.rs:288-290]
POST /v1/block/produce	Admin	actor Produce command (prove and seal the pending block) [crates/node/src/main.rs:733] [crates/node/src/main.rs:689-701] [crates/node/src/auth.rs:285]
POST /v1/block/settle	Admin	actor Settle command; returns the L1 transaction hash [crates/node/src/main.rs:734] [crates/node/src/main.rs:703-715] [crates/node/src/auth.rs:287]
POST /v1/admin/rotate-key	Admin	ACL entry only: no route is registered, so the router's default not-found response applies and the guard does not run (cross-reference [L-03]) [crates/node/src/auth.rs:292] [crates/node/src/main.rs:726-734]

Table 2: Node endpoints and the minimum role each requires. A path not listed in the ACL is denied. README lists GET /health with no role [README.md:120]; the ACL requires ReadOnly (D-082) [crates/node/src/auth.rs:273-277].

1.5 Actors and trust boundaries

Table 3 enumerates the actors and what each is able to do. The contract's stated trust model treats the prover and sequencer as untrusted and the injected verifier as correct [contracts/src/ShieldedPool.sol:51-55]; in code, every state transition is gated on **IWhirVerifier.verify** returning **true** [contracts/src/ShieldedPool.sol:145].

Actor	Capability / trust	Boundary
Wallet / note owner	Holds SPHINCS+ keys in a password-protected vault in wasm; builds and signs transfers locally [README.md:52-54]. README states that the node sees only nullifiers, commitments and a signature [README.md:166-167]; the client verifier object handed to the block producer carries further per-transfer constants (cross-reference [H-03])	wallet → node
Submitter-token holder	May POST /v1/transfer a SPHINCS+ envelope [crates/node/src/auth.rs:283]	client → node
Admin-token holder	Drives /v1/block/produce, /v1/block/settle, /v1/demo/transfer [crates/node/src/auth.rs:285-290]; the ACL entry for /v1/admin/rotate-key has no route [crates/node/src/auth.rs:292]	operator → node
Sequencer / prover (node)	Stated as untrusted for <i>what</i> and <i>when</i> it batches [contracts/src/ShieldedPool.sol:53-55]; every transition it settles must pass the verifier gate, whose binding to the honest circuits is examined in [V-01] and [V-06]	node → L1
Settlement submitter (node)	Encodes applyBlock(uint256[],bytes) calldata and sends it with eth.sendTransaction from the manifest’s configured sender address; holds no L1 key [crates/node/src/settlement.rs:10-17][crates/node/src/settlement.rs:198-212]	node → L1
L1 caller of applyBlock	Any account; the function is external with no caller restriction (cross-reference [H-01]) [contracts/src/ShieldedPool.sol:144]	L1 → pool
Deployer	Sets the immutable verifier and the fee recipient (both required nonzero; no setter for either exists) and the genesis commitment and nullifier roots, a zero nullifier root selecting EMPTY_NULLIFIER_ROOT [contracts/src/ShieldedPool.sol:68][contracts/src/ShieldedPool.sol:89][contracts/src/ShieldedPool.sol:123-136]	deploy-time
Fee recipient	The only address that may call withdrawFees, which sends the contract balance [contracts/src/ShieldedPool.sol:181-189]; the contract has no payable function, so applyBlock credits no balance (cross-reference [I-04])	pool → EOA

Table 3: Actors and trust boundaries.

Permissionless settlement. applyBlock is declared **external** with no caller restriction: any account may submit a (**statement**, **proof**) pair, and beyond the verifier’s boolean the only checks are statement decoding and root continuity against stored state [contracts/src/ShieldedPool.sol:144-157]. The event carries only the block number, the four roots and the total fee [contracts/src/ShieldedPool.sol:92-99]. The soundness of the system therefore rests on what the verifier binds; the adequacy of this gate is examined in [H-01].

Verifier bound to the proof bundle, not the circuit. The injected verifier consumes a single caller-supplied bundle: a 16-byte header (magic WBND, a version byte, and the CONFIG and PROOF word counts, accepted versions 5, 7 and 8), then the CONFIG and PROOF sections and a STATEMENT section prefixed by its own u32 word count [contracts/src/verifier/WhirVerifier.sol:13-19][contracts/src/verifier/WhirVerifier.sol:219-257]. CONFIG holds the batch framing prefix (seed, degree bits, preprocessed digest, grind difficulties), each opening round’s schedule and framing labels [contracts/src/verifier/WhirVerifier.sol:23-25], and the constraint programs consumed by the constraint-identity check [contracts/src/verifier/WhirVerifier.sol:305-318]. The contract reads CONFIG from calldata and does not pin it; the source comment only recommends that a deployment pin $H_K(\text{configSection})$ [contracts/src/verifier/WhirVerifier.sol:21-25][contracts/src/verifier/WhirVerifier.sol:259-265]. Because the circuit description the verifier replays is taken from the same bundle a caller supplies, the binding between the on-chain verifier and the honest block circuit is not established on-chain; this is examined in [V-01].

Spec delta 1.2. The security-model prose states that the chain trusts “the operator set allowed to call applyBlock” [README.md:160-162]. The deployed contract contains no such set: applyBlock carries no access-control modifier and no operator mapping, so settlement is permissionless in the code as written [contracts/src/ShieldedPool.sol:144] (cross-reference [H-01], [I-02]). Likewise, the constructor comment describes a Keccak depth-256 empty nullifier root [contracts/src/ShieldedPool.sol:114-118], whereas the code substitutes the pinned depth-96 Poseidon2 constant [contracts/src/ShieldedPool.sol:79-86][contracts/src/ShieldedPool.sol:134-135]; and the verifier’s header comment names wire version 3 [contracts/src/

verifier/WhirVerifier.sol:13] while the code accepts only versions 5, 7 and 8 [contracts/src/verifier/WhirVerifier.sol:242].

2 Cryptographic primitives and encodings

This section states the field and extension arithmetic, the Poseidon2 commitment hashes, the byte/limb/digest encodings, the shielded-layer hashes, and the Keccak Fiat–Shamir transcript *as the deployed code computes them*. Where a source comment and the code’s own behaviour test disagree, the tested behaviour is taken as the specification and the discrepancy is recorded as a spec delta.

2.1 Base field

Definition 2.1 (KoalaBear prime). The base field is $\mathbb{F}_p$ with

$$p = 2^{31} - 2^{24} + 1 = 0x7f000001 = 2130706433.$$

[contracts/lib/sol-whir-p3/field/KoalaBear.sol:5] The Rust commitment layer pins the same value as KOALABEAR_P_U32 = 0x7F00_0001. [crates/pq-hash/src/poseidon.rs:79]

The Solidity field library treats its two additive operations as acting on *canonical* representatives in $[0, p)$ and does not re-reduce arbitrary accumulators [contracts/lib/sol-whir-p3/field/KoalaBear.sol:8-9]:

$$\text{add}(a, b) = \begin{cases} a + b - p & a + b \geq p \\ a + b & \text{otherwise} \end{cases} \quad [\text{KoalaBear.sol:10-15}] \quad (8)$$

$$\text{sub}(a, b) = \begin{cases} a - b & a \geq b \\ a + p - b & \text{otherwise} \end{cases} \quad [\text{KoalaBear.sol:19-23}] \quad (9)$$

$$\text{mul}(a, b) = a \cdot b \bmod p \quad (\text{mulmod}) \quad [\text{KoalaBear.sol:25-27}] \quad (10)$$

Multiplication uses the EVM `mulmod` and is therefore total; $\text{inv}(a) = a^{p-2}$ via square-and-multiply (`pow`), reverting when $a \equiv 0 \pmod{p}$. [KoalaBear.sol:29-47]

2.2 Quartic extension

Definition 2.2 (Extension field). The extension is

$$\mathbb{K} = \mathbb{F}_p[X]/(X^4 - W), \quad W = 3,$$

so the reduction relation is $X^4 = W = 3$. [contracts/lib/sol-whir-p3/field/KoalaBear.sol:6] [contracts/lib/sol-whir-p3/field/KoalaBearExt4.sol:6-7]

An element $a = c_0 + c_1X + c_2X^2 + c_3X^3 \in \mathbb{K}$ with canonical coefficients $c_i \in [0, p)$ is *packed* into a single 256-bit word in the top four 32-bit lanes, the low 128 bits being unused padding:

$$\text{pack}(a) = (c_0 \ll 224) \mid (c_1 \ll 192) \mid (c_2 \ll 160) \mid (c_3 \ll 128).$$

[KoalaBearExt4.sol:19-31] The multiplicative identity is `ONE = 1` ($\ll 224$ (i.e. $c_0 = 1$)). `unpack` takes c_0 as the top 32 bits and masks lanes c_1, c_2, c_3 with `COEFF_MASK = 0xffffffff`; no lane is reduced or range-checked.

[KoalaBearExt4.sol:12, 8, 26-31]

Extension multiplication is the W -twisted cyclic convolution under $X^4 = W$, each output lane reduced once mod p :

$$c_0 = a_0b_0 + W(a_1b_3 + a_2b_2 + a_3b_1) \quad (11)$$

$$c_1 = a_0b_1 + a_1b_0 + W(a_2b_3 + a_3b_2) \quad (12)$$

$$c_2 = a_0b_2 + a_1b_1 + a_2b_0 + W(a_3b_3) \quad (13)$$

$$c_3 = a_0b_3 + a_1b_2 + a_2b_1 + a_3b_0 \quad (14)$$

[KoalaBearExt4.sol:365-388] Two further constants are declared: `DTH_ROOT = 2113994754 =  $W^{(p-1)/4} \bmod p$` , a primitive fourth root of unity z giving the Frobenius map $\varphi(\sum_i c_i X^i) = \sum_i c_i z^i X^i$, and `INV_TWO = 1065353217 =  $(p+1)/2 = 2^{-1} \bmod p$` . [KoalaBearExt4.sol:14-15, 329-351] The named `INV_TWO` constant is not referenced; the quadratic extrapolation inlines the same value. [KoalaBearExt4.sol:423-439] Inversion computes $\rho = \varphi(a) \varphi^2(a) \varphi^3(a)$, the norm $N(a) = (a \cdot \rho)_0 = a_0 \rho_0 + W \sum_{i=1}^3 a_i \rho_{4-i} \in \mathbb{F}_p$, and returns $\rho \cdot N(a)^{-1}$; it reverts when the packed word is zero. [KoalaBearExt4.sol:96-109, 353-363]

Property 2.1 (Packed addition/subtraction assume canonical lanes). `KoalaBearExt4.add` performs a single 256-bit `add` of the two packed words and then extracts and reduces each lane; it is correct only when every lane sum stays below 2^{32} , which the code documents as the canonical-input bound “each lane sum $\in [0, 2M - 2]$ ” (here $2p - 2 < 2^{32}$). `sub` adds a per-lane bias p before subtracting under the same assumption. [KoalaBearExt4.sol:33-72] Proof-carried extension elements consumed by this arithmetic are not uniformly range-checked to canonical per-lane form, and the transcript absorb path reduces each limb modulo p (`observeExt4Mont`, [KeccakChallenger.sol:154-179]) whereas these packed `add/sub` routines propagate a carry across the 32-bit lane boundary on non-canonical input; see [M-04].

2.3 Montgomery encoding on the wire

Observed base words travel in Montgomery form while sampled challenges are raw canonical values. [contracts/lib/sol-whir-p3/PATCHES.md:84-87]

Definition 2.3 (Montgomery map).

$$\text{mont}(v) = v \cdot R \bmod p, \quad R = 2^{32} \bmod p = 33554430 = 0x01fffffe.$$

[contracts/src/verifier/WhirVerifier.sol:442-444]
[KeccakChallenger.sol:160]

[contracts/lib/sol-whir-p3/transcript/

The verifier checks a public statement coefficient against its wire word by comparing `mont(statementi)` (computed with `mulmod`) to the byte-swapped blob word; `statementi` itself is not range-checked to $[0, p)$, so the comparison holds modulo p ([M-03]). [WhirVerifier.sol:426-440] The transcript’s `observeExt4Mont` absorbs a packed $\mathbb{K}$ element as four words `mont(ci mod p)`, in the order c_0, c_1, c_2, c_3 , each in little-endian byte order; the low 128 bits of the packed word are ignored. [KeccakChallenger.sol:147-179]

2.4 The Poseidon2 permutation

Definition 2.4 (Permutation). π is `Poseidon2KoalaBear<16>`: the Plonky3 default KoalaBear Poseidon2 over $\mathbb{F}_p$ with width `WIDTH = 16`, x^3 S-box, and the audited Plonky3 constant set (`default_koalabear_poseidon2_16`). The sponge rate is `RATE = 8` and a digest is `DIGEST_ELEMS = 8` field elements. [crates/pq-hash/src/poseidon.rs:46-88]

Three hash functions are built from π ; the first two (sponge and compression) are byte-for-byte the WHIR MMCS leaf and node hashes. [poseidon.rs:3-40,56-66]

Definition 2.5 (Overwrite sponge H_{P2}). For input $x = (x_0, \dots, x_{n-1}) \in \mathbb{F}_p^*$, initialise $s := 0^{16}$. For each successive chunk c of x of length up to `RATE = 8` (the final chunk may be shorter), set $s[0..|c|) := c$ and then $s := \pi(s)$; output the first 8 state elements. This is `PaddingFreeSponge<_,16,8,8>` in overwrite mode. [poseidon.rs:60-62,196-206]

Spec delta 2.1 (Short final chunk is overwrite-only, not zero-padded). The module documentation states a “zero-padded final chunk” [poseidon.rs:33,197], but the enforced behaviour, pinned by the behaviour test, is pure overwrite: a short final chunk overwrites only the positions it fills and leaves the remaining rate lanes at the value the previous permutation left — the sponge does *not* zero the tail. [poseidon.rs:318-328] An empty input performs no permutation at all, so $H_{P2}(\varepsilon) = 0^8$ and the byte hash of the empty string is the all-zero digest. [poseidon.rs:331-338] The in-circuit sponge must reproduce both the zero initial state and these carried tail lanes; its enforcement of them is addressed in [V-07].

Definition 2.6 (Compression C_{P2}). For two digests l, r , $C_{P2}(l, r) = \pi(l \parallel r)[0..8)$: one permutation of the 16-element state $l \parallel r$, output the first 8 elements. This is `TruncatedPermutation<_,2,8,16>` and is not symmetric ($C_{P2}(l, r) \neq C_{P2}(r, l)$ in general). [poseidon.rs:64-66,208-216,289-307] The children here are decoded by the *lenient* word decoder (§2.5), so C_{P2} is a total function of its byte inputs: a non-canonical child word is reduced mod p rather than rejected. [poseidon.rs:124-133,212-216]

Definition 2.7 (Byte hash $H_{P2}(\cdot)$). Overloading H_{P2} on byte lists: for a list of byte parts, concatenate them, require an even total length (else panic), split into 16-bit limbs (Def. 2.8), sponge-absorb (Def. 2.5), and encode the 8-element output as a digest:

$$H_{P2}(\text{parts}) = \text{elements.to_digest}(H_{P2}(\text{limbs}(\text{concat parts}))).$$

Concatenating first means a part boundary never straddles a limb. [poseidon.rs:226-238,341-350]

Two pinned known-answer anchors fix the construction (C on two all-zero 32-byte digests):

```
H(b"pq-rollup/note-commit/v1")
= d806ed0043c329383dab56651706f32567961328cd3099435d362d5fc60b7c53
C(0^256, 0^256)
= 44917757aa9a1104a1ad4b7cbe60fe659316d4334d2ca2295e2525771877632b
```

[poseidon.rs:361-374]

2.5 Byte, limb and digest encodings

Definition 2.8 (Little-endian 16-bit limbs). For an even-length byte string b , the limb encoding emits one field element per pair of bytes:

$$\text{limbs}(b)_i = b_{2i} + 2^8 b_{2i+1} \in \mathbb{F}_p.$$

Odd length is a programmer error (panics). This is the `p3_circuit bytes_to_limbs` convention the circuit mirrors. [poseidon.rs:90-111,30-34]

Definition 2.9 (Digest serialization). A digest $d = (d_0, \dots, d_7) \in \mathbb{F}_p^8$ serializes to the 32-byte string $\text{LE32}(d_0) \parallel \dots \parallel \text{LE32}(d_7)$; `elements_to_digest` emits each element’s canonical `u32` little-endian, and the `Digest32` newtype carries these 32 bytes on the wire. [poseidon.rs:155-163] [crates/pq-hash/src/digest.rs:10-32]

Two decoders read a digest back to field elements, differing at trust boundaries:

Decoder	Non-canonical word ($\geq p$)	Use
<code>decode_words</code> (lenient)	reduced mod p via <code>Field::new</code>	inside C_{P2} ; total function
<code>digest_to_elements</code> (strict)	returns <code>None</code> (rejected)	wire trust boundary, anti-malleation

[poseidon.rs:124-153,275-286] The strict decoder rejects any 4-byte word $\geq \text{KOALABEAR_P_U32}$; a test pins that p , $p+1$, and `u32::MAX` are all rejected rather than silently reduced. [poseidon.rs:142-153,274-286]

Definition 2.10 (Digest limb export $L(d)$). For the in-circuit mirror, each digest element d_i with canonical value $w_i < p < 2^{31}$ is split into a low and a high limb,

$$L(d) = (w_0 \bmod 2^{16}, \lfloor w_0/2^{16} \rfloor, \dots, w_7 \bmod 2^{16}, \lfloor w_7/2^{16} \rfloor),$$

i.e. per base coefficient the low 16 bits then the high 15 bits — the little-endian 16-bit limbs of the digest’s 32 bytes, 16 limbs in all. [crates/prover/src/block.rs:292-305] [crates/prover/src/commitment_gadget.rs:401-406,417-430] In-circuit, each coefficient is constrained only through a 31-bit decomposition, so for $w_i < 2^{31} - p = 2^{24} - 1$ the representative $w_i + p$ is also a valid witness; the source comment relies on comparison of the exported limbs with the contract’s stored root bytes to fix the canonical value. [crates/prover/src/commitment_gadget.rs:408-412,425-427]

2.6 Shielded-layer hashes

The shielded layer defines two domain-separated hashes over byte strings, each returning a 32-byte digest.

Definition 2.11 (SHA3-256 shielded hash). For parts $p_1, \dots, p_k$,

$$\text{hash_to_digest}(\text{dom}, \text{parts}) = \text{SHA3-256}(\text{LE64}(|\text{dom}|) \parallel \text{dom} \parallel \text{LE64}(|p_1|) \parallel p_1 \parallel \dots \parallel \text{LE64}(|p_k|) \parallel p_k).$$

Every length prefix is a little-endian `u64`, making the framing unambiguous ($((\mathbf{a}, \mathbf{c}) \neq (\mathbf{a}, \mathbf{bc}))$). [crates/pq-hash/src/shielded.rs:13-34,64-70]

Definition 2.12 (Poseidon2 shielded hash).

$$\text{hash_to_digest}(\text{dom}, \text{parts}) = H_{P2}(\text{dom} \parallel p_1 \parallel p_2 \parallel \dots),$$

i.e. the domain tag is prepended as an extra part and the whole is run through the commitment byte hash H_{P2} of Def. 2.7, with *no* length prefixes (every consensus preimage here has fixed-length parts). [crates/pq-hash/src/shielded.rs:87-105] This trades SHA3’s margin for the Poseidon2 sponge collision bound, a documented operator decision. The node instantiates this variant. [crates/pq-hash/src/shielded.rs:83-85] [crates/node/src/main.rs:58,301]

2.7 Keccak Fiat–Shamir transcript

The settlement verifier replays the WHIR transcript natively with a `HashChallenger`-style Keccak sponge. Its state is `(inputBuffer, inputLen, outputBlock, outputIndex)`. `[contracts/lib/sol-whir-p3/transcript/KeccakChallenger.sol:10–15]` A `flush` hashes the first `inputLen` bytes of the input buffer with `keccak256`, replaces the input by that 32-byte digest (`inputLen := 32`), sets `outputBlock := digest` and `outputIndex := 32`; subsequent input is appended after that digest, so the sponge chains. `[KeccakChallenger.sol:1215–1231]`

Definition 2.13 (Observe). Every observe entry point clears the squeeze cursor (`outputIndex := 0`), so a sample after an observe re-flushes rather than resuming a stale block. `observeBase` range-checks $v < p$ and appends the four bytes of v little-endian; `observeBytes` appends raw bytes. `[KeccakChallenger.sol:17–41, 1277–1290]`

Definition 2.14 (Sample). `_sampleUint32` flushes if the cursor $\iota = \text{outputIndex}$ is 0 and then consumes four bytes from the current 32-byte `outputBlock` $\delta = (\delta_0, \dots, \delta_{31})$ from its *end*: it returns

$$u = \sum_{j=0}^3 2^{8j} \delta_{\iota-1-j}, \quad \iota := \iota - 4,$$

so on a fresh block the byte stream is $\delta_{31}, \delta_{30}, \dots$ and the low byte of the first `uint32` is δ_{31} . `[KeccakChallenger.sol:1233–1243, 1093–1116]` The decrement is unchecked and there is no $\iota \geq 4$ check; in the verifier `sampleByte` is reached only through `checkWitness`, whose following `observeBase` resets ι . `[KeccakChallenger.sol:1238–1241, 1201–1205]` The derived samplers are:

Operation	Enforced behaviour	Source
<code>sampleByte</code> / <code>sampleBytes</code>	one byte $\delta_{\iota-1}$ (then $\iota := \iota - 1$) / loop; same end-of-block order	<code>[KeccakChallenger.sol:1105–1124]</code>
<code>sampleBase</code>	reject-sample <code>_sampleUint32 & 0x7fffffff</code> until $< p$	<code>[KeccakChallenger.sol:1126–1135]</code>
<code>sampleBits(b)</code>	require $b < 256$; $b=0 \Rightarrow 0$ without squeezing; else require $2^b \leq p$ and return low b bits of <code>_sampleUint32</code>	<code>[KeccakChallenger.sol:1137–1154]</code>
<code>sampleExt4/5/8Coeffs</code>	4/5/8 independent <code>sampleBase</code> draws (canonical coeffs)	<code>[KeccakChallenger.sol:1156–1178]</code>
<code>checkWitness(b,w)</code> (grind)	$b=0 \Rightarrow$ accept without touching the transcript; else <code>sampleByte</code> (discarded), <code>observeBase(w)</code> (requires $w < p$), accept iff <code>sampleBits(b)=0</code>	<code>[KeccakChallenger.sol:1192–1206]</code>

The sample mask is `KOALABEAR_SAMPLE_MASK = 0x7fffffff` (31 bits), so each `sampleBase` draw is a 31-bit value retried until it falls below p . `[KeccakChallenger.sol:6, 1126–1135]` In the grinding check the leading `sampleByte` is load-bearing: when the cursor is empty (as it is in the WHIR flow, which has just observed a commitment) it flushes the accumulated observations into a digest before the witness is appended, so the proof-of-work bits are measured over the correct chaining digest. `[KeccakChallenger.sol:1179–1206]`

Spec delta 2.2 (Vendored challenger patches re-applied). The transcript diverges from upstream `sol-whir-p3` by two documented patches: an additive one, `sampleByte/sampleBytes` for byte-level replay (end-of-block order, matching p3’s pop-from-end), and a correctness patch that modifies two existing functions — `observeBase` now clears `outputIndex` (so an observe-after-sample no longer desynchronises later samples) and `checkWitness` now performs the discarded squeeze before absorbing the witness; an upstream refresh must re-apply the latter. A 3,551-operation recorded transcript (224 field samples, 779 bit draws, 23 proof-of-work checks) pins the result. `[contracts/lib/sol-whir-p3/PATCHES.md:5–10, 12–41, 42–69, 71–76]`

Sampled challenges are canonical field values whereas observed field *inputs* are Montgomery wire bytes; both forms are intentional and are what a faithful verifier must reproduce. `[contracts/lib/sol-whir-p3/PATCHES.md:82–89]`

3 Shielded state: keys, notes, trees

This section fixes the objects that make up the shielded state: the spend key pair, the note and its two derived digests (commitment and nullifier), the append-only note-commitment tree, and the sparse nullifier

tree. For each object it gives the native definition (the `shielded` crate, which describes itself as the executable specification of the rules the STARK proves [crates/shielded/src/lib.rs:6-9]) and, where the circuit re-derives the object, the in-circuit relation actually enforced. Notation follows Section 2: $H_{P2}(\cdot)$ is the Poseidon2 byte hash over little-endian 16-bit limbs, C_{P2} is the Poseidon2 two-to-one compression, $\parallel$ is byte concatenation, and a digest is 8 canonical $\mathbb{F}_p$ -elements $d_0, \dots, d_7$ serialized as $\text{LE32}(d_0) \parallel \dots \parallel \text{LE32}(d_7)$ [crates/pq-hash/src/poseidon.rs:155-163].

3.1 Hash instantiation in the shielded layer

The `shielded` crate is generic over two unrelated traits, `ShieldedHasher` (spend key, nullifier) and `CommitmentHasher` (note commitment, tree nodes) [crates/pq-hash/src/traits.rs:19-44]. Every production path instantiates both with Poseidon2:

Object	Trait	Instance	Where instantiated
pk_d	<code>ShieldedHasher</code>	<code>Poseidon2Shielded</code>	[crates/wallet-wasm/src/lib.rs:347] [crates/prover/src/fixtures.rs:108]
nf	<code>ShieldedHasher</code>	<code>Poseidon2Shielded</code>	[crates/prover/src/client.rs:204] [crates/wallet-wasm/src/lib.rs:536]
cm	<code>CommitmentHasher</code>	<code>Poseidon2Commitment</code>	[crates/prover/src/client.rs:147,155]
tree node	<code>hash_pair</code>	<code>Poseidon2Commitment</code>	[crates/prover/src/client.rs:122]
SMT node	<code>hash_pair</code>	<code>Poseidon2Commitment</code>	[crates/prover/src/client.rs:123,199]

`Poseidon2Commitment` implements `hash` as the byte hash H_{P2} and overrides `hash_pair` with C_{P2} [crates/pq-hash/src/poseidon.rs:225-243]. `Poseidon2Shielded` prepends the domain tag as an extra part and calls the same `Poseidon2Commitment::hash` [crates/pq-hash/src/shielded.rs:96-105]. Consequently

$$\text{Poseidon2Shielded.hash_to_digest}(\text{dom}, (p_1, \dots, p_k)) = H_{P2}(\text{dom} \parallel p_1 \parallel \dots \parallel p_k), \quad (15)$$

with no length prefixes [crates/pq-hash/src/shielded.rs:87-94]: the spend-key, nullifier and commitment digests are all the *same* function H_{P2} applied to different byte strings, and their separation rests on the distinct domain tags together with the fixed preimage lengths of Table 4.

Spec delta 3.1 (Stale hash names in comments). Several comments still name SHA3-256 or Keccak-256 for these objects: the spend-key derivation (“SHA3-256 preimage knowledge” [crates/shielded/src/keys.rs:36-38]), the commitment (“Keccak-256” [crates/shielded/src/note.rs:104-106]), the nullifier (“SHA3-256” [crates/shielded/src/note.rs:118-120]), the trait contracts [crates/pq-hash/src/traits.rs:10-12, 24-32], and the nullifier tree (“keeps Keccak”) [crates/shielded/src/tree.rs:30-31] [crates/prover/src/commitment_gadget.rs:15-16] [crates/pq-hash/src/poseidon.rs:19-22]. The enforced instantiation is Poseidon2 throughout, as tabulated above and as the circuit itself states [crates/prover/src/transfer.rs:30-43] [crates/prover/src/nullifier_gadget.rs:12-18]. The SHA3/Keccak instances appear only in unit tests of the `shielded` crate [crates/shielded/src/note.rs:185] [crates/shielded/src/nullifier_tree.rs:364]. See [I-02].

3.2 Keys

Definition 3.1 (Spend secret). The spend secret is a byte string $\text{sk}_d \in \{0, 1\}^{256}$. The `shielded` crate has no type for it and only receives it as a borrowed `[u8; 32]` [crates/shielded/src/keys.rs:11-16]. The wallet draws it as 32 bytes from an RNG seeded by caller entropy [crates/wallet-wasm/src/lib.rs:300-303].

Definition 3.2 (Spend public key). With `DOMAIN_PK = "pq-rollup/spendpk/v1"` (20 bytes) [crates/shielded/src/keys.rs:29],

$$\text{pk}_d = H_{P2}(\text{DOMAIN_PK} \parallel \text{sk}_d) \in \{0, 1\}^{256}. \quad (16)$$

[crates/shielded/src/keys.rs:40-42] The preimage is 52 bytes, i.e. 26 limbs, absorbed in rate-8 chunks of sizes 8, 8, 8, 2 (a short final chunk; cf. Spec delta 2.1). `SpendPublicKey` is an opaque 32-byte newtype [crates/shielded/src/keys.rs:49-64].

The tag length is even by design so that the preimage is limb-aligned [crates/shielded/src/keys.rs:22-28]; the byte hash asserts an even total length [crates/pq-hash/src/poseidon.rs:231-235].

Spec delta 3.2 (pk_d is not a signature verifying key). The module documentation describes `SpendPublicKey` as “the SPHINCS+ verifying key, published in the note preimage” [crates/shielded/src/keys.rs:5-7]. The enforced pk_d is the Poseidon2 digest of Definition 3.2; the wallet generates the SPHINCS+ key pair independently of sk_d from the same RNG stream [crates/wallet-wasm/src/lib.rs:300-305] and stores both secrets side by side [crates/wallet-wasm/src/lib.rs:309-312]. No relation between pk_d and the SPHINCS+ verifying key is computed; see [M-02].

Definition 3.3 (Incoming viewing key). `IncomingViewingKey` is an opaque 32-byte newtype with a constructor and an accessor only [crates/shielded/src/keys.rs:80-95].

Spec delta 3.3 (No viewing-key derivation). The documentation states $\text{ivk} = H(\text{DOMAIN_IVK} \parallel \text{sk}_d)$ [crates/shielded/src/keys.rs:77-79], but no `DOMAIN_IVK` constant and no derivation function exist in the workspace; the type is only constructed from raw bytes [crates/shielded/src/keys.rs:85-87].

3.3 Notes, commitments and nullifiers

Definition 3.4 (Note). A note is a tuple

$$n = (v, \rho, \psi, \text{pk}_d), \quad v \in [0, 2^{64}), \quad \rho, \psi \in \{0, 1\}^{256}, \quad \text{pk}_d \in \{0, 1\}^{256},$$

stored as `u64`, `[u8;32]`, `[u8;32]`, `SpendPublicKey` [crates/shielded/src/note.rs:48-58]. `Note::new` performs no validation [crates/shielded/src/note.rs:66-74].

The value domain is restricted elsewhere: natively, `check_balance` rejects any input, output or fee value above `MAX_VALUE` = $2^{62} - 1$ [crates/shielded/src/transfer.rs:45,77-84]; in-circuit, each amount is four private 16-bit limbs with the top limb range-checked to 14 bits, so $v < 2^{62}$ [crates/prover/src/transfer.rs:111-125,157-172].

Definition 3.5 (Note commitment). With `DOMAIN_NOTE` = “pq-rollup/note-commit/v1” (24 bytes) [crates/shielded/src/note.rs:38],

$$\text{cm}(n) = H_{P2}(\text{DOMAIN_NOTE} \parallel \text{LE64}(v) \parallel \rho \parallel \psi \parallel \text{pk}_d). \quad (17)$$

[crates/shielded/src/note.rs:108-112]

Definition 3.6 (Nullifier). With `DOMAIN_NULLIFIER` = “pq-rollup/nullifier/v1” (22 bytes) [crates/shielded/src/note.rs:41],

$$\text{nf}(n, \text{sk}_d) = H_{P2}(\text{DOMAIN_NULLIFIER} \parallel \text{sk}_d \parallel \rho). \quad (18)$$

[crates/shielded/src/note.rs:125-127]

Digest	Byte layout (offset:length)	Bytes	Limbs	Rate chunks
pk_d	<code>DOMAIN_PK</code> 0:20, sk_d 20:32	52	26	8,8,8,2
nf	<code>DOMAIN_NULLIFIER</code> 0:22, sk_d 22:32, ρ 54:32	86	43	8,8,8,8,8,3
cm	<code>DOMAIN_NOTE</code> 0:24, $\text{LE64}(v)$ 24:8, ρ 32:32, ψ 64:32, pk_d 96:32	128	64	8×8

Table 4: Preimage layouts of the three shielded digests. Limbs are the little-endian 16-bit limbs of the concatenation (Def. 2.8). [crates/shielded/src/keys.rs:29,41] [crates/shielded/src/note.rs:38,41,109-111,126]

Property 3.1 (What the nullifier depends on). nf is a function of (sk_d, ρ) only: it does not depend on v , ψ , pk_d or the note’s position in the commitment tree [crates/shielded/src/note.rs:125-127]. Two notes with the same (sk_d, ρ) therefore share a nullifier; for output notes ρ is a free private input, bound into cm but otherwise constrained in-circuit only by its 16-bit limb range checks (Table 5). See [M-08].

In-circuit re-derivation. The transfer circuit recomputes all three digests with the in-circuit sponge `p2_sponge_limbs` [crates/prover/src/commitment_gadget.rs:306-367], prepending the domain tag as constant limbs exactly as Equation (15) (`p2_framed`) [crates/prover/src/transfer.rs:254-264]. Each private byte string is allocated as 16-bit limbs, each range-checked to 16 bits [crates/prover/src/transfer.rs:185-207]. For an input note, pk_d is *derived* from sk_d and fed into the commitment preimage as its exported limbs $L(\text{pk}_d)$ (Section 2.5) [crates/prover/src/transfer.rs:423-435]; for an output note pk_d is a circuit constant [crates/prover/src/transfer.rs:573-580]. Table 5 summarises the roles. Whether the in-circuit sponge is a function of its input on the shared permutation table (chain-start capacity and partial-chunk carries) is the subject of [V-07].

Field	Input note	Output note	Source
sk_d	private, 16 limbs	—	[crates/prover/src/transfer.rs:414]
ρ	private, 16 limbs	private, 16 limbs	[crates/prover/src/transfer.rs:415,566]
ψ	private, 16 limbs	private, 16 limbs	[crates/prover/src/transfer.rs:416,567]
v	private, 4 limbs, $< 2^{62}$	private, 4 limbs, $< 2^{62}$	[crates/prover/src/transfer.rs:417,568]
pk_d	$H_{P2}(\text{DOMAIN_PK} sk_d)$ in-circuit	constant	[crates/prover/src/transfer.rs:425,579]
cm	in-circuit, leaf of membership fold	in-circuit, exported, appended	[crates/prover/src/transfer.rs:435,580-581]
nf	in-circuit, exported, SMT address	—	[crates/prover/src/transfer.rs:456,471]

Table 5: Per-note roles in the transfer circuit.

3.4 Note-commitment tree

Definition 3.7 (Shape and empty subtrees). The commitment tree is a binary Merkle tree of fixed depth $D = \text{DEPTH} = 32$ (capacity 2^{32} leaves) [crates/shielded/src/tree.rs:39]. A leaf is the 32-byte commitment cm viewed as a digest [crates/shielded/src/tree.rs:46-48]. The empty-subtree digests are

$$E_0 = 0^{256} \text{ (all-zero digest), } \quad E_h = C_{P2}(E_{h-1}, E_{h-1}) \quad (1 \leq h \leq D), \quad (19)$$

[crates/shielded/src/tree.rs:65-73] and the empty-tree root is E_D [crates/shielded/src/tree.rs:85-87]. The empty leaf E_0 is the same byte string as an all-zero leaf digest.

Definition 3.8 (Root). For leaves $\ell_0, \dots, \ell_{n-1}$ appended in order ($n \leq 2^D$), define for a leaf range $[s, e)$ at height h

$$N(s, e, h) = \begin{cases} E_h & s \geq e, \\ \ell_s & h = 0, \\ C_{P2}(N(s, m, h-1), N(m, e, h-1)), & m = \min(s + 2^{h-1}, e) \text{ otherwise,} \end{cases} \quad (20)$$

[crates/shielded/src/tree.rs:223-235] and $\text{root} = N(0, n, D)$ [crates/shielded/src/tree.rs:214-216]. **append** assigns index n and panics (assertion) when $n = 2^D$ [crates/shielded/src/tree.rs:202-210].

Definition 3.9 (Membership path and fold). A path is a sibling list $(\sigma_0, \dots, \sigma_{D-1})$, leaf-to-root, and an index i [crates/shielded/src/tree.rs:99-105]. Natively the path is rejected if $|\sigma| \neq D$ or $i \geq 2^D$; otherwise, with b_h the bit h of i ,

$$c_0 = \ell, \quad c_{h+1} = \begin{cases} C_{P2}(\sigma_h, c_h) & b_h = 1, \\ C_{P2}(c_h, \sigma_h) & b_h = 0, \end{cases} \quad \text{root} = c_D. \quad (21)$$

[crates/shielded/src/tree.rs:127-150]

In-circuit, the input leaf cm is folded by the same recurrence (**fold_membership_p2**): each b_h is a private input constrained boolean, each σ_h enters as a circuit constant, and the side is chosen with **select** [crates/prover/src/transfer.rs:716-757]. The final digest is constrained equal, by arithmetic difference, to the constant root taken from the public statement [crates/prover/src/transfer.rs:405-410,442-447]. The in-circuit fold iterates over the supplied sibling list without checking $|\sigma| = D$ ([**L-05**]), and the siblings being circuit constants is the subject of [**H-03**].

Definition 3.10 (Frontier). The append witness is a frontier $(f_0, \dots, f_{D-1}; b_0, \dots, b_{D-1})$ with $b_h \in \{0, 1\}$ and implied count $n = \sum_h b_h 2^h$ [crates/prover/src/commitment_gadget.rs:144-174]. In the honest frontier built by replaying appends, for $b_h = 1$ the slot f_h holds the root of the complete 2^h -leaf block of leaves

$[s_h, s_h + 2^h)$ with $s_h = \sum_{j>h} b_j 2^j$ [crates/prover/src/commitment_gadget.rs:214-252]; the type comment’s “ending at the last leaf” holds only for the lowest set bit [crates/prover/src/commitment_gadget.rs:146-147]. Slots with $b_h = 0$ are unused; the comment says they hold the zero digest, but after a carry the old digest is left in place [crates/prover/src/commitment_gadget.rs:148,225-229]. Its root is

$$c \leftarrow E_0; \quad \text{for } h = 0, \dots, D-1 : \quad c \leftarrow \begin{cases} C_{P2}(f_h, c) & b_h = 1, \\ C_{P2}(c, E_h) & b_h = 0, \end{cases} \quad \text{Fold}(f, b) = c. \quad (22)$$

[crates/prover/src/commitment_gadget.rs:191-204] Native append of a leaf λ carries upward: $g \leftarrow \lambda$; for $h = 0, 1, \dots$: if $b_h = 1$ then $g \leftarrow C_{P2}(f_h, g)$, $b_h \leftarrow 0$; else $f_h \leftarrow g$, $b_h \leftarrow 1$ and stop; failing to stop means the tree is full [crates/prover/src/commitment_gadget.rs:214-237].

Definition 3.11 (In-circuit append relation). **constrain_append** takes (f, b) as private inputs (2 extension elements per slot, then D bits, each constrained boolean) [crates/prover/src/commitment_gadget.rs:489-506] and enforces, for a leaf λ and a pinned R_{before} :

$$\text{Fold}(f, b) = R_{\text{before}} \quad [\text{commitment_gadget.rs:508-517}] \quad (23)$$

$$g_{-1} = \lambda, \quad g_h = b_h ? C_{P2}(f_h, g_{h-1}) : g_{h-1} \quad [\text{commitment_gadget.rs:519-530}] \quad (24)$$

$$\kappa_0 = 1, \quad b'_h = b_h + \kappa_h - 2b_h\kappa_h, \quad f'_h = \kappa_h(1 - b_h) ? g_h : f_h, \quad \kappa_{h+1} = \kappa_h b_h \quad [\text{commitment_gadget.rs:532-552}] \quad (25)$$

$$\kappa_D = 0 \quad [\text{commitment_gadget.rs:553-554}] \quad (26)$$

$$R_{\text{after}} = \text{Fold}(f', b') \quad [\text{commitment_gadget.rs:556-557}] \quad (27)$$

The empty-subtree constants are derived at circuit-build time from **Poseidon2Commitment** and enter as constants [crates/prover/src/commitment_gadget.rs:117-133,577-585] [crates/prover/src/transfer.rs:557].

Within one transfer the appends chain: the first R_{before} is the public root as a constant, output i ’s R_{after} is output $i+1$ ’s R_{before} , and the last R_{after} is exported and pinned to the public **root_after** [crates/prover/src/transfer.rs:548-598,494-504]. The frontier witness for output i is computed natively by replaying the first i outputs [crates/prover/src/transfer.rs:610-626]. R_{before} constrains (f, b) only through Equation (22); the root does not separately commit to the count n , and E_0 coincides with the all-zero digest, so b is not uniquely determined by R_{before} . The resulting freedom in the append position is [H-02].

Property 3.2 (On-chain treatment). The settlement contract never folds the commitment tree. It stores **currentRoot**, requires the block’s **rootBefore** to equal it, and then stores the proof-attested **rootAfter** [contracts/src/ShieldedPool.sol:9-23,73-74,152-154,165]. The genesis root is a constructor argument [contracts/src/ShieldedPool.sol:108-112,126,133].

3.5 Nullifier sparse Merkle tree

Definition 3.12 (Shape, address, leaf, empty constants). The nullifier map is a sparse binary Merkle tree of depth $D_{\text{nf}} = \text{NULLIFIER_TREE_DEPTH} = 96$ [crates/shielded/src/nullifier_tree.rs:64]. For a nullifier digest $\text{nf} \in \{0, 1\}^{256}$ with bytes $\text{nf}[0..32]$, the address is

$$a(\text{nf}) = (a_0, \dots, a_{95}), \quad a_i = \lfloor \text{nf}[\lfloor i/8 \rfloor] / 2^{i \bmod 8} \rfloor \bmod 2, \quad (28)$$

i.e. the low 96 bits of the byte string, little-endian within each byte [crates/shielded/src/nullifier_tree.rs:66-70]. The leaf value stored at $a(\text{nf})$ is the full digest nf [crates/shielded/src/nullifier_tree.rs:200-204,307-318]. The empty constants follow the same recurrence and hasher as the commitment tree, $E_0 = 0^{256}$, $E_h = C_{P2}(E_{h-1}, E_{h-1})$ for $1 \leq h \leq 96$ [crates/shielded/src/nullifier_tree.rs:128-140], so the two trees share $E_0, \dots, E_{32}$, and the empty-map root is E_{96} [crates/shielded/src/nullifier_tree.rs:187-189].

Since bytes 0..11 of a digest are $\text{LE32}(d_0) \parallel \text{LE32}(d_1) \parallel \text{LE32}(d_2)$, the address bits are $a_{32k+j} = \text{bit } j \text{ of } d_k$ for $k \in \{0, 1, 2\}$, $0 \leq j < 32$. Every digest emitted by H_{P2} has canonical elements $d_k < p < 2^{31}$ [crates/pq-hash/src/poseidon.rs:155-163], so $a_{31} = a_{63} = a_{95} = 0$ for every honestly derived nullifier.

Definition 3.13 (Native root). For the spent set S (32-byte strings) and $A \subseteq S$,

$$T(h, A) = \begin{cases} E_h & A = \emptyset, \\ \text{first element of } A & h = 0, \\ C_{P2}(T(h-1, A_0), T(h-1, A_1)), \quad A_\beta = \{x \in A : a_{h-1}(x) = \beta\} & \text{otherwise,} \end{cases} \quad (29)$$

[crates/shielded/src/nullifier_tree.rs:196-220] and $\text{root}_{\text{nf}} = T(96, S)$ [crates/shielded/src/nullifier_tree.rs:180-183].

Spec delta 3.4 (Address coincidences in the native map). The depth comment states that two nullifiers agreeing on the 96 address bits “are treated as duplicates (the insert refuses)” [crates/shielded/src/nullifier_tree.rs:57-63]. The enforced behaviour differs: the spent set is a `HashSet` keyed on all 32 bytes [crates/shielded/src/nullifier_tree.rs:122,172-176], so `insert` accepts a second nullifier with an equal address; at height 0, T returns the first element of A in the set’s iteration order [crates/shielded/src/nullifier_tree.rs:181,200-204]; and `highest_differing_bit` returns `None` for such a pair, so the witness builder does not bucket it [crates/shielded/src/nullifier_tree.rs:78-92,265-272]. See [M-09]. The module header still describes full 256-bit addressing [crates/shielded/src/nullifier_tree.rs:13-22,45-49]; see [I-02].

Definition 3.14 (Fold and native witness). For a start height s , start node x , siblings $(\sigma_s, \dots, \sigma_{95})$ and address a ,

$$\text{Fold}_s(x; \sigma; a) : \quad c_s = x, \quad c_{h+1} = \begin{cases} C_{P2}(\sigma_h, c_h) & a_h = 1, \\ C_{P2}(c_h, \sigma_h) & a_h = 0, \end{cases} \quad \text{output } c_{96}. \quad (30)$$

[crates/shielded/src/nullifier_tree.rs:223-241] For an unspent nf , the witness has start height $h^* = \min_{x \in S} \text{hdb}(a(\text{nf}), a(x))$ (or 96 if no spent address differs), where `hdb` is the highest differing address bit, and siblings $\sigma_h = T(h, B_h)$ for $h^* \leq h < 96$, with B_h the spent addresses whose highest differing bit is h [crates/shielded/src/nullifier_tree.rs:250-283]. Natively,

$$R_{\text{before}} = \text{Fold}_{h^*}(E_{h^*}; \sigma; a(\text{nf})) \quad [\text{nullifier_tree.rs:292-299}] \quad (31)$$

$$R_{\text{after}} = \text{Fold}_0(\text{nf}; (E_0, \dots, E_{h^*-1}, \sigma_{h^*}, \dots, \sigma_{95}); a(\text{nf})) \quad [\text{nullifier_tree.rs:307-318}] \quad (32)$$

The standalone checker additionally requires $h^* \leq 96$ and $|\sigma| = 96 - h^*$ and compares the folded root bitwise [crates/shielded/src/nullifier_tree.rs:328-359].

Definition 3.15 (In-circuit absence and insertion). The circuit fixes `FOLD_DEPTH` = 32 and `FOLD_START` = $96 - 32 = 64$ [crates/prover/src/nullifier_gadget.rs:71-74]. The native witness is padded to heights 64..95 by inserting E_h for $64 \leq h < h^*$, and proving is refused when $h^* < 64$ [crates/prover/src/nullifier_gadget.rs:128-155]. With nf the in-circuit nullifier digest, the address bits are the 16-bit decompositions of its first 6 exported limbs $L(\text{nf})_0, \dots, L(\text{nf})_5$ [crates/prover/src/nullifier_gadget.rs:76-78,166-182], and the gadget enforces

$$R_{\text{before}} = \text{Fold}_{64}(E_{64}; (\sigma_{64}, \dots, \sigma_{95}); a(\text{nf})) \quad [\text{nullifier_gadget.rs:271-276}] \quad (33)$$

$$R_{\text{after}} = \text{Fold}_0(\text{nf}; (E_0, \dots, E_{63}, \sigma_{64}, \dots, \sigma_{95}); a(\text{nf})) \quad [\text{nullifier_gadget.rs:285-288}] \quad (34)$$

where every σ_h and E_h enters as a circuit constant built from the prover-side witness (`const_digest`) [crates/prover/src/nullifier_gadget.rs:191-197,203-231]. The 31-bit decomposition behind $L(\cdot)$ is constrained canonical [crates/prover/src/commitment_gadget.rs:417-431] [vendor/p3-recursion/circuit/src/builder/circuit_builder.rs:1435-1449], and odd-indexed limbs are 15-bit high halves, so $a(\text{nf})$ is uniquely determined by the digest and $a_{31} = a_{63} = a_{95} = 0$ in-circuit as well.

Across the spends of one transfer the root is threaded: the first R_{before} is the public `nullifier_roots.before` as a constant, each spend’s R_{before} is constrained equal to the running root, the running root becomes that spend’s R_{after} , and the final running root is constrained equal to the public `nullifier_roots.after` [crates/prover/src/transfer.rs:390-398,456-470,513-525].

Property 3.3 (What in-circuit absence certifies). Because the absence fold starts from the constant E_{64} at height 64, a satisfying assignment asserts (under collision resistance of C_{P2}) that the height-64 node on the path selected by $a_{64}, \dots, a_{95}$ is empty under R_{before} , i.e. that no leaf has an address agreeing with $a(\text{nf})$ on bits 64 through 95; the low bits $a_0, \dots, a_{63}$ enter only the insertion fold, which places nf as the sole leaf of that height-64 subtree. Bits $a_{64..95}$ are the bits of the third digest element d_2 , so the in-circuit absence predicate separates nullifiers by d_2 alone. The spendability consequences of this, together with *Spec delta 3.4*, are [M-09].

Property 3.4 (On-chain treatment). The contract holds no nullifier set. It stores `currentNullifierRoot`, requires the block’s `nullifierBefore` to equal it, and stores the attested `nullifierAfter` [contracts/src/ShieldedPool.sol:76-77,155-157,166]. A zero genesis argument selects the pinned constant [contracts/src/ShieldedPool.sol:79-86,133-135]

```
EMPTY_NULLIFIER_ROOT =
  0x83265b7a2c40cc6eca237b6016b764606335cd0c6ff0732470e20a484d5a7329
```

documented as the Poseidon2 E_{96} . It equals the E_{96} of Definition 3.12 under `Poseidon2Commitment`, i.e. the `empty_root` of an empty `NullifierMap<Poseidon2Commitment>` [crates/shielded/src/nullifier_tree.rs:128-140,187-189] (checked by recomputation with the workspace `crates`); the golden-vector generator derives `nullifier_before_hex` from such an empty map and the vector file carries the same bytes [crates/prover/tests/golden_vectors.rs:413,495] [contracts/test/vectors/block_vectors.json:4]. The constructor comment still describes a depth-256 Keccak empty root [contracts/src/ShieldedPool.sol:114-118] ([I-02]).

3.6 Constants

Name	Value	Source
DOMAIN_PK	"pq-rollup/spendpk/v1" (20 B, 10 limbs)	[crates/shielded/src/keys.rs:29]
DOMAIN_NOTE	"pq-rollup/note-commit/v1" (24 B, 12 limbs)	[crates/shielded/src/note.rs:38]
DOMAIN_NULLIFIER	"pq-rollup/nullifier/v1" (22 B, 11 limbs)	[crates/shielded/src/note.rs:41]
MAX_VALUE	$2^{62} - 1$	[crates/shielded/src/transfer.rs:45]
DEPTH (D)	32	[crates/shielded/src/tree.rs:39]
NULLIFIER_TREE_DEPTH	96	[crates/shielded/src/nullifier_tree.rs:64]
FOLD_DEPTH / FOLD_START	32 / 64	[crates/prover/src/nullifier_gadget.rs:71-74]
E_0 (both trees)	0^{256}	[crates/shielded/src/tree.rs:67] [crates/shielded/src/nullifier_tree.rs:130]

4 The transfer relation

This section states the relation R_{transfer} that the client transfer circuit enforces for one shielded transfer with k inputs (spends) and m outputs. The whole relation is laid into a builder by `constrain_transfer` [crates/prover/src/transfer.rs:358-537]; the stand-alone client circuit wraps it with the Poseidon2 permutation and recompose tables enabled, exports the returned expressions as the statement, and witnesses it [crates/prover/src/transfer.rs:645-701]. The function’s documentation describes reuse inside a larger (block) builder [crates/prover/src/transfer.rs:328-334], but in the current source its only caller is `build_transfer_circuit` [crates/prover/src/transfer.rs:664]; the block circuit instead re-verifies client transfer proofs recursively and reads their statements [crates/prover/src/block.rs:400-419]. Everything below is read from the constraints the code emits; the module-level description of the relation [crates/prover/src/transfer.rs:1-62] is used only where the code agrees with it. Hash and encoding notation (H_{P2} , C_{P2} , limbs, L) is that of §2.

4.1 Shape parameters and domain tags

Constant	Value	Role and source
VALUE_LIMBS	4	16-bit limbs per amount [crates/prover/src/transfer.rs:113]
LIMB_BITS	16	range of a secret limb and of amount limbs 0..2 [crates/prover/src/transfer.rs:116]
TOP_LIMB_BITS	14	range of amount limb 3, so an amount lies in $[0, 2^{62})$ [crates/prover/src/transfer.rs:123]
CARRY_BITS	8	range of an interior balance carry [crates/prover/src/transfer.rs:135]
MAX_PARTIES	16	cap on $\max(k, m + 1)$ [crates/prover/src/transfer.rs:141]
DEPTH	32	commitment-tree depth (frontier slots per append) [crates/shielded/src/tree.rs:39]
NULLIFIER_TREE_DEPTH	96	nullifier-map depth (address bits) [crates/shielded/src/nullifier_tree.rs:64]
FOLD_DEPTH	32	levels covered by the absence fold [crates/prover/src/nullifier_gadget.rs:71]
FOLD_START	$96 - 32 = 64$	height at which the absence fold starts [crates/prover/src/nullifier_gadget.rs:74]
ADDR_LIMBS	$96/16 = 6$	digest limbs the address bits are read from [crates/prover/src/nullifier_gadget.rs:78]
DIGEST_EXT	2	a digest is two $\mathbb{K}$ elements (8 base coefficients) [crates/prover/src/commitment_gadget.rs:72]
DIGEST_LIMBS	16	wire limbs per digest [crates/prover/src/commitment_gadget.rs:82]

The native amount domain matches the circuit range: $\text{MAX_VALUE} = 2^{62} - 1$ [crates/shielded/src/transfer.rs:45], checked at compile time against 2^{62} [crates/prover/src/transfer.rs:125].

Symbol	Byte string	Bytes	Limbs
D_{PK}	pq-rollup/spendpk/v1	20	10
D_{NOTE}	pq-rollup/note-commit/v1	24	12
D_{NF}	pq-rollup/nullifier/v1	22	11

[crates/shielded/src/keys.rs:29][crates/shielded/src/note.rs:38,41] Each tag enters the circuit as the constant limb vector $\text{limbs}(D)$ (`const_limbs`) [crates/prover/src/transfer.rs:234–239].

4.2 In-circuit gadgets

All circuit values are elements of $\mathbb{K}$ (the builder is `CircuitBuilder<Challenge>`). A digest is carried as a pair in $\mathbb{K}^2$.

Definition 4.1 (Digest constants). For a 32-byte digest d , $\langle d \rangle \in \mathbb{K}^2$ is `digest_to_ext(d)`: the strict decoder of §2 followed by packing base coefficients 0..3 and 4..7 into one $\mathbb{K}$ element each; a non-canonical word yields no value and the circuit is not built. [crates/prover/src/commitment_gadget.rs:101–106]

Definition 4.2 (Compression gadget $C_{\text{P}_2}^c$). $C_{\text{P}_2}^c(l, r)$ for $l, r \in \mathbb{K}^2$ is one row of the width-16 Poseidon2 table with `new_start` set, all four extension slots supplied as (l_0, l_1, r_0, r_1) , and the two rate outputs returned. [crates/prover/src/commitment_gadget.rs:265–287]

Definition 4.3 (Sponge gadget $H_{\text{P}_2}^c$). $H_{\text{P}_2}^c(x)$ for a limb vector x is `p2_sponge_limbs`: x is cut into chunks of 8 base elements; each chunk is recomposed (ALU `mul_add` chain) into the rate slots it fills and absorbed by one permutation row. The first row has `new_start` set and supplies only the rate slots; later rows leave the capacity unsupplied. In a short final chunk a partially filled rate slot is completed with the coefficients that `decompose_ext_to_base_coeffs_via_alu` returns for the previous row's output (zero if there is no previous row), and a wholly empty rate slot is left unsupplied. The output is the last row's two rate elements; $H_{\text{P}_2}^c(\varepsilon) = (0, 0)$. [crates/prover/src/commitment_gadget.rs:306–367] The framed form $H_{\text{P}_2}^c(D; p_1, \dots)$ is $H_{\text{P}_2}^c(\text{limbs}(D) \parallel p_1 \parallel \dots)$ (`p2_framed`). [crates/prover/src/transfer.rs:254–264]

The gadget is intended to mirror the native overwrite sponge of Def. 2.5. In the Poseidon2 AIR the chain-start zero-capacity constraint applies only to challenger sponge rows (a challenger-only table, or the trusted challenger-start selector of a shared table); on a table that also carries caller-supplied rows a `new_start` row's capacity is whatever the caller fed it [vendor/p3-recursion/poseidon2-circuit-air/src/air.rs:119–128,1163–1185]. The configuration these rows are keyed by has `challenger: false` and `shared: false` [vendor/p3-recursion/circuit/src/ops/poseidon2_perm/config.rs:135–145] [crates/prover/

src/commitment_gadget.rs:69]. An unsupplied input limb is chained to the previous row's output only on a row without **new_start** [vendor/p3-recursion/poseidon2-circuit-air/src/air.rs:843-845,1150-1161]. The via-ALU decomposition used for the short-final-chunk completion constrains only $\sum_i c_i e_i = y$ over $\mathbb{K}$ (e_i the basis of $\mathbb{K}$ over $\mathbb{F}_p$), without confining each c_i to $\mathbb{F}_p$ [vendor/p3-recursion/circuit/src/builder/circuit_builder.rs:1938-1946,1981-1983]. Consequently H_{P2}^c does not bind its input as the native H_{P2} does (initial capacity and short-final-chunk handling) [V-07]. The preimage lengths used by this relation are:

Use	Limbs	Perm. rows	Short final chunk
pk derivation (T2): $\text{limbs}(D_{PK})\ sk$	$10 + 16 = 26$	4	yes (2 limbs)
leaf / commitment (T3, T6): $\text{limbs}(D_{NOTE})\ v\ \rho\ \psi\ pk$	64	8	no
nullifier (T5): $\text{limbs}(D_{NF})\ sk\ \rho$	$11 + 16 + 16 = 43$	6	yes (3 limbs)

(The leaf preimage is $12 + 4 + 16 + 16 + 16 = 64$ limbs.)

Definition 4.4 (Digest export L). $L(y)$ for $y \in \mathbb{K}^2$ is **export_digest_limbs**: each $\mathbb{K}$ element is decomposed into its 4 base coefficients by ALU rows, each coefficient into 31 bits, and each bit vector recombined into a low 16-bit and a high 15-bit limb, giving 16 limbs. [crates/prover/src/commitment_gadget.rs:417-431]

Selection is the affine map $\text{sel}(c, x, y) = y + c \cdot (x - y)$, computed per $\mathbb{K}$ element by a **sub** and a **mul_add** [vendor/p3-recursion/circuit/src/builder/circuit_builder.rs:1021-1036]; it equals x if $c = 1$ and y if $c = 0$. Every selector used below is either a boolean-constrained bit [crates/prover/src/transfer.rs:742-753] or, in **Append**, a product $\kappa_h(1 - b_h)$ of such bits.

Definition 4.5 (Path fold). For $x \in \mathbb{K}^2$, constants $s_0, \dots, s_{n-1} \in \mathbb{K}^2$ and boolean expressions $a_0, \dots, a_{n-1}$,

$$c_0 = x, \quad c_{\ell+1} = C_{P2}^c(\text{sel}(a_\ell, s_\ell, c_\ell), \text{sel}(a_\ell, c_\ell, s_\ell)), \quad (35)$$

$$\text{Fold}(x, s, a) = c_n, \quad (36)$$

i.e. $a_\ell = 1$ places the sibling on the left. The membership fold [crates/prover/src/transfer.rs:716-757] and the nullifier-map fold [crates/prover/src/nullifier_gadget.rs:203-231] are both of this form.

Definition 4.6 (Frontier fold and append). Let $E_h = \langle \text{empty}_h \rangle$ be the commitment-tree empty-subtree constants for $h < 32$ [crates/prover/src/commitment_gadget.rs:121-133]. For $f \in (\mathbb{K}^2)^{32}$ and booleans $b \in \{0, 1\}^{32}$,

$$y_0 = (0, 0), \quad y_{h+1} = C_{P2}^c(\text{sel}(b_h, f_h, y_h), \text{sel}(b_h, y_h, E_h)), \quad \text{FoldF}(f, b) = y_{32}, \quad (37)$$

[crates/prover/src/commitment_gadget.rs:570-599], the circuit form of the native **fold_to_root** [crates/prover/src/commitment_gadget.rs:191-204]. For a leaf $C \in \mathbb{K}^2$, **Append**(f, b, C) is defined by

$$g_{-1} = C, \quad g_h = \text{sel}(b_h, C_{P2}^c(f_h, g_{h-1}), g_{h-1}), \quad (38)$$

$$\kappa_0 = 1, \quad \kappa_{h+1} = \kappa_h b_h, \quad (39)$$

$$b'_h = b_h + \kappa_h - 2\kappa_h b_h, \quad f'_h = \text{sel}(\kappa_h - \kappa_h b_h, g_h, f_h), \quad (40)$$

with the constraint $\kappa_{32} = 0$ (the tree is not full), and **Append**(f, b, C) = **FoldF**(f', b'). [crates/prover/src/commitment_gadget.rs:519-557]

4.3 Instance, circuit constants and witness

Three kinds of data enter the circuit.

- **Statement x** : the exported limbs (§4.5).
- **Circuit constants** (**define_const**), fixed when the circuit is built and therefore part of the circuit description: $\langle rt \rangle$, $\langle R_{\text{before}} \rangle$, $\langle R_{\text{after}} \rangle$, $\text{limbs}(rt)$, $\text{limbs}(rt')$, $\text{limbs}(R_{\text{before}})$, $\text{limbs}(R_{\text{after}})$ [crates/prover/src/transfer.rs:390-398,405-410,444,492,499,513-527,561-564]; each input's membership siblings s_i [crates/prover/src/transfer.rs:727-734]; each output's recipient key limbs $\text{limbs}(pk'_j)$ [crates/prover/src/transfer.rs:573-579]; each input's nullifier-map siblings $\sigma_i \in (\mathbb{K}^2)^{32}$ and lower empty-subtree digests $\epsilon_i \in (\mathbb{K}^2)^{65}$, taken from the **NullifierWitness** and checked by count only [crates/prover/src/nullifier_gadget.rs:249-260,271-288]; and the E_h .
- **Private inputs**, in allocation order:

Block (repeated)	Elements	Content and constraint source
input i ($i = 1..k$)	$16 + 16 + 16 + 4$	sk_i, ρ_i, ψ_i limbs, amount limbs v_i [crates/prover/src/transfer.rs:414-421]
output j ($j = 1..m$)	n_i $16 + 16 + 4$ $64 + 32$	membership index bits $a_{i,\ell}$ [crates/prover/src/transfer.rs:735-742] ρ'_j, ψ'_j limbs, amount limbs v'_j [crates/prover/src/transfer.rs:566-571] frontier $f_j \in (\mathbb{K}^2)^{32}$, count bits b_j [crates/prover/src/commitment_gadget.rs:489-506]
fee (once)	4	fee limbs φ [crates/prover/src/transfer.rs:529-530]
carries (once)	3	d_1, d_2, d_3 [crates/prover/src/transfer.rs:823-831]

For a limb vector $w = (w_0, \dots, w_3)$ write $\text{val}(w) = \sum_{t=0}^3 2^{16t} w_t$.

4.4 The relation

Definition 4.7 (R_{transfer}). $(x; w) \in R_{\text{transfer}}$ for the circuit built from constants as above iff all of the following hold.

T1 (ranges). Every limb of $sk_i, \rho_i, \psi_i, \rho'_j, \psi'_j$ lies in $[0, 2^{16})$ (16-bit decomposition) [crates/prover/src/transfer.rs:191-207]; for every amount $w \in \{v_i\} \cup \{v'_j\} \cup \{\varphi\}$, $w_0, w_1, w_2 \in [0, 2^{16})$ and $w_3 \in [0, 2^{14})$, so $\text{val}(w) \in [0, 2^{62})$ [crates/prover/src/transfer.rs:157-172]; $d_1, d_2, d_3 \in [0, 2^8)$ [crates/prover/src/transfer.rs:827-829]; every $a_{i,\ell}$ and every entry of b_j is boolean [crates/prover/src/transfer.rs:742] [crates/prover/src/commitment_gadget.rs:501-506].

T2 (ownership). For each input,

$$P_i = H_{P2}^c(D_{PK}; sk_i) \in \mathbb{K}^2.$$

P_i is computed, never supplied; it constrains the witness only through T3. [crates/prover/src/transfer.rs:423-426]

T3 (input leaf).

$$\lambda_i = H_{P2}^c(\text{limbs}(D_{\text{NOTE}}) \parallel v_i \parallel \rho_i \parallel \psi_i \parallel L(P_i)).$$

[crates/prover/src/transfer.rs:428-435] This is the circuit form of the native `commit` = $H_{P2}(D_{\text{NOTE}}, \text{LE64}(v), \rho, \psi, pk)$, since $\text{limbs}(\text{LE64}(v)) = v$ by `split_value` [crates/shielded/src/note.rs:108-111] [crates/prover/src/transfer.rs:224-231].

T4 (membership). With $n_i = |s_i|$,

$$\text{Fold}(\lambda_i, s_i, (a_{i,0}, \dots, a_{i,n_i-1})) = \langle rt \rangle,$$

as two equalities in $\mathbb{K}$. [crates/prover/src/transfer.rs:437-447] The index bits are tied to nothing else; n_i is the length of the supplied path and is not compared to `DEPTH` [L-05].

T5 (nullifier and map transition). For each input,

$$N_i = H_{P2}^c(D_{\text{NF}}; sk_i, \rho_i),$$

[crates/prover/src/transfer.rs:456] and the address bits are $u_{i,16q+r} = \text{bit } r \text{ of } L(N_i)_q$ for $q < 6$, $r < 16$ (so bits 31, 63, 95 are identically zero, the high limbs being 15-bit) [crates/prover/src/nullifier_gadget.rs:166-182, 262-263]. With $R_0 = \langle R_{\text{before}} \rangle$,

$$\text{Fold}(\epsilon_{i,64}, \sigma_i, (u_{i,64}, \dots, u_{i,95})) = R_{i-1} \quad (\text{absence}) \quad (41)$$

$$R_i = \text{Fold}(N_i, (\epsilon_{i,0}, \dots, \epsilon_{i,63}) \parallel \sigma_i, (u_{i,0}, \dots, u_{i,95})) \quad (\text{insertion}) \quad (42)$$

[crates/prover/src/nullifier_gadget.rs:265-290] [crates/prover/src/transfer.rs:457-470] and finally $R_k = \langle R_{\text{after}} \rangle$ [crates/prover/src/transfer.rs:513-525]. For $k = 0$ this forces $R_{\text{before}} = R_{\text{after}}$.

T6 (outputs and commitment-tree transition). For each output,

$$C_j = H_{P2}^c(\text{limbs}(D_{\text{NOTE}}) \parallel v'_j \parallel \rho'_j \parallel \psi'_j \parallel \text{limbs}(pk'_j)),$$

where $\text{limbs}(pk'_j)$ is a circuit constant [crates/prover/src/transfer.rs:573-581]. With $T_0 = \langle rt \rangle$,

$$\text{FoldF}(f_j, b_j) = T_{j-1}, \quad T_j = \text{Append}(f_j, b_j, C_j), \quad L(T_m) = \text{limbs}(rt'). \quad (43)$$

[crates/prover/src/transfer.rs:557-598] [crates/prover/src/commitment_gadget.rs:508-517] [crates/prover/src/transfer.rs:498-503] Each append allocates its own (f_j, b_j) , constrained only through $\text{FoldF}(f_j, b_j) = T_{j-1}$ [H-02]. For $m = 0$, $T_0 = \langle rt \rangle$ and the pin forces $rt' = rt$.

T7 (value conservation). With $\beta = \max(k, m + 1)$, $A_t = \sum_i v_{i,t}$, $B_t = \varphi_t + \sum_j v'_{j,t}$, $d_0 = d_4 = \beta$ (constants), for $t = 0, 1, 2, 3$:

$$A_t + 2^{16} d_{t+1} = B_t + d_t + \beta (2^{16} - 1) \quad \text{in } \mathbb{K}. \quad (44)$$

[crates/prover/src/transfer.rs:792-844]

T8 (shape). $\max(k, m + 1) \leq 16$; the number of nullifier witnesses equals k ; $|\sigma_i| = 32$, $|\epsilon_i| = 65$; $|f_j| = |b_j| = 32$. These are build-time checks that refuse to construct the circuit, not constraints.

[crates/prover/src/transfer.rs:366-383] [crates/prover/src/nullifier_gadget.rs:249-260] [crates/prover/src/commitment_gadget.rs:482-487]

Emission order. Constraints are laid in the order: for each input $i = 1..k$ in spend order, T1 (secrets, amount), T2 (with the export $L(P_i)$), T3, T4, T5 (nullifier; export of $L(N_i)$ and its address bits; absence fold; insertion fold; pin of the absence root to R_{i-1} ; a second export of $L(N_i)$ for the statement); then for each output $j = 1..m$ in output order, T1, T6 (commitment, export of $L(C_j)$, append); then $\text{limbs}(rt)$, $L(T_m)$ with its pin to $\text{limbs}(rt')$, the R_k pin, $\text{limbs}(R_{\text{before}})$ and $\text{limbs}(R_{\text{after}})$, the fee limbs with their range check, and T7. [crates/prover/src/transfer.rs:413-534] [crates/prover/src/transfer.rs:565-597] [crates/prover/src/nullifier_gadget.rs:262-290] Because R_{i-1} is the previous spend's insertion output, the order of nullifiers within a transfer is part of the relation. [crates/prover/src/transfer.rs:385-389]

4.5 Statement layout

The returned expressions are exported one base element each (**StatementExport::Base**) [crates/prover/src/transfer.rs:673-677], and the build refuses unless the export width equals the length of the native flattening **statement_limbs(public)** [crates/prover/src/transfer.rs:680-688], which emits the same order [crates/prover/src/transfer.rs:862-900]:

$$x = L(N_1) \parallel \dots \parallel L(N_k) \parallel L(C_1) \parallel \dots \parallel L(C_m) \parallel \text{limbs}(rt) \parallel L(T_m) \parallel \text{limbs}(R_{\text{before}}) \parallel \text{limbs}(R_{\text{after}}) \parallel \varphi. \quad (45)$$

Slice	Offset	Len	Origin in circuit
$L(N_i), i = 1..k$	$16(i - 1)$	16	computed nullifier digest [crates/prover/src/transfer.rs:471]
$L(C_j), j = 1..m$	$16(k + j - 1)$	16	computed output commitment [crates/prover/src/transfer.rs:581]
$\text{limbs}(rt)$	$16(k + m)$	16	constant limbs of the published root [crates/prover/src/transfer.rs:492]
$L(T_m)$	$16(k + m + 1)$	16	computed root after appends, pinned to $\text{limbs}(rt')$ [crates/prover/src/transfer.rs:498-504]
$\text{limbs}(R_{\text{before}})$	$16(k + m + 2)$	16	constant [crates/prover/src/transfer.rs:526]
$\text{limbs}(R_{\text{after}})$	$16(k + m + 3)$	16	constant [crates/prover/src/transfer.rs:527]
φ	$16(k + m + 4)$	4	private fee limbs (T1 ranges) [crates/prover/src/transfer.rs:529-531]

The statement length is therefore $|x| = 16(k + m + 4) + 4$, the same formula and offsets the block circuit uses to locate a child's fields [crates/prover/src/block.rs:163-215]: 100 for one input and one output (consistent with the “100-limb transfer” the block fold is costed against [crates/prover/src/commitment_gadget.rs:380-381]) and 132 for two-in/two-out. The fee value is fixed by the statement: the exported φ must equal $\text{split_value}(fee)$ [crates/prover/src/transfer.rs:898].

4.6 Value conservation over the integers

The column constraint as emitted:

```
// A_j + 2^16 d_{j+1} - B_j - d_j - bias(2^16 - 1) = 0
let lhs = builder.mul_add(two_16, next_expr, a_expr);
let carries = builder.add(b_expr, carry_expr);
let rhs = builder.add(carries, offset_expr);
let diff = builder.sub(lhs, rhs);
builder.assert_zero(diff);
```

[crates/prover/src/transfer.rs:833-838]

Property 4.1 (T7 is an integer identity). If T1, T7 and T8 hold then $\sum_{i=1}^k \text{val}(v_i) = \sum_{j=1}^m \text{val}(v'_j) + \text{val}(\varphi)$ in $\mathbb{Z}$.

Derivation. By T1 every term is a small non-negative integer. With $k, m+1, \beta \leq 16$ (T8), each side of a column equation is bounded by

$$A_t + 2^{16}d_{t+1} \leq 16(2^{16} - 1) + 2^{16}(2^8 - 1) < 2^{25}, \quad B_t + d_t + \beta(2^{16} - 1) \leq 32(2^{16} - 1) + 2^8 < 2^{22},$$

(the top column has $d_4 = \beta \leq 16$), both below $p > 2^{30}$, so each field equation is an integer equation [crates/prover/src/transfer.rs:127-135]. Multiplying column t by 2^{16t} and summing telescopes the carries:

$$\sum_t 2^{16t}(A_t - B_t) = d_0 - 2^{64}d_4 + \beta(2^{64} - 1) = 0 \quad \text{since } d_0 = d_4 = \beta,$$

and $\sum_t 2^{16t}A_t = \sum_i \text{val}(v_i)$, $\sum_t 2^{16t}B_t = \text{val}(\varphi) + \sum_j \text{val}(v'_j)$. [crates/prover/src/transfer.rs:759-791] The honest prover computes $d_{t+1} = \lfloor (B_t + d_t + \beta(2^{16} - 1) - A_t)/2^{16} \rfloor$ [crates/prover/src/transfer.rs:826]. $\square$

4.7 Scope of the relation

The following are stated neutrally as properties of what is and is not enforced; impact is discussed in the findings report.

- **Sponge binding.** T2, T3, T5 and T6 rest on H_{P2}^c , whose first-row capacity and short-final-chunk completion are not pinned as in the native definition; the nullifier and **pk** preimages have short final chunks [V-07].
- **Instance-specific constants.** Membership siblings s_i and recipient keys $\text{limbs}(pk'_j)$ are circuit constants, hence part of the preprocessed circuit description rather than of the witness [H-03]. The nullifier-map siblings σ_i and lower empties ϵ_i are likewise constants and are checked by count only.
- **Append position.** The frontier f_j and count bits b_j are fresh private inputs at every append, related to the tree only through $\text{FoldF}(f_j, b_j) = T_{j-1}$; the count $\sum_h 2^h b_{j,h}$ is not carried from one append to the next [H-02].
- **Path length.** n_i is not constrained to equal DEPTH [L-05].
- **Spend authorization.** No SPHINCS+ verification and no DOMAIN_TX message appear in `constrain_transfer`; spend authority inside R_{transfer} is knowledge of sk_i satisfying T2–T5 [crates/prover/src/transfer.rs:358-537] [crates/shielded/src/signing.rs:35] [M-02].

4.8 Spec deltas

Spec delta 4.1 (Nullifier map is Poseidon2, not Keccak). Two comments still describe the nullifier map as Keccak: [crates/prover/src/commitment_gadget.rs:15-16] and the proving entry point’s list of non-primitive operations [crates/prover/src/transfer.rs:904-906]. The enforced map fold is C_{P2}^c on the same Poseidon2 table as the note tree [crates/prover/src/nullifier_gadget.rs:203-231], and besides the statement export the client circuit enables only the Poseidon2 and recompose tables [crates/prover/src/transfer.rs:657-661, 677].

Spec delta 4.2 (Digest export is canonical). The export comment states that each 31-bit coefficient decomposition admits one non-canonical witness [crates/prover/src/commitment_gadget.rs:408-412]. The vendored `decompose_to_bits` equates each coefficient with a sum of boolean-constrained bits (so the coefficient lies in $\mathbb{F}_p$) and additionally pins every full-width (31-bit for $\mathbb{F}_p$) chunk below p [vendor/p3-recursion/circuit/src/builder/circuit_builder.rs:1435-1449, 1555-1569]; together with the weighted-sum constraint of the coefficient decomposition, $L(y)$ is the unique canonical limb vector of y .

Spec delta 4.3 (One frontier per append, not one per transfer). The entry-point documentation describes a single frontier witness “at `public.root`” that the gadget pins before appending [crates/prover/src/transfer.rs:346-351]. The circuit instead takes a separately allocated (f_j, b_j) per output, derived natively by `frontier_step` [crates/prover/src/transfer.rs:600-626], and pins each only against the running root T_{j-1} (T6) [H-02].

5 The block relation and statement fold

A block proof attests that $n \geq 1$ client-produced transfer proofs were each verified inside one recursion circuit, that their commitment-tree and nullifier-map transitions compose into a single chain, and that their full statements were absorbed into one Poseidon2 running digest. The block circuit is assembled per call by `build_multi_transfer_circuit` [crates/prover/src/block.rs:416-615]; it carries the Poseidon2, recompose and statement tables only [crates/prover/src/block.rs:23-30] and is proven under the Keccak settlement configuration by `settle_block_circuit` [crates/prover/src/block.rs:769-812]. This section states the relation that construction enforces, the fold (D-089), the exported statement layout, and its decoding on L1. Field and limb notation follows Section 2; the per-transfer statement x is the one of Section 4.5.

5.1 Transfer shapes and statement projections

Definition 5.1 (Transfer shape). A shape is a pair $s = (k, m)$ of input and output counts (`num_nullifiers`, `num_outputs`). [crates/prover/src/block.rs:170-176] With $\sigma(s) = k + m$, the implied statement length is

$$\ell(s) = 16(\sigma(s) + 4) + 4, \quad (46)$$

sixteen limbs per hash and four fee limbs (`LIMBS_PER_HASH=16`, `FEE_LIMBS=4`). [crates/prover/src/block.rs:157-161, 179-183]

Definition 5.2 (Statement projections). For a child statement $x \in \mathbb{F}_p^{\ell(s)}$ and $\sigma = \sigma(s)$, write $x[a, b]$ for the slice of limbs $a, \dots, b - 1$. The block circuit reads exactly the following slices:

Projection	Slice	Section 4.5 field	Source
$\text{rt}(x)$	$x[16\sigma, 16\sigma + 16)$	$\text{limbs}(\text{rt})$	<code>root_offset</code> [crates/prover/src/block.rs:185-189]
$\text{rt}'(x)$	$x[16\sigma + 16, 16\sigma + 32)$	$L(T_m)$	<code>root_after_offset</code> [crates/prover/src/block.rs:191-196]
$\text{nb}(x)$	$x[16\sigma + 32, 16\sigma + 48)$	$\text{limbs}(R_{\text{before}})$	<code>nullifier_before_offset</code> [crates/prover/src/block.rs:198-202]
$\text{na}(x)$	$x[16\sigma + 48, 16\sigma + 64)$	$\text{limbs}(R_{\text{after}})$	<code>nullifier_after_offset</code> [crates/prover/src/block.rs:204-208]
$\varphi(x)$	$x[16\sigma + 64, 16\sigma + 68)$	φ	<code>fee_offset = \ell(s) - 4</code> [crates/prover/src/block.rs:210-214]

Each 16-limb slice is extracted bounds-checked, as an error rather than a panic or short read; the fee slice is read the same way. [crates/prover/src/block.rs:635-650] [crates/prover/src/block.rs:540-544]

Every offset in Def. 5.2, and $\ell(s)$ itself, is a function of $\sigma(s) = k + m$ only; the individual split (k, m) enters the block relation solely through the exported header (Def. 5.8).

5.2 Child objects and in-circuit verification

Definition 5.3 (Child). A child is a tuple (V_c, π_c, x_c, s_c) : a retained circuit verifier object V_c , a batch-STARK proof π_c , the statement $x_c \in \mathbb{F}_p^*$ it was proven against, and a shape s_c (`ChildProof`). The verifier “travels with the proof” so that a block may mix transfer shapes. [crates/prover/src/block.rs:374-388]

Build-time gates. The following are checks that refuse to construct the circuit; they are not constraints of the relation.

- $n \geq 1$. [crates/prover/src/block.rs:420-422]
- $|x_c| = \ell(s_c)$ for every c . [crates/prover/src/block.rs:480-488]
- V_c declares a statement-table instance. [crates/prover/src/block.rs:490-494]
- V_c natively accepts (π_c, x_c) : the trusted entry point calls `verifier.verify(proof, statement)` before allocating any target. [vendor/p3-recursion/recursion/src/verifier/batch_stark.rs:761-763]
- The verified statement-target slice has length $|x_c|$. [crates/prover/src/block.rs:524-531]
- Every header count fits a `u16`. [crates/prover/src/block.rs:238-256]

In-circuit verification. For each child, in order, the block calls `verify_trusted_p3_batch_proof_circuit` with V_c , π_c , x_c , the inner WHIR PCS parameters and the KOALA_BEAR_D4.W16 challenger permutation. [crates/prover/src/block.rs:424-425,496-514] The verified relation (table AIRs, trace degree bits, per-table public values) is reconstructed from V_c , not from the proof’s transport metadata. [vendor/p3-recursion/recursion/src/verifier/batch_stark.rs:524-580] [vendor/p3-recursion/recursion/src/verifier/batch_stark.rs:764] The call allocates, in this order: per-instance AIR public-value targets as circuit public-input slots, the proof targets, and the common-data targets, the latter including the child’s preprocessed commitment as public-input cap entries. [vendor/p3-recursion/recursion/src/public_inputs.rs:738-746] [vendor/p3-recursion/recursion/src/types/proof.rs:167-177] [vendor/p3-recursion/recursion/src/pcs/fri/targets.rs:1072-1080] The block packs the corresponding values in the same order (AIR public values, proof values, common data) and assigns them as the runner’s public inputs. [vendor/p3-recursion/recursion/src/public_inputs.rs:365-389] [crates/prover/src/block.rs:551-557,595-596]

Definition 5.4 (Verified statement targets). τ_c is the AIR public-value target slice of V_c ’s statement-table instance, i.e. the targets the in-circuit verifier constrained against π_c ; its witness values are x_c . [crates/prover/src/block.rs:516-523] [vendor/p3-recursion/circuit-prover/src/batch_stark_prover.rs:2783-2803] Every value the block derives from child c (fold input, chain limbs, fees) is read from τ_c . [crates/prover/src/block.rs:537-549]

Each child’s in-circuit verification instantiates a fresh circuit challenger and, in order: observes the batch-shape domain-separator seed, each instance’s extended degree bits (as constants), the trace commitment, every instance’s public values (including τ_c), and the child’s preprocessed commitment if present; samples the lookup challenges; observes the permutation commitment (if any) and every present lookup terminal; samples α ; observes the quotient commitment and the randomization commitment (if any); and samples ζ . [vendor/p3-recursion/recursion/src/verifier/batch_stark.rs:1204-1279] Each child carries its own private-data replay keyed on the op ids its verification allocated; the replays are applied per child and never merged. [crates/prover/src/block.rs:437-440,559-564,597-607] The remainder of the in-circuit verifier, and its soundness, is the subject of Section 6.6 (see also [V-08]).

Child verifying key. The child’s preprocessed-commitment targets are public-input slots filled from V_c ’s common data [crates/prover/src/block.rs:551-556]. The WHIR backend provides a helper that constrains these targets to a fixed constant (`constrain_trusted_preprocessing`) [vendor/p3-recursion/recursion/src/backend/whir.rs:1619-1640], but `build_multi_transfer_circuit` does not invoke it. The settlement verifier assigns an empty public vector to every primitive table (Const, Public, ALU), the retained relation’s fixed values to every other non-primitive table, and the caller’s statement only to the statement table; the Public table AIR itself has no constraints. No public-input slot of the block circuit is therefore compared with an externally supplied value. [vendor/p3-recursion/circuit-prover/src/batch_stark_prover.rs:2757-2803] [vendor/p3-recursion/circuit/src/ops/op.rs:233-238] [vendor/p3-recursion/circuit-prover/src/air/public_air.rs:14-16] The child preprocessed commitment is therefore a prover-assigned value that the in-circuit verifier consumes but does not constrain to a canonical transfer verifying key [V-06].

5.3 The block relation

Definition 5.5 (Header). For shapes $s_1, \dots, s_n$ with $s_c = (k_c, m_c)$,

$$\text{Hdr}(s_1, \dots, s_n) = (n, k_1, m_1, \dots, k_n, m_n) \in \mathbb{F}_p^{1+2n},$$

emitted by `shape_header`; its width is `shape_header_len(n) = 1 + 2n`. [crates/prover/src/block.rs:217-256]

Definition 5.6 (Block relation R_{block}). R_{block} is the set of pairs $(X; (V_c, \pi_c, x_c, s_c)_{c=1}^n)$ with $n \geq 1$ such that

$$(B1) \quad \text{Verify}_{V_c}^{\text{circ}}(\pi_c, \tau_c) \text{ holds, with } \tau_c = x_c, \quad c = 1, \dots, n, \quad (47)$$

$$(B2) \quad \text{rt}'(x_c) = \text{rt}(x_{c+1}), \quad 1 \leq c < n, \quad (48)$$

$$(B3) \quad \text{na}(x_c) = \text{nb}(x_{c+1}), \quad 1 \leq c < n, \quad (49)$$

$$(B4) \quad \Sigma_0 = 0^8, \quad \Sigma_c = H_{P2}(\Sigma_{c-1} \parallel x_c), \quad c = 1, \dots, n, \quad (50)$$

$$(B5) \quad X = \text{Hdr}(s_1, \dots, s_n) \parallel L(\Sigma_n) \parallel \text{rt}(x_1) \parallel \text{rt}'(x_n) \parallel \text{nb}(x_1) \parallel \text{na}(x_n) \parallel \varphi(x_1) \parallel \dots \parallel \varphi(x_n). \quad (51)$$

[crates/prover/src/block.rs:479-590]

(B1) is the in-circuit verifier of Section 5.2 [crates/prover/src/block.rs:496-514]. (B2) and (B3) are imposed by `CommitmentChain::advance` and `NullifierChain::advance`: the first child records its *before* slice as the block endpoint; every later child’s *before* slice is constrained equal to the previous child’s *after* slice; the last *after* slice is the other endpoint. [crates/prover/src/block.rs:669-705] [crates/prover/src/block.rs:718-753] Each slice equality is sixteen constraints $a_i - b_i = 0$ over $\mathbb{K}$ (`sub` then `assert_zero`), deliberately not `connect`, which aliases witness slots and would desynchronise the per-instance LogUp multiplicities of the statement tables. [crates/prover/src/block.rs:617-633] For $n = 1$, (B2) and (B3) are empty. (B4) is Section 5.4; (B5) is Section 5.5.

Property 5.1 (Single transitions). Under (B2), the commitment-tree transitions $\text{rt}(x_c) \rightarrow \text{rt}'(x_c)$ compose to one transition $\text{rt}(x_1) \rightarrow \text{rt}'(x_n)$ with every intermediate root equal to a predecessor’s output; likewise (B3) composes the nullifier-map transitions to $\text{nb}(x_1) \rightarrow \text{na}(x_n)$. Order within the block is therefore part of the relation. [crates/prover/src/block.rs:60-90] [crates/prover/src/block.rs:574-587]

Whether each per-child edge is itself a valid tree or map transition is a property of the child’s relation, which (B1) takes from V_c (Section 5.2).

5.4 The statement fold (D-089)

Definition 5.7 (Statement fold). With H_{P_2} the overwrite sponge over $\mathbb{F}_p^*$ (Def. 2.5),

$$\begin{aligned}\Sigma_0 &= 0^8, \\ \Sigma_c &= H_{P_2}(\Sigma_{c-1} \parallel x_c) \in \mathbb{F}_p^8, \quad c = 1, \dots, n,\end{aligned}\tag{52}$$

where the sponge input is the eight elements of Σ_{c-1} in order followed by the $|x_c|$ limbs of x_c , and $\text{statementRoot} = L(\Sigma_n)$. [crates/prover/src/block.rs:273-290] [crates/prover/src/commitment_gadget.rs:363-399]

In-circuit form. The running digest is held as two $\mathbb{K}$ expressions $(\Sigma^{\text{lo}}, \Sigma^{\text{hi}})$ and seeded with the constant pair $(0, 0)$. [crates/prover/src/block.rs:447-452] `fold_statement` decomposes each element to its four base coefficients by an ALU chain (lo first, then hi), appends the targets τ_c , and absorbs the result with `p2_sponge_limbs`. [crates/prover/src/commitment_gadget.rs:388-399] [crates/prover/src/block.rs:533-537] That gadget absorbs eight base elements per permutation row, packed four per $\mathbb{K}$ lane; the first row starts a fresh chain; a short final chunk takes the missing lane coefficients from the previous row’s output (overwrite mode); the output is the two rate lanes of the last row. [crates/prover/src/commitment_gadget.rs:289-367] Since Σ_{c-1} is exactly one rate block, each child costs $1 + \lceil |x_c|/8 \rceil$ permutations (14 for $|x_c| = 100$). [crates/prover/src/commitment_gadget.rs:378-381] No domain tag and no length prefix is absorbed; the input length $8 + \ell(s_c)$ of each fold step is fixed at build time by s_c .

Native mirror. `fold_statement_native` computes (52) with `Poseidon2Commitment::hash_elements`, which is the overwrite sponge over the element sequence. [crates/prover/src/block.rs:279-290] [crates/pq-hash/src/poseidon.rs:203-206] Agreement of the two chains, including a fold step that ends mid-chunk ($8 + 68$ elements), is pinned by `fold_statement_matches_native`. [crates/prover/src/commitment_gadget.rs:792-847] The binding of Σ_n to the absorbed x_c inherits the properties of the in-circuit sponge gadget [V-07].

Export. Σ_n is exported with `export_digest_limbs`: each of the eight base coefficients is bit-decomposed to 31 bits and emitted as a low 16-bit and a high 15-bit limb, giving $L(\Sigma_n)$ (Def. 2.8 and the digest limb export of Section 2.5, including its caveat that a 31-bit decomposition also admits $w_i + p$ when $w_i < 2^{31} - p$). [crates/prover/src/commitment_gadget.rs:401-431] [crates/prover/src/block.rs:567-573] The native encoder `digest_to_limbs` produces the same sixteen limbs from `elements_to_digest`(Σ_n) by reading each little-endian u32 word as (low 16 bits, high 16 bits). [crates/prover/src/block.rs:292-305]

L1 consumer. The L1 decoder (Def. 5.11) copies `statementRoot` into the decoded block [contracts/src/BlockStatement.sol:93]; the pool’s `applyBlock` reads only the four endpoint digests and `totalFee` from it, and the `BlockApplied` event does not carry it. [contracts/src/ShieldedPool.sol:144-179] The fold root therefore has no on-chain consumer [L-04].

5.5 Block statement layout

Definition 5.8 (Block statement). The exported statement (B5) is the limb vector

$$X = [n, (k_c, m_c)_{c=1}^n, L(\Sigma_n), \text{rt}(x_1), \text{rt}'(x_n), \text{nb}(x_1), \text{na}(x_n), (\varphi(x_c))_{c=1}^n] \quad (53)$$

of length

$$|X| = (1 + 2n) + 16 + 4 \cdot 16 + 4n = 81 + 6n. \quad (54)$$

[crates/prover/src/block.rs:265-271] [crates/prover/src/block.rs:456-477,567-590]

Field	Offset (0-based)	Len	Origin in circuit
n	0	1	constant [crates/prover/src/block.rs:473-476]
k_c, m_c	$2c - 1, 2c$	1, 1	constants from s_c [crates/prover/src/block.rs:467-476]
$L(\Sigma_n)$	$1 + 2n$	16	fold export [crates/prover/src/block.rs:571-573]
$\text{rt}(x_1)$	$17 + 2n$	16	τ_1 slice, commitment chain start [crates/prover/src/block.rs:579-586]
$\text{rt}'(x_n)$	$33 + 2n$	16	τ_n slice, commitment chain end
$\text{nb}(x_1)$	$49 + 2n$	16	τ_1 slice, nullifier chain start [crates/prover/src/block.rs:582-586]
$\text{na}(x_n)$	$65 + 2n$	16	τ_n slice, nullifier chain end
$\varphi(x_c)$	$81 + 2n + 4(c - 1)$	4	τ_c fee slice, child order [crates/prover/src/block.rs:539-544,588-589]

Every entry is a `StatementExport::Base`; the export list is fixed by `set_statement_exports` and its width is asserted (debug) to equal `block_statement_len(n)`. [crates/prover/src/block.rs:590-592] The header limbs are created with `define_const`, so they are entries of the block circuit’s constant table rather than prover-supplied values, and are not placed in the public-input vector. [crates/prover/src/block.rs:456-476] The Const table constrains each main-trace value to equal a preprocessed (committed) column, so the header is part of the block circuit’s preprocessing. [vendor/p3-recursion/circuit-prover/src/air/const.air.rs:9-18] The native reconstruction `block_statement` (used by the node driver, the vector generators and tests) emits the same layout from the same `shape_header` function, after checking $|x_c| = \ell(s_c)$ and that the shape and statement counts agree. [crates/prover/src/block.rs:307-372]

Header binding. The values (k_c, m_c) are taken from the client-supplied shapes [crates/prover/src/block.rs:467]. No constraint relates a shape to its child; the only (build-time) check is $|x_c| = \ell(s_c)$ [crates/prover/src/block.rs:480-488], and by (46) and Def. 5.2 both that check and every projection that (B2)–(B5) consume depend on $k_c + m_c$ alone. The individual split in the header is therefore bound to the child only through its sum [L-05].

5.6 L1 decoding: LimbCodec and BlockStatement

Definition 5.9 (Digest from limbs). For a statement array S and offset o , `digestFromLimbs( $S, o$ )` requires $|S| \geq o + 16$ and $S[o + i] \leq 0\text{xffff}$ for $i = 0, \dots, 15$, else reverts. With $l_i = S[o + i]$ it sets bytes $\beta_{2i} = l_i \bmod 2^8$ and $\beta_{2i+1} = \lfloor l_i / 2^8 \rfloor$ and returns

$$D = \sum_{j=0}^{31} \beta_j 2^{8(31-j)} \quad (\text{bytes32}),$$

i.e. β_0 is the most significant byte. [contracts/src/LimbCodec.sol:16-46]

Property 5.2 (Round trip with the prover encoding). For a digest $d \in \mathbb{F}_p^8$ with canonical words $w_i < p$, the byte string $\beta_0 \dots \beta_{31}$ that `digestFromLimbs` assigns to $L(d)$ is `elements.to_digest( $d$ ) = LE32( $w_0$ ) || ... || LE32( $w_7$ )`: limb $2i$ of $L(d)$ is $w_i \bmod 2^{16}$, whose two bytes are bytes $4i, 4i + 1$ of `LE32( $w_i$ )`, and limb $2i + 1$ is $\lfloor w_i / 2^{16} \rfloor$, whose two bytes are bytes $4i + 2, 4i + 3$. Equivalently D is the inverse of limbs (Def. 2.8) on 32-byte strings, read big-endian. [contracts/src/LimbCodec.sol:22-26,41-44] [crates/pq-hash/src/poseidon.rs:157-163] [crates/prover/src/block.rs:292-305]

Definition 5.10 (Value from limbs). `u64FromLimbs( $S, o$ )` requires $|S| \geq o + 4$ and $S[o + i] \leq 0\text{xffff}$ for $i = 0, \dots, 3$, else reverts, and returns $v = \sum_{i=0}^3 S[o + i] 2^{16i} \leq 2^{64} - 1$ as `uint64`. [contracts/src/LimbCodec.sol:48-64]

Spec delta 5.1 (Top fee limb width). The `u64FromLimbs` documentation states that the top limb holds 14 bits and the range is 2^{62} [contracts/src/LimbCodec.sol:50-51]; the code bounds every limb, including the top one, only by `0xffff`. The 14-bit top-limb bound is enforced inside the transfer relation’s amount range checks [crates/prover/src/transfer.rs:115-123,152-172,529-531], so on L1 it holds only to the extent that (B1) verifies that relation (Section 5.2).

Definition 5.11 (Block decoding). `BlockStatement.decode( $S$ )` performs no cryptographic check (the caller must already have verified the proof against S) and evaluates, in order:

#	Step	Source
1	require $ S \geq 1$; $n := S[0]$; require $1 \leq n \leq 2^{16} - 1$	[BlockStatement.sol:80-83]
2	$cur := 1 + 2n$ (header counts skipped)	[BlockStatement.sol:85-89]
3	require $ S = \text{expectedLen}(n) = 1 + 2n + 5 \cdot 16 + 4n = 81 + 6n$	[BlockStatement.sol:63-72,90]
4	<code>statementRoot</code> , <code>rootBefore</code> , <code>rootAfter</code> , <code>nullifierBefore</code> , <code>nullifierAfter</code> := <code>digestFromLimbs</code> at $cur, cur + 16, \dots, cur + 64$	[BlockStatement.sol:92-102]
5	<code>totalFee</code> := $\sum_{c=1}^n \text{u64FromLimbs}(S, 81 + 2n + 4(c - 1))$ (<code>uint256</code>)	[BlockStatement.sol:104-107]
6	require $cur = S $	[BlockStatement.sol:108-111]

[contracts/src/BlockStatement.sol:74-112]

`ShieldedPool.applyBlock` verifies the proof against S before calling `decode`, then requires `rootBefore` = `currentRoot` and `nullifierBefore` = `currentNullifierRoot` before storing `rootAfter` and `nullifierAfter`. [contracts/src/ShieldedPool.sol:144-166] The verifier’s binding of S to the proof, including how limb values are compared to the proof’s statement words, is specified in Section 7.2.

Spec delta 5.2 (Header split is not read on L1). Comments state that the contract “re-derives the same split from the statement’s own length” [crates/prover/src/block.rs:463-466] and “reads the split from the proof” [contracts/src/BlockStatement.sol:26-29]. The enforced behaviour is that `decode` reads only $n = S[0]$, skips $S[1], \dots, S[2n]$ without inspecting them, and constrains the statement only through $|S| = 81 + 6n$, which is independent of every (k_c, m_c) . [contracts/src/BlockStatement.sol:85-90] Together with the in-circuit header binding of Section 5.5, the split is checked only by its sum [L-05].

5.7 Construction order

`build_multi_transfer_circuit` emits the circuit in one linear pass:

1. Prepare the WHIR recursion backend (`WhirRecursionBackend<16,8>`, extension degree 4) on a fresh builder. [crates/prover/src/block.rs:424-432]
2. Define the constant 0 seed of the fold; export the $1 + 2n$ header constants. [crates/prover/src/block.rs:451-477]
3. For $c = 1, \dots, n$: length gate; in-circuit verification of π_c under V_c (B1); fold τ_c into Σ (B4); collect $\varphi(\tau_c)$; advance the commitment chain (B2) and the nullifier chain (B3); pack public and private values; record the replay. [crates/prover/src/block.rs:479-565]
4. Export $L(\Sigma_n)$, the four chain endpoints, and the fees; fix the export schema; build. [crates/prover/src/block.rs:567-593]
5. Run the witness: set public inputs, set private inputs, apply each child’s private-data replay, generate traces. [crates/prover/src/block.rs:595-614]

`settle_block_circuit` then proves the resulting circuit under `crate::whir::config(BLOCK_CAP_HEIGHT, log_max_lde)` with `BLOCK_CAP_HEIGHT = 0`, registering the shared Poseidon2, recompose and statement tables (the latter from the circuit’s export schema) and returning the proof together with the settle-ment verifier. [crates/prover/src/block.rs:769-812] [crates/prover/src/block.rs:155] The driver value `BLOCK_LOG_MAX_LDE = 25` is documented as measured for fan-in 2; fan-in 1 settles at 24. [crates/prover/src/block.rs:115-138]

5.8 Scope of the relation

Stated neutrally; impact is assessed in the findings report.

- The relation each child is verified against is the one carried by V_c ; the child’s preprocessed commitment is not constrained to a fixed value [V-06].
- The header counts (k_c, m_c) are bound to the children only through $k_c + m_c$, in circuit and on L1 [L-05].
- $L(\Sigma_n)$ is exported and decoded but has no L1 consumer [L-04].
- Native verification of each child, the length gates and the shape/statement agreement are build-time checks of the honest builder, not constraints.
- Per-child tree/map validity, fee range, and value conservation are properties of the child relation (Section 4) and reach the block only through (B1).

6 Proof system: WHIR PCS, batch-STARK, recursion

This section states what the proving stack enforces underneath the application relations: the WHIR-backed univariate polynomial commitment scheme (vendored and locally patched), the batch-STARK verifier transcript, the recursion relation that the in-circuit verifier proves, and the circuit-prover table AIRs that carry it. Everything below is read from the code; module comments are used only where the code agrees with them, and disagreements are collected in §6.8.

The stack has three layers.

1. *Inner layer.* Client transfer circuits are proven under `InnerWhirConfig` [crates/prover/src/client.rs:171-172]: a width-16 Poseidon2 duplex challenger, a Poseidon2 field-native Merkle tree with 8-element digests, and `WhirUniPcs` with its const generic ZK left at its default `true` [crates/prover/src/whir_recursion.rs:84-116] [vendor/p3-recursion/recursion/src/pcs/whir/uni/pcs.rs:322].
2. *Recursion circuit.* A circuit over $\mathbb{K}$ that re-runs the inner batch-STARK (or uni-STARK) verifier with WHIR openings, Merkle paths and the Poseidon2 challenger realised as non-primitive tables; the block circuit is the multi-child instance of it [crates/prover/src/whir_recursion.rs:522-598,735-818] [crates/prover/src/block.rs:416-615].
3. *Settlement layer.* The recursion (or block) circuit is proven once more under a Keccak-256 challenger and Keccak Merkle tree with ZK = `false` [crates/prover/src/whir.rs:77,86-89] [crates/prover/src/config.rs:66-78] [crates/prover/src/whir_recursion.rs:625] [crates/prover/src/block.rs:787]; this is the proof the on-chain verifier replays.

Notation. $\mathbb{F}_p$ is KoalaBear and $\mathbb{K} = \mathbb{F}_p[X]/(X^4 - 3)$ with $d = [\mathbb{K} : \mathbb{F}_p] = 4$ [crates/prover/src/whir.rs:64-70] (p3-koala-bear 0.8.0 [koala_bear.rs:93-94]). k is the WHIR folding factor and $z = [\text{ZK}] \in \{0, 1\}$. For a two-adic coset $D = s\langle\omega\rangle$ of size n write $Z_D(X) = X^n - s^n$ (unnormalised) and $v_D(X) = (X/s)^n - 1$ (normalised), so $Z_D = s^n v_D$. The natural domain of size n is the subgroup itself, $s = 1$ [vendor/p3-recursion/recursion/src/pcs/whir/uni/pcs.rs:819-822]. $\text{idFT}_D(M)$ is the column-wise coset interpolation of an $n \times w$ evaluation matrix M on D , returning ascending coefficients.

6.1 Instantiations and parameters

Item	Settlement layer (<code>crate::whir</code>)	Inner layer (<code>InnerWhirConfig</code>)
PCS type	<code>WhirUniPcs<..., PrefixProver, false></code> [whir.rs:86]	<code>WhirUniPcs<..., PrefixProver></code> (ZK default <code>true</code>) [whir_recursion.rs:106-113]
Challenger	<code>SerializingChallenger32<F, HashChallenger<u8, Keccak256Hash, 32>></code> [whir.rs:77]	<code>DuplexChallenger<F, Poseidon2KoalaBear<16>, 16, 8></code> [whir_recursion.rs:84,103]
MMCS	Keccak-256 leaf hash, keccak($\ell \parallel r$) compression, 32-byte digest [config.rs:66-78]	Poseidon2 sponge (16/8/8) + truncated permutation 2-to-1, 8-element digest [whir_recursion.rs:87-116]
<code>security_level</code>	<code>security_level(): env WHIR_SECURITY_LEVEL, default 96</code> [whir.rs:120-131]	same function [whir_recursion.rs:212]
Folding factor k	<code>Constant(folding_factor_final()): env WHIR_FOLDING_FINAL ∈ {4,8}, default 4</code> [whir.rs:137-153,214]	<code>Constant(4)</code> [whir_recursion.rs:126,215]
Soundness regime	<code>env WHIR_SOUNDNESS_REGIME, default JohnsonBound</code> [whir.rs:182-188,215]	<code>env WHIR_INNER_SOUNDNESS_REGIME, default JohnsonBound</code> [whir_recursion.rs:185-194,216]
<code>starting_log_inv_rate</code>	1 (callers may pass a rate) [whir.rs:216,332-342]	<code>env WHIR_INNER_RATE ∈ {1,2,3}, default 1</code> [whir_recursion.rs:314-318]
<code>round_log_inv_rates</code>	empty (schedule derived per commit) [whir.rs:213]	empty [whir_recursion.rs:214]
<code>pow_bits</code>	<code>PoW($n+2, r$)</code> [whir.rs:337]	<code>PoW_{in}($n+2, r$)</code> , replaced by <code>min(floor, 30)</code> when an <code>env floor (WHIR_INNER_POW_FLOOR, else WHIR_POW_FLOOR)</code> exceeds it [whir_recursion.rs:336-353]
log max LDE / cap	per call / per call [whir.rs:317,348]	per call (module constants 22 / 0; the node passes its own height and cap 0) [whir_recursion.rs:135,138,305-308] [crates/node/src/sequencer.rs:234]
Lookup / OOD PoW bits	0 / 0 (<code>StarkConfig::new</code>) [whir.rs:350]	0 / 0 (trait default) [whir_recursion.rs:376-388]

File paths: `whir.rs`, `config.rs` and `whir_recursion.rs` are under `crates/prover/src/`. In the `pow_bits` row n is the configured log max LDE height; PoW is Definition 6.1, and `PoWin` is the same search re-implemented over the inner parameters (Poseidon2 challenger, folding factor 4, inner soundness regime) [whir_recursion.rs:196-274]. The inner-layer PoW defaults are the `StarkGenericConfig` trait defaults, which `InnerWhirConfig` does not override (p3-uni-stark 0.8.0 [config.rs:78-98]); `StarkConfig::new` sets both to 0 (p3-uni-stark 0.8.0 [config.rs:120-127]).

All security-relevant parameters above that are marked “env” are read from the process environment in library code [I-01]. The semantic settlement configuration used by the replay/exporter builds `WhirUniPcs` with the same field, DFT, Keccak MMCS and ZK = `false`, over a recording wrapper of the Keccak challenger [crates/prover/src/settlement_replay.rs:83-89,119-130], and adds one further knob, `WHIR_POW_FLOOR` [settlement_replay.rs:144-156].

Definition 6.1 (Grinding budget `PoW( $n, r$ )`). For arity n and starting log inverse rate r , `required_pow_bits_with` iterates $a = n, n-1, \dots, 0$ and, for each a , $b = 0, 1, \dots, 95$; it returns `max_pow_bits()` of the first schedule `WhirConfig::new(a, params[pow_bits = b, starting_log_inv_rate = r])` that builds [crates/prover/src/whir.rs:267-300]. The inner loop bound is the constant `SECURITY_LEVEL = 96`, while the schedule’s target is `security_level()` (env-overridable) [whir.rs:276,211]. With ℓ the configured log max LDE height, the settlement config evaluates it at arity $\ell + \text{ZK_ARITY_SLACK}$ with `ZK\_ARITY\_SLACK = 2` [whir.rs:171,337]; the semantic/export parameters evaluate it at arity $\ell + \text{NON_ZK_ARITY_SLACK}$ with value 1 and then apply `max(·, WHIR_POW_FLOOR)` (no clamp) [crates/prover/src/settlement_replay.rs:164-182] [whir.rs:176].

6.2 WHIR univariate PCS: encoding

p3-whir commits multilinear polynomials. The adapter `WhirUniPcs` commits each univariate column by its coefficient vector, read as the hypercube evaluations of a multilinear, and stacks all columns of one

commitment into a single multilinear [vendor/p3-recursion/recursion/src/pcs/whir/uni/pcs.rs:1-11]. **new** rejects any folding factor that is not **FoldingFactor::Constant** [pcs.rs:395-404].

Definition 6.2 (Padded arity and stacked arity). A committed coefficient matrix of height 2^n and width w has padded arity $a = \max(n, k)$ [vendor/p3-recursion/recursion/src/pcs/whir/uni/plan.rs:76-82]. A commitment over matrices $(2^{n_t}, w_t)_t$ has stacked arity

$$m = \left\lceil \log_2 \sum_{t: w_t > 0} w_t 2^{a_t} \right\rceil, \quad m = 0 \text{ if the sum is 0,} \quad (55)$$

computed with checked arithmetic (shift, product, sum and next-power-of-two overflows are errors) [plan.rs:85-113].

Definition 6.3 (Stacked layout). Tables are stably sorted by padded arity ascending and placed in reverse order, so the largest tables take the lowest offsets and, among equal arities, the later-indexed table is placed first. Each column claims one contiguous slot of 2^a points at the running offset o ; its selector has $m - a$ bits and index $\text{bitrev}_{m-a}(o \gg a)$ [plan.rs:176-242]. Under the **PrefixProver** layout the selector bits are appended *after* the local point, big-endian [plan.rs:9-22, 141-155]. A column $c(X) = \sum_{j < 2^n} c_j X^j$ becomes the multilinear f_c on $\{0, 1\}^a$ with $f_c(b) = c_{\text{int}(b)}$, zero-extended from 2^n to 2^a points; the commit path transposes each coefficient matrix so that one **Table** row is one polynomial [pcs.rs:489-493] [plan.rs:67-75].

Definition 6.4 (Univariate-to-multilinear bridge). For $\zeta \in \mathbb{K}$ and arity a , let $Y = (\zeta^{2^{a-1}}, \dots, \zeta^2, \zeta)$ (big-endian) [p3-multilinear-util-0.8.0/src/point.rs:322-334], and set

$$X_i = \frac{Y_i}{1 + Y_i}, \quad s_a(\zeta) = \prod_{i < a} (1 + Y_i) = \sum_{j < 2^a} \zeta^j. \quad (56)$$

Then for every column, $c(\zeta) = s_a(\zeta) \cdot f_c(X)$ [vendor/p3-recursion/recursion/src/pcs/whir/uni/bridge.rs:1-21]. The native routine asserts $1 + Y_i \neq 0$ for every i before inverting [bridge.rs:41-58]; the circuit routine computes the same (X, s_a) with $a - 1$ squarings and a divisions [bridge.rs:65-85].

6.3 WHIR univariate PCS: commit

All commitments funnel into **commit_coefficient_matrices_as**, which builds the stacked witness, runs the WHIR commit under **WhirConfig**(m, params), and directs the WHIR-internal root absorption into a discarded clone of the challenger prototype; the STARK layer absorbs the commitment itself [pcs.rs:479-523].

Definition 6.5 (Non-hiding commitment, **ZK = false**). Given pairs (D_t, M_t) with $|D_t| = \text{height}(M_t)$:

$$\text{Commit}^0((D_t, M_t)_t) = \text{WHIR.Commit}_m(\text{Stack}(\text{iDFT}_{D_t}(M_t))_t). \quad (57)$$

The trace, the preprocessed matrices and the quotient chunks are all committed this way, with no randomness added; no randomisation commitment is produced (**get_opt_randomization_poly_commitment** returns **None**); and a single quotient chunk is permitted [pcs.rs:846-857, 1000-1008, 1037-1039, 1097-1109].

Definition 6.6 (Hiding commitment, **ZK = true**). The masking stream is a **StdRng** seeded from the OS entropy source at construction; each commitment forks a fresh stream, and **Clone** forks the stream rather than copying it [pcs.rs:353-364, 419-430].

1. *Trace*. The caller passes an $N \times w$ matrix M with the doubled domain D , $|D| = 2N$. The adapter forms $M' \in \mathbb{F}_p^{2N \times w}$ with $M'[2r] = M[r]$ and $M'[2r + 1] \leftarrow_{\$} \mathbb{F}_p^w$ (by appending w random columns and reinterpreting at width w) and commits $\text{iDFT}_D(M')$ [pcs.rs:883-897]. Since the even rows of D form the coset $s\langle \omega_N \rangle$, the committed column is $T' = \hat{T} + (X^N - s^N)\rho$ with $\deg \hat{T} < N$ the interpolant of the real rows and $\deg \rho < N$ fixed by the random rows ($s = 1$ for natural domains).
2. *Preprocessed*. Same interleave with *zero* rows in place of random rows; the prover data is tagged public [pcs.rs:993-1020].
3. *Randomisation R*. For each requested domain D_i , a uniformly random coefficient matrix $R_i \in \mathbb{F}_p^{|D_i| \times d}$, committed directly as coefficients (no interleave) [pcs.rs:1033-1074].

4. *Quotient chunks.* Let the chunks $q_1, \dots, q_L$ live on cosets D_i of size h and shift σ_i , width w . The adapter requires $L > 1$ (assertion) and $h \cdot w \geq d$ [pcs.rs:1110-1145]. With $c_i = 1 / \prod_{j \neq i} v_{D_j}(\sigma_i)$ [pcs.rs:1413-1428], it samples $T_1, \dots, T_{L-1}$ uniformly with $\deg T_i < h$ and sets

$$T_L = - \sum_{i < L} \frac{c_i \sigma_i^h}{c_L \sigma_L^h} T_i, \quad \text{so that} \quad \sum_{i \leq L} c_i \sigma_i^h T_i = 0, \quad (58)$$

[pcs.rs:1169-1191]. Each chunk is replaced by

$$q'_i = \text{iDFT}_{D_i}(q_i) + (X^h - \sigma_i^h) T_i, \quad \deg q'_i < 2h, \quad (59)$$

[pcs.rs:1193-1217], and committed through the coefficient path with unit-shift placeholder domains of height $2h$ [pcs.rs:566-578, 1228-1233].

Definition 6.7 (Hiding-budget predicate). Let $Q(m)$ be the total query count of `WhirConfig`(m, params) (sum over rounds plus the terminal round), falling back to `security_level` if no schedule builds [pcs.rs:1377-1388]. The check is

$$\text{HB}(\text{cells}, \text{pts}, m) : \iff 2(d \cdot \max(\text{pts}, 1) + Q(m)) \leq \text{cells} \quad (60)$$

[pcs.rs:1323-1350]. It is evaluated once per commitment batch: for the trace with $\text{cells} = \sum_t N_t w_t$, $\text{pts} = 2$ and m read at log height $+1$ [pcs.rs:862-882]; for R with $\text{cells} = \sum_i |D_i| d$, $\text{pts} = 1$ [pcs.rs:1045-1060]. The quotient path checks only $h \cdot w \geq d$ [pcs.rs:1134-1145]. The budget is pooled over the batch and no per-column or per-table bound is enforced [M-07].

Property 6.1 (Single-use private openings). `open_with_preprocessing` asserts, for every round, that the prover data's `preprocessed` tag equals “this round is the preprocessed round”, and returns `AlreadyOpened` if a non-preprocessed prover data was opened before; the flag is an `AtomicBool` shared across clones [pcs.rs:124-130, 1241-1265]. This is a prover-side discipline; the verifier has no counterpart.

6.4 WHIR univariate PCS: open and verify

Definition 6.8 (Opening schedule). For one commitment with matrix shapes $(n_t, w_t)_t$ and opening points $(Z_t)_t$, every column of matrix t is opened at every $\zeta \in Z_t$; each pair (t, ζ) is one batch naming all w_t columns, at the substituted point $X^{(t, \zeta)}$ with scale $s_{a_t}(\zeta)$ of Definition 6.4. Batches are ordered matrix-major, then point [pcs.rs:253-313]. The prover reports the true univariate values $c(\zeta)$ (Horner on the stored coefficients) and one WHIR argument per commitment, all on the caller's challenger in commitment order [pcs.rs:586-637].

Definition 6.9 (Verification). Given commitments with per-matrix $(D_t, ((\zeta, v_{t, \zeta}))_\zeta)$ and a proof $\pi = (\pi_r)_r$, `verify_rounds` enforces, in order [pcs.rs:662-791]:

1. $|\pi|$ = number of commitments [pcs.rs:671-676];
2. for *every* round, before the challenger is touched: each matrix has at least one point and all claim vectors of a matrix have equal length w_t ; m is computed from $(\log |D_t|, w_t)$; the round parameters and the proof's WHIR context validate against `WhirConfig`(m) [pcs.rs:678-716];
3. for each round r in order: `WHIR.VerifyAt`($C_r, \pi_r, \text{schedule}, (X^{(t, \zeta)}))$ on the shared challenger, which does not absorb C_r and returns bound multilinear values $b_{t, \zeta} \in \mathbb{K}^{w_t}$ [pcs.rs:641-646, 746-760];
4. for every batch and column j :

$$b_{t, \zeta}[j] \cdot s_{a_t}(\zeta) = v_{t, \zeta}[j] \quad (61)$$

[pcs.rs:762-787].

The claimed values are compared as supplied; nothing is truncated or re-joined in the hiding variant [pcs.rs:1267-1303]. The per-matrix shapes are *not* authenticated by the commitment (distinct shape vectors can share a stacked arity); the caller must fix them, which the batch-STARK verifier does from AIR widths and degree bits [pcs.rs:648-660].

In-circuit, the WHIR PCS sets `PRE_OBSERVES_OPENED_VALUES = false`, so the generic STARK verifier does not pre-absorb opened values on its behalf [vendor/p3-recursion/recursion/src/pcs/whir/uni/recursive_pcs.rs:381-384], and the bridge check is emitted as `connect( $b \cdot s_a(\zeta), v$ )` per column [vendor/p3-recursion/recursion/src/pcs/whir/uni/circuit.rs:393-416]. Merkle openings of WHIR queries are checked by the Poseidon2 Merkle-path lowering; on the binary path that lowering emits path rows with `mmcs_index_sum` unset and a `new_start` first row [vendor/p3-recursion/circuit/src/ops/mmcs.rs:116-160]; on non-arity-4 Merkle rows the Poseidon2 AIR gates its input-limb sends by $1 - \text{merkle_path}$ and checks the direction bit only for booleanity [vendor/p3-recursion/poseidon2-circuit-air/src/air.rs:1028,1965-1990], so, per the structural analysis in [V-08], the resulting rows bind neither the leaf digest nor the direction bits.

6.5 Batch-STARK transcript

The native verifier transcript is `BatchVerifierTranscript` of the crates.io dependency `p3-batch-stark 0.8.0` [Cargo.toml:103]; the settlement replay drives it phase by phase and is the specification the Solidity transcript is written from [crates/prover/src/settlement_replay.rs:429-620]. The in-circuit verifier `verify_batch_circuit` mirrors it [vendor/p3-recursion/recursion/src/verifier/batch_stark.rs:1204-1550]. Below, “native” citations are to `p3-batch-stark-0.8.0/src/transcript.rs` and “circuit” citations to `batch_stark.rs`. Let e_i be the extended degree bits of instance i and $b_i = e_i - z$.

- T0. **Seed.** Absorb a domain-separator seed derived from `BatchShape` = (trace widths, public-value counts, preprocessed widths, has-preprocessed, number of lookup instances, lookup PoW bits, has-randomisation = z , OOD PoW bits); the separator binds the protocol name/version, the step pattern, the instance count, and every trace and preprocessed width. Native [transcript.rs:408-432,773-780]; circuit [batch_stark.rs:1207-1221]; replay [settlement_replay.rs:507-522].
- T1. **Instance bindings.** For each i , observe $\iota(e_i) \in \mathbb{K}$ (the base-field integer embedded in $\mathbb{K}$). Native [transcript.rs:789-796]; circuit, as circuit constants [batch_stark.rs:1222-1226].
- T2. **Main.** Observe the main-trace commitment, then each instance’s public values. Native [transcript.rs:805-815]; circuit [batch_stark.rs:1228-1234].
- T3. **Preprocessed.** If present, observe the global preprocessed commitment. Native [transcript.rs:818-823]; circuit [batch_stark.rs:1235-1237].
- T4. **Lookup challenges.** If any instance has lookups: proof-of-work step at `lookup_pow_bits` (at 0 bits nothing is absorbed and the witness must equal 0), then sample $\alpha_L, \beta \in \mathbb{K}$; with no lookups, no step exists and a supplied witness is rejected. Native [transcript.rs:833-896], with the zero-bit check absorbing nothing (p3-challenger 0.8.0 [grinding_challenger.rs:48-54]); circuit [batch_stark.rs:1251-1256,1714-1721]. The per-lookup challenge pair is $(\alpha_L + (\text{bus} + 1)\gamma, \beta)$ with $\gamma = \beta^W$, $W = \max(1, \text{widest message})$, buses numbered in first-appearance order with global buses deduplicated by name and local buses fresh [batch_stark.rs:1723-1777].
- T5. **Permutation.** If present, observe the permutation commitment, then every present lookup terminal (in $\mathbb{K}$); sample the constraint-folding challenge $\alpha \in \mathbb{K}$. Native [transcript.rs:909-923]; circuit [batch_stark.rs:1258-1267].
- T6. **Quotient.** Observe the quotient-chunk commitment, then the randomisation commitment if $z = 1$. Native [transcript.rs:926-933]; circuit [batch_stark.rs:1269-1277].
- T7. **OOD point.** Proof-of-work step at `ood_pow_bits` (same zero-bit rule as T4), then sample $\zeta \in \mathbb{K}$. Native [transcript.rs:941-964]; circuit [batch_stark.rs:1278-1279].
- T8. **Opening argument.** One WHIR argument per commitment round on the same challenger, rounds in the order of Table 6, bracketed as the `OPENING_ARGUMENT` sub-protocol natively [transcript.rs:975-981]; circuit [batch_stark.rs:1294-1550].
- T9. **Constraint identity** per instance (Definition 6.10).
- T10. **Terminal sum.** $\sum_i \tau_i = 0$ over all present terminals. Circuit [batch_stark.rs:1685-1687] [vendor/p3-recursion/recursion/src/traits/recursive.rs:91-98]; replay [settlement_replay.rs:616-620].

The circuit replays no proof-of-work step for T4 or T7 and therefore rejects any configuration with non-zero lookup or OOD PoW bits [batch_stark.rs:1195-1202]; both deployed configurations use 0 (§6.1). The proof’s lookup and OOD PoW witnesses are not allocated as circuit targets [vendor/p3-recursion/recursion/src/

types/proof.rs:403-412], so the native zero-witness rule has no in-circuit counterpart. The native settlement replay additionally checks `check_multiplicity_height_bound` before the transcript [settlement_replay.rs:497-498]; no counterpart of that check appears in the vendored recursion verifier [I-07].

Round	Present when	Matrices, domain	Points
R	$z = 1$	one per instance, extended domain 2^{e_i}	ζ
Trace	always	one per instance, extended domain 2^{e_i}	ζ ; ζg_i if next row opened
Quotient	always	2^{ℓ_i+z} chunks per instance, natural domain of size $ D_{i,j} \cdot 2^z$	ζ
Preprocessed	has prep. commitment	one per prep. matrix, natural domain 2^{e_i}	ζ ; ζg_i if next row opened
Permutation	any lookups	one per lookup instance, extended domain 2^{e_i}	ζ , ζg_i

Table 6: Commitment rounds of T8, in order. g_i is the generator of the base trace domain H_i of size 2^{b_i} . Circuit [batch_stark.rs:1294-1520]; native `commitments_with_opening_points` (p3-batch-stark 0.8.0 [verifier/mod.rs:103]) called at [settlement_replay.rs:559-569].

Before the transcript starts, the circuit builder fixes the proof shape (these are construction-time checks that return an error, not in-circuit constraints): degree bits are bounds-checked [batch_stark.rs:974-987]; randomisation presence must equal z [batch_stark.rs:1021-1028]; terminal presence must equal the AIR’s own `declares_interactions` (not the prover-supplied lookup list) [batch_stark.rs:1097-1105]; each instance has exactly 2^{ℓ_i+z} quotient chunks of length d , and a randomisation opening of length d when present, where ℓ_i is recomputed from the AIR [batch_stark.rs:1107-1145]; and the commitment count equals $[R] + 2 + [\text{prep}] + [\text{perm}]$ [batch_stark.rs:1180-1189]. Flattened permutation openings must have length $(n_{L,i} + 1) \cdot d$, with $n_{L,i}$ the number of lookups of instance i (or 0 when it has none), and are recombined into $\mathbb{K}$ by the basis $\{e_k\}$ [batch_stark.rs:1076-1095, 1574-1623].

Definition 6.10 (Per-instance constraint identity). For instance i let H_i be the natural domain of size 2^{b_i} , let $\mathcal{Q}_i$ be the coset of size $2^{b_i+\ell_i+z}$ disjoint from the extended domain, split into chunks $D_{i,1}, \dots, D_{i,L_i}$, $L_i = 2^{\ell_i+z}$ [batch_stark.rs:1357-1373]. With opened chunk values $u_{i,j} \in \mathbb{K}^d$, set $q_{i,j}(\zeta) = \sum_{k < d} e_k u_{i,j}[k]$ and

$$\Lambda_{i,j}(\zeta) = \prod_{j' \neq j} \frac{v_{D_{i,j'}}(\zeta)}{v_{D_{i,j'}}(\text{first}(D_{i,j}))}, \quad Q_i(\zeta) = \sum_j \Lambda_{i,j}(\zeta) q_{i,j}(\zeta) \quad (62)$$

[vendor/p3-recursion/recursion/src/verifier/quotient.rs:60-87, 172-269, 294-349]. Let $C_0, \dots, C_{K-1}$ be the AIR’s constraints (including the LogUp constraints under the T4 challenges and the instance’s terminal), evaluated at the opened trace, preprocessed, permutation, periodic and public values with the selectors of H_i at ζ , and folded in Horner order $F_i = \sum_{k < K} \alpha^{K-1-k} C_k(\zeta)$ (recurrence $acc \leftarrow acc \cdot \alpha + C_k$, base-field constraints first, then extension-field constraints) [vendor/p3-recursion/recursion/src/traits/air.rs:183-209]. The circuit enforces

$$F_i \cdot Z_{H_i}(\zeta)^{-1} = Q_i(\zeta) \quad (63)$$

by `connect` [batch_stark.rs:1552-1683].

6.6 Recursion relation

Definition 6.11 (Child verifying data). A child batch proof is checked against $vk_c = (\mathcal{A}_c, \mathbf{e}_c, \Lambda_c, \mathbf{pk}_c, P_c)$: the table AIRs $\mathcal{A}_c$ with their packing, the extended degree bits $\mathbf{e}_c$, the lookup contexts Λ_c , the statement-table position $\mathbf{pk}_c$, and the global preprocessed commitment P_c .

Definition 6.12 (Recursion relation).

$$R_{rc} = \left\{ (S_{rc}; (\pi_c, S_c, P_c, \mathbf{aux}_c)_c) : \forall c \text{ BatchVerify}_{vk_c}^{\text{WHIR}}(\pi_c, S_c) = 1 \wedge S_{rc} = \text{Export}((S_c)_c) \right\} \quad (64)$$

where `BatchVerifyWHIR` is T0–T10 with the WHIR checks of Definition 6.9 over the inner (Poseidon2, $z = 1$) configuration, and $\mathbf{aux}_c$ collects Merkle siblings and WHIR proof data supplied as private inputs [crates/prover/src/whir_recursion.rs:796-811]. This is the relation the construction targets; the enforced relation is weaker where P_c is left free (table below, [V-06]) and where in-circuit Merkle openings do not bind leaf and direction (§6.4, [V-08]).

How each component of vk_c is fixed in the circuit:

Component	Enforced status
$\mathcal{A}_c$, packing	Rebuilt from the retained <code>CircuitVerifier</code> descriptor, not from the proof; circuit structure [batch_stark.rs:524-580,717-787]
e_c	Taken from the proof at build time, bounds-checked, absorbed as circuit constants [batch_stark.rs:974-987,1222-1226]
Λ_c	From the retained descriptor’s common data (the trusted path passes <code>rebuild.lookups = false</code>) [batch_stark.rs:784,853-866]
pk_c	<code>statement_layout().table_instance()</code> of the descriptor [whir_recursion.rs:747-750] [crates/prover/src/block.rs:490-494]
P_c	Allocated as recursion-circuit public-input targets (<code>alloc_public_input_array</code>) [vendor/p3-recursion/recursion/src/types/proof.rs:167-177] [vendor/p3-recursion/recursion/src/pcs/fri/targets.rs:1071-1079]; the backend’s <code>constrain_trusted_preprocessing</code> [vendor/p3-recursion/recursion/src/backend/whir.rs:1619-1640] is not called by <code>build_batch_recursion_circuit</code> or the block builder, so P_c is not pinned [V-06]

`verify_trusted_p3_batch_proof_circuit` also runs `verifier.verify(proof, statement)` natively before allocating anything [batch_stark.rs:761-763]; that is a prover-side precondition, not part of the proven relation. The uni-STARK entry point (`build_recursion_circuit`) is the analogous relation for a single `p3_uni_stark` proof with no preprocessed commitment [crates/prover/src/whir_recursion.rs:522-583].

Export. Export installs a list of circuit targets as the statement via `set_statement_exports`:

- uni-STARK child: the child’s AIR public-value targets [whir_recursion.rs:572-577];
- batch child: the AIR public-value targets of the child’s statement-table instance [whir_recursion.rs:782-790];
- block: shape-header constants, the statement fold, the four endpoint digests and the per-child fee limbs, as stated in the block relation [crates/prover/src/block.rs:440-592].

Settlement wrap. The witnessed recursion circuit is proven under the settlement configuration with the shared-challenger Poseidon2 table, the recompose tables (plain and the split `recompose/coeff` table with per-coefficient lookups), and the statement table registered [crates/prover/src/whir_recursion.rs:677-699]. The settlement statement is the statement table’s public values, i.e. S_{rc} (Table 7, Statement row). With $z = 0$ at this layer the settlement openings are unmasked (Definition 6.5); hiding of inner witnesses rests on the inner layer’s $z = 1$ commitments.

6.7 Circuit-prover table AIRs

All tables communicate through the global `WitnessChecks` bus with tuples $(idx, v_0, \dots, v_{d-1})$ and preprocessed (signed) multiplicities. Paths are relative to `vendor/p3-recursion/circuit-prover/src/air/` unless stated.

The interleave that realises the trace mask of Definition 6.6 is short enough to quote [vendor/p3-recursion/recursion/src/pcs/whir/uni/pcs.rs:886-894]:

```
for (domain, mat) in evaluations {
    let width = mat.width();
    let masked = {
        let mut m = mat.with_random_cols(width, &mut rng);
        m.width = width;
        m
    };
    debug_assert_eq!(masked.height(), domain.size());
    coeffs.push(self.dft.coset_idft_batch(masked, domain.shift()));
}
```

6.8 Spec deltas

Spec delta 6.1 (PATCHES.md versus the patched PCS). PATCHES.md lists a single change, “const ZK: bool = false → true” with “evaluation truncation”, and refers to a pristine copy at `.scratch/vendor/p3-recursion-pristine` [vendor/p3-recursion/PATCHES.md:15-37]. In the code, ZK is a const generic defaulting to `true` that the settlement layer instantiates as `false` [pcs.rs:322,967] [crates/

Table	Enforced local constraints	Bus interaction(s)
Public	none [public_air.rs:14-16,202-229]	per lane (idx, v) , multiplicity from preprocessing
Const	$v_j = prep_j$ for $j < d$ [const_air.rs:160-182]	(idx, v) , preprocessed multiplicity
ALU	with $act = -mult_a$ and $s_{mul} = act - s_{bool} - s_{muladd} - s_{horner} - s_{add}$: ADD $s_{add}(a+b-o) = 0$; MUL $s_{mul}(ab-o) = 0$; BOOL $s_{bool}a_0(a_0 - 1) = 0$, $s_{bool}a_{j \geq 1} = 0$; MULADD $s_{muladd}(ab + c - o) = 0$; HORNER $s'_{horner}(ob' + c' - a' - o') = 0$; when the packed-Horner columns are present the single-step gate becomes $s'_{horner} - \sum_{k \geq 2} s'_k$ and lane 0 additionally carries the packed K -step constraints and a zero chain-head seed [alu_air.rs:885-1066]	a, b, c, o per lane and packed Horner operands [alu_air.rs:1070-1157]
Recompose	none [recompose_air.rs:1-6,157-205]	$(idx_{out}, v_0, \dots, v_{d-1})$; with coefficient lookups also $(idx_{c,i}, v_i, 0, \dots, 0)$ for each $i < d$
Statement	$act(v_j - pv_j) = 0$, $(1-act)v_j = 0$ [statement_air.rs:86-104]	$(idx_j, v_j, 0, \dots, 0)$, multiplicity $-act$ [statement_air.rs:106-113]
Poseidon2	upstream Poseidon2 permutation AIR, sponge chaining, Merkle-path chaining, MMCS leaf-index accumulator [vendor/p3-recursion/poseidon2-circuit-air/src/air.rs:27-35]	input/output limbs per CTL flags; Merkle binding [V-08]

Table 7: Table AIRs of the recursion circuit, one row each.

prover/src/whir.rs:86]; the non-hiding branches and the quotient-chunk mask are not inventoried [pcs.rs:846-857,1092-1218]; there is no truncation step [pcs.rs:1267-1272]; and the pristine directory is absent from the repository [I-02]. The module header still states that the adapter’s ZK is **true** [pcs.rs:13-16].

Spec delta 6.2 (Stated security target). The settlement module header says **SECURITY_LEVEL** is “set directly to the 128-bit target” [crates/prover/src/whir.rs:44-46]; the constant is 96 and the effective value is the env-overridable `security_level()` [whir.rs:120-131] [I-01]. The constant’s own doc comment states the 96-bit choice and explains that 128 bits is above the structural ceiling on this field [whir.rs:91-119]; only the module header is stale.

Spec delta 6.3 (Two grinding-budget arities for the settlement layer). The production settlement config derives PoW at $n + \text{ZK_ARITY_SLACK} = n + 2$ [whir.rs:337], while the semantic/export parameters use $n + \text{NON_ZK_ARITY_SLACK} = n + 1$ [settlement_replay.rs:169-172]; the comment above the latter says **ZK_ARITY_SLACK** [settlement_replay.rs:132-134]. Whether the two yield the same `pow_bits` at the deployed sizes was not evaluated here.

7 The deployed Solidity settlement verifier

This section states what the on-chain settlement verifier enforces when the pool calls `verify(statement, proof)`. The deployment script creates the `TerminalWeight` satellite first and then the bare `WhirVerifier` engine with the satellite’s address; the pool is constructed with that engine [contracts/script/Deploy.sol:40-45], and `applyBlock` accepts a block only if `verify` returns **true** [contracts/src/ShieldedPool.sol:144-145]. The engine replays the batch-STARK transcript of §6.5, then one WHIR opening argument per commitment round on the same Keccak sponge, then the constraint identity of Definition 6.10.

Conventions. Unless a directory is given, `.sol` files cited below are under `contracts/src/verifier/`; `KeccakChallenger.sol` is under `contracts/lib/sol-whir-p3/transcript/` and `KoalaBearExt4.sol` under `contracts/lib/sol-whir-p3/field/`. Field, packing, Montgomery map $\text{mont}(v) = v \cdot 2^{32} \bmod p$ and the Keccak sponge are those of Section 2. We write $\text{Obs}_{\mathbb{F}_p}(w)$ for absorbing one wire word $w < p$ (four little-endian bytes, `observeBase`), $\text{Obs}_{\mathbb{K}}(a)$ for absorbing a packed $\mathbb{K}$ element as four words $\text{mont}(c_k \bmod p)$ (`observeExt4Mont`, which reduces each lane modulo p before converting) [KeccakChallenger.sol:154-179], $\text{Obs}_{32}(h)$ for appending a 32-byte digest verbatim [KeccakChallenger.sol:181-183,1292-1304] [WhirVerifierCore.sol:113-115], `Draw $\mathbb{K}$` for four `sampleBase` draws packed limb 0 first [BatchTranscript.sol:195-201] [SumcheckCore.sol:222-231], `Bits $_b$` for `sampleBits(b)`, and `Grind $_b(w)$` for `checkWitness(b, w)`. ω_k denotes the k -th entry of the two-adic generator table ($k \leq 24$, reverting otherwise) [WhirVerifier.sol:142-196].

The verifier is a function of its two arguments, the immutable satellite address and the satellite’s code; it reads no storage and binds neither the calling pool nor the chain [WhirVerifier.sol:117-132,209-214] (see

[M-01]).

7.1 Wire format

Definition 7.1 (Bundle). The proof argument π is a byte string read in place from calldata. All counts are u32 little-endian (“words”); a “word offset” o denotes byte $4o$ of π [WhirVerifier.sol:1115-1121].

Bytes	Field	Enforced
[0, 4)	magic	equals ASCII WBND (0x57424E44) [WhirVerifier.sol:61, 228, 233]
4	version v	$v \in \{5, 7, 8\}$; only byte 4 is read, bytes 5–7 are ignored [WhirVerifier.sol:229, 242]
[8, 12)	c = CONFIG word count	u32 LE [WhirVerifier.sol:230, 235]
[12, 16)	f = PROOF word count	u32 LE [WhirVerifier.sol:231, 236]
[16, 16+4 c)	CONFIG	word offsets [4, 4+ c] [WhirVerifier.sol:261]
[16+4 c , 16+4 c +4 f)	PROOF	word offsets [4+ c , 4+ c + f] [WhirVerifier.sol:262]
[16+4(c + f), +4)	s = STATEMENT word count	u32 LE [WhirVerifier.sol:246-254]
[20+4(c + f), +4 s)	STATEMENT	byte slice handed to the satellite [WhirVerifier.sol:255]

The only length rules are, in order,

$$|\pi| \geq 16, \quad |\pi| \geq 20 + 4(c + f), \quad |\pi| \geq 20 + 4(c + f) + 4s \quad (65)$$

[WhirVerifier.sol:219, 243, 257]; trailing bytes are permitted.

Definition 7.2 (Item encodings). Section decoders are cursor-driven readers over π [WhirVerifier.sol:1115-1285]:

Reader	Layout and checks
<code>_word</code>	one u32 LE
<code>_arr</code>	count n , then n u32 LE words [WhirVerifier.sol:1123-1145]
<code>_blob</code>	byte count n , then n bytes padded to a word boundary [WhirVerifier.sol:1148-1167]
<code>_blob32</code>	byte count, which must equal 32, then one digest [WhirVerifier.sol:1170-1182]
<code>_blobArr32</code>	byte count n , then $\lfloor n/32 \rfloor$ digests [WhirVerifier.sol:1186-1203]
<code>_raw32</code>	one 32-byte item, no prefix [WhirVerifier.sol:1206-1215]
<code>_raw32Arr</code>	count n , then n packed $\mathbb{K}$ words; reverts if any low-128-bit padding is nonzero [WhirVerifier.sol:1218-1240]
<code>_extArr</code>	byte count with bit 31 as an in-band flag (masked off before use): set $\Rightarrow$ 16 bytes per element (four big-endian u32 lanes placed in the high 128 bits, lane offsets 224/192/160/128, low 128 bits zero); clear $\Rightarrow$ 32 bytes per element, low-128-bit padding must be zero [WhirVerifier.sol:1243-1285]

Lanes of packed $\mathbb{K}$ words are not range-checked against p by these readers [M-04]. The version byte selects nothing: the compact-limb (v7) and pruned-path (v8) layouts are selected per array by the bit-31 flags [WhirVerifier.sol:1048-1061, 1250-1253].

Definition 7.3 (CONFIG grammar). Starting at word offset 4:

1. *Batch prefix*: length L , then `seedBytes` (blob), `degreeBytes` (blob), `preDigest` (raw32), `lookupPowBits`, `oodPowBits`, `hasRand`; the cursor must land exactly L words after the length word [WhirVerifier.sol:378-393].
2. R = number of opening rounds; an array `round.starts` that is skipped [WhirVerifier.sol:336-340].
3. R round schedules Σ_r , each the fields of `RoundCfg` in order: M =`nInter`, `claimPerm`, `framingHex`, `framingPre`, `framingClaim`, `framingBatching`, `framingSeps`, `claimWidths`, k_0 =`numVariables`, `startingPowBits`, `commitmentOodSamples`, `schedPowBits`, `schedFoldPowBits`, `schedNumQueries`, `schedOodSamples`, `schedLogFolded`, `schedLogInvRate`, `finalPowBits`, `finalFoldPowBits`, `finalNumQueries`, `finalLogFolded`, `finalLogInvRate`, `params`, `rowsIsBase` [WhirVerifier.sol:923-948, 989-1019]. `commitmentOodSamples`, `schedLogInvRate` and `finalLogInvRate` are decoded and not consumed.
4. word count c_s , then c_s words of CONSTRAINTS (Definition 7.9) [WhirVerifier.sol:313-318].

Definition 7.4 (PROOF grammar). Starting at word offset $4 + c$:

1. *Batch prefix*: length L , then `mainDigest` (raw32), `pvBytes` (blob), `lookupPow`, `permDigest` (raw32), `terminals` (`_raw32Arr`), `quotDigest` (raw32), `randDigest` (raw32, present iff `hasRand`), `oodPow`; the cursor must land exactly L words after the length word; one further word (a round count) is skipped [WhirVerifier.sol:395-419].
2. R round blocks, each: `batchCommitment` (`_blob32`); `boundEvals`, `initOodAnswers`, `initScA`, `initScInf` (`_extArr`); `initScPow` (`_arr`); intermediate rows (limb count, then that many u32 LE canonical limbs); intermediate paths (byte count with bit 31 = pruned flag, the digests, then `prunedLens` if pruned); `roundCommitments` (`_blobArr32`); `oodAnswers` (`_extArr`); `oodAnswerLens`, `powWitnesses` (`_arr`); `scA`, `scInf` (`_extArr`); `scPow`, `scLens`, `scPowLens` (`_arr`); `finalPoly` (`_extArr`); `finalPowWitness`; final rows; final paths; `finalScA`, `finalScInf` (`_extArr`); `finalScPow` (`_arr`) [WhirVerifier.sol:951-987, 1021-1103].

Definition 7.5 (STATEMENT grammar). Byte-packed, parsed only on the satellite: u32 R' ; per round u32 n_{mats} ; per matrix u32 `log_size`, u32 `width`, u32 n_{pts} ; per point a u32 length word followed by one 32-byte packed $\mathbb{K}$ element z . The point is read at matrix offset $16 + 36q$; the length word is not checked; the consumed round index must be $< R'$ and the slice must lie inside the satellite frame [TerminalWeight.sol:120-138, 186-188, 205-218].

Property 7.1 (What fixes each section). The header counts c and f locate PROOF and STATEMENT only. The two length-prefixed batch prefixes are self-checked; the round schedules, round blocks and CONSTRAINTS are read at cursor positions that are not compared with $4 + c$ or $4 + c + f$, and individual reads are `calldataloads` at computed offsets [WhirVerifier.sol:261-318, 328-349]. CONFIG is read from the caller's bundle and is not compared with any value held by the deployed contract: the engine has no CONFIG digest, and the wrapper that pins $H_{\mathbb{K}}(\text{CONFIG})$ (WhirVerifierV6, constructor check [WhirVerifierV6.sol:61-76]) is not the deployed contract [contracts/script/Deploy.s.sol:40-45] [V-01]. STATEMENT is neither absorbed into the transcript nor pinned.

7.2 Statement binding and terminal pre-check

Let $(pv_i)_{i < n}$ be the u32 LE words of `pvBytes`, $n = \lfloor |\text{pvBytes}|/4 \rfloor$, and (st_i) the caller's uint256 statement. `_checkStatement` enforces

$$|st| = n, \quad (st_i \cdot R) \bmod p = pv_i \quad (i < n), \quad R = 2^{32} \bmod p = 33554430 \quad (66)$$

[WhirVerifier.sol:426-444]. The comparison is on residues modulo p , so every $st'_i \equiv st_i \pmod{p}$ passes [M-03].

```
for (uint256 i; i < pvCount; ++i) {
    ...
    if (mulmod(statement[i], MONTGOMERY_R, FIELD_P) != _swapBytes(v)) {
        revert StatementMismatch(i);
    }
}
```

Before the transcript, the proof-supplied LogUp terminals $\tau_0, \dots, \tau_{T-1}$ (packed $\mathbb{K}$) must satisfy

$$\bigoplus_{t < T} \tau_t = 0 \quad (67)$$

where $\oplus$ is `KoalaBearExt4.add` [WhirVerifier.sol:276-280] [KoalaBearExt4.sol:33-51]. T is the length of the PROOF array and is not fixed by CONFIG [V-05]; on non-canonical lanes $\oplus$ differs from addition in $\mathbb{K}$ (Property 2.1, [M-04]).

7.3 Batch transcript

One sponge $\mathcal{S}$ runs the whole verification [WhirVerifier.sol:283-296]. The batch layer [BatchTranscript.sol:77-153] performs, in order:

- S0. **Seed and degrees** (CONFIG). $\text{Obs}_{\mathbb{F}_p}$ of every word of `seedBytes`, then of every word of `degreeBytes`; each word must be $< p$ [BatchTranscript.sol:77-84, 208-218] [KeccakChallenger.sol:32-41].
- S1. **Main** (PROOF). $\text{Obs}_{32}(\text{mainDigest})$, then $\text{Obs}_{\mathbb{F}_p}$ of each of the $\lceil |\text{pvBytes}|/4 \rceil$ wire words of the blob [BatchTranscript.sol:87-93, 208-218]; this is $pv_0, \dots, pv_{n-1}$ when $|\text{pvBytes}| \equiv 0 \pmod{4}$, and otherwise one further (partial) word that (66) does not cover.

- S2. **Preprocessed** (CONFIG). $\text{Obs}_{32}(\text{preDigest})$, unconditionally [BatchTranscript.sol:99-101].
- S3. **Lookup**. If $\text{lookupPowBits} = 0$ then lookupPow must be 0 and nothing is absorbed; otherwise $\text{Grind}_{\text{lookupPowBits}}(\text{lookupPow})$ must accept. Then $\alpha_L \leftarrow \text{Draw}_{\mathbb{K}}$, $\beta \leftarrow \text{Draw}_{\mathbb{K}}$ [BatchTranscript.sol:107-115, 177-192].
- S4. **Permutation** (PROOF). $\text{Obs}_{32}(\text{permDigest})$, $\text{Obs}_{\mathbb{K}}(\tau_t)$ for $t < T$, then $\alpha \leftarrow \text{Draw}_{\mathbb{K}}$ [BatchTranscript.sol:119-129] [SumcheckCore.sol:80-92].
- S5. **Quotient** (PROOF). $\text{Obs}_{32}(\text{quotDigest})$, then $\text{Obs}_{32}(\text{randDigest})$ iff hasRand [BatchTranscript.sol:135-143].
- S6. **OOD point**. Grind step as in S3 with oodPowBits , oodPow ; then $\zeta \leftarrow \text{Draw}_{\mathbb{K}}$ [BatchTranscript.sol:146-153].

S0–S6 correspond to T0–T7 of §6.5: S0 covers T0 and T1, and S_k corresponds to $T(k+1)$ for $1 \leq k \leq 6$. Only the randomisation digest is conditional; S2, S3 and S4 run regardless of whether the shape has preprocessed columns or lookups. The engine then hands $\mathcal{S}$ to the WHIR core without reseeding [WhirVerifier.sol:295-296]. The challenges $(\alpha_L, \beta, \alpha, \zeta)$ are consumed only by the constraint identity (§7.6).

7.4 WHIR opening rounds

For $r = 0, \dots, R-1$ the engine decodes Σ_r and the r -th round block and runs one opening [WhirVerifier.sol:341-348, 465-517]. Each opening starts its own claim; only $\mathcal{S}$ is continuous. **framingHex** is a byte string of pre-encoded wire words (“framing”); *absorb n framing words* appends the next $4n$ bytes of it verbatim, reverting if it is exhausted, with no range check [WhirVerifierCore.sol:75-88] [KeccakChallenger.sol:113-145]. No check requires the framing to be fully consumed.

Definition 7.6 (Sumcheck replay). $\text{SC}_b(\sigma; (a_j, h_j)_{j < n}; (w_j))$ requires $|h| = |a| = n$ and $|w| = n$ if $b > 0$, $|w| = 0$ otherwise; then for each j : $\text{Obs}_{\mathbb{K}}(a_j)$, $\text{Obs}_{\mathbb{K}}(h_j)$, if $b > 0$ $\text{Grind}_b(w_j)$ must accept, $r_j \leftarrow \text{Draw}_{\mathbb{K}}$, and

$$\sigma \leftarrow a_j(1 - r_j) + (\sigma - a_j)r_j + h_j r_j(r_j - 1). \quad (68)$$

It returns $(\sigma, (r_j)_{j < n})$ [SumcheckCore.sol:108-149, 165-216]. The round count n is the length of the proof arrays.

Definition 7.7 (Opened row, leaf and fold). A row of 2^m elements of $\mathbb{K}$ (or of $\mathbb{F}_p$ when **rowsIsBase**) arrives as u32 LE canonical limbs v_t , four per $\mathbb{K}$ element; each must satisfy $v_t < p$. Its leaf is

$$\lambda = H_{\mathbb{K}}(\text{le}_{32}(\text{mont}(v_0)) \parallel \dots \parallel \text{le}_{32}(\text{mont}(v_{L-1}))) \quad (69)$$

[WhirVerifierCore.sol:1047-1122] [TerminalWeight.sol:576-615]. A path of depth ℓ for index ι with siblings $s_0, \dots, s_{\ell-1}$ authenticates against root C iff $x_{\ell} = C$ where $x_0 = \lambda$ and $x_{j+1} = H_{\mathbb{K}}(s_j \parallel x_j)$ if bit j of ι is set, else $H_{\mathbb{K}}(x_j \parallel s_j)$ [StarkMerkle.sol:125-167]. The fold at $\mathbf{u} = (u_0, \dots, u_{m-1})$ is the multilinear evaluation with u_0 binding the most significant index bit:

$$\text{Fold}(e, \mathbf{u}) = \sum_{b \in \{0,1\}^m} e_b \prod_{t < m} (b_t u_t + (1 - b_t)(1 - u_t)), \quad b = (b_0, \dots, b_{m-1}) \text{ MSB first} \quad (70)$$

[StirOpenings.sol:175-192] [KoalaBearExt4.sol:212-243].

Initial phase [WhirVerifierCore.sol:259-331]. Inputs: virtual answers $y_0, \dots, y_{m_0-1}$ (**initOodAnswers**), bound evaluations $\mathbf{b}$ (**boundEvals**), widths (w_c) and **claimPerm** from Σ_r .

1. For $i < m_0$: absorb **framingPre**[i] framing words, $v_i \leftarrow \text{Draw}_{\mathbb{K}}$, $\text{Obs}_{\mathbb{K}}(y_i)$.
2. For each claim c : absorb **framingClaim**[c] framing words, then $\text{Obs}_{\mathbb{K}}$ of its w_c evaluations; reverts if they do not fit in $\mathbf{b}$. No opening point is absorbed.
3. Absorb **framingBatching** framing words; $\gamma_0 \leftarrow \text{Draw}_{\mathbb{K}}$.
4. With $N = |\mathbf{b}|$ and ϖ the evaluation order that **claimPerm** and (w_c) induce from placement order to proof order (the identity when **claimPerm** is empty; positions it does not fill map to 0) [WhirVerifierCore.sol:202-227],

$$\sigma_0 = \sum_{k < N} b_{\varpi(k)} \gamma_0^k + \sum_{i < m_0} y_i \gamma_0^{N+i}. \quad (71)$$

5. Absorb `framingSeps[0]` framing words; then, with $b_0^{\text{sc}} = \text{startingPowBits}$,

$$(\sigma'_0, \mathbf{r}^{(0)}) = \text{SC}_{b_0^{\text{sc}}}(\sigma_0; \text{initScA}, \text{initScInf}; \text{initScPow}). \quad (72)$$

$m_0, |\mathbf{b}|$ and the sumcheck length are proof array lengths; m_0 is bounded only by `|framingPre|` and $\sum_c w_c = N$ is not enforced [M-05]. The opening points of the claims are not inputs here; they enter only the terminal weight (Definition 7.8).

Intermediate round $i < M$ [WhirVerifierCore.sol:675-828] [WhirVerifier.sol:552-637]. Schedule from Σ_r : OOD count m_i , query PoW bits b_i^q , query count Q_i , depth ℓ_i (`schedLogFolded`), sumcheck PoW bits b_i^{sc} , arity $k_{i+1} = \text{params}[4i]$. Let C_i be `roundCommitments[i]`, C_{-1} the round block's `batchCommitment`, and $\mathbf{u} = \mathbf{r}^{(i)}$ the previous sumcheck randomness.

1. Shape checks before any absorb: $b_i^q = 0 \Rightarrow \text{witness} = 0$; $|\mathbf{y}_i| = m_i$; row element count $= 2^{|\mathbf{u}|}$; limbs per row $\in \{2^{|\mathbf{u}|}, 4 \cdot 2^{|\mathbf{u}|}\}$ [WhirVerifierCore.sol:685-707]. The row-buffer length compared there is computed by the engine as Q_i times the row width [WhirVerifier.sol:583-585]; the wire limb count of the intermediate rows only advances the decoder cursor and is not compared with it [WhirVerifier.sol:1033-1044].
2. $\text{Obs}_{32}(C_i)$; for $j < m_i$: $z_{i,j} \leftarrow \text{Draw}_{\mathbb{K}}, \text{Obs}_{\mathbb{K}}(y_{i,j})$.
3. $\text{Grind}_{b_i^q}(\cdot)$ must accept.
4. $\iota_{i,q} \leftarrow \text{Bits}_{\ell_i}$ for $q < Q_i$.
5. $\gamma_{i+1} \leftarrow \text{Draw}_{\mathbb{K}}$. For each q , the opened row $e_{i,q}$ authenticates at $\iota_{i,q}$ with depth ℓ_i against C_{i-1} , and $f_{i,q} = \text{Fold}(e_{i,q}, \mathbf{u})$. Then

$$\sigma_{i+1} = \sigma'_i + \sum_{j < m_i} y_{i,j} \gamma_{i+1}^{1+j} + \sum_{q < Q_i} f_{i,q} \gamma_{i+1}^{1+m_i+q}. \quad (73)$$

6. Absorb `framingSeps[1+i]` framing words; $(\sigma'_{i+1}, \mathbf{r}^{(i+1)}) = \text{SC}_{b_i^{\text{sc}}}(\sigma_{i+1}; \dots)$ on the round's slices of `scA`, `scInf`, `scPow`, whose lengths are `scLens[i]`, `scPowLens[i]`.

The per-round slices of `oodAnswers`, `scA`, `scInf` and `scPow` are cut at running offsets by `_slice`, which does not compare offset plus length with the length of the source array [WhirVerifier.sol:569-571, 605-611, 1297-1306]. In step 5, when `prunedLens[i] ≠ 0` (a v8 pruned path stream), the engine requires $2^{|\mathbf{u}|} = 16$, $|\mathbf{u}| = 4$ and calldata rows, draws γ_{i+1} , and delegates the row decode, leaves, folds, the sum σ_{i+1} and the authentication of all $(\iota_{i,q}, \lambda_{i,q})$ to the satellite's QFOLD frame in one call [WhirVerifierCore.sol:749-761, 592-646]; the transcript order is unchanged because the query loop absorbs nothing. Without a pruned stream each query is authenticated on its own path, then γ_{i+1} is drawn [WhirVerifierCore.sol:762-814]. The QFOLD walk sorts the indices, keeps one leaf per distinct index, consumes boundary siblings from the stream level by level, and requires the stream to be consumed exactly, a single remaining node, and that node equal to C_{i-1} [TerminalWeight.sol:648-737]; the folds of all Q_i queries, including repeated indices, enter σ_{i+1} [TerminalWeight.sol:637-647]. Rows of repeated indices other than the retained one are not compared with it [V-04]. The commitment C_{-1} that round 0 opens against is taken from the round block, is not itself absorbed into the transcript, and is not compared with `mainDigest`, `quotDigest`, `preDigest` or `permDigest` absorbed in S1-S5 [WhirVerifier.sol:567-568, 670, 1027] [V-03].

Final phase [WhirVerifierCore.sol:938-1040] [WhirVerifier.sol:653-699]. Schedule: PoW bits $b_\star^q$, query count $Q_\star$, depth $\ell_\star$, sumcheck PoW bits $b_\star^{\text{sc}}$. Let $C_\star = C_{M-1}$ (C_{-1} if $M = 0$), $\mathbf{u} = \mathbf{r}^{(M)}$, and $\phi = (\phi_0, \dots, \phi_{d-1}) = \text{finalPoly}$.

1. Shape checks as above; final rows are always extension rows and their limb count must equal $Q_\star \cdot 4 \cdot 2^{|\mathbf{u}|}$ [WhirVerifierCore.sol:945-966] [WhirVerifier.sol:676-680].
2. $\text{Obs}_{\mathbb{K}}(\phi_j)$ for $j < d$; $\text{Grind}_{b_\star^q}$ must accept; $\iota_q \leftarrow \text{Bits}_{\ell_\star}$ for $q < Q_\star$.
3. $x_q = \omega_{\ell_\star}^{\iota_q}$ (computed, not read) [WhirVerifierCore.sol:987-992] [WhirVerifier.sol:685]. Each row authenticates against $C_\star$ on its own path, and

$$\text{Fold}(e_q, \mathbf{u}) = \sum_{j < d} \phi_j x_q^j \quad (74)$$

(Horner) [WhirVerifierCore.sol:998-1026] [StirOpenings.sol:201-207]. These claims are checked directly and not batched into the running claim.

4. Absorb `framingSeps[1 + M]` framing words; $(\sigma_*, \mathbf{r}^*) = \text{SC}_{b^{\text{sc}}}(\sigma'_M; \text{finalScA}, \text{finalScInf}; \text{finalScPow})$. d and $|\mathbf{r}^*|$ are proof array lengths, tied to each other by the terminal identity below [M-05].

7.5 Terminal identity

Definition 7.8 (Constraint weights). Round r yields $M + 1$ weight descriptors [WhirVerifier.sol:501-513, 619-632]. For a descriptor with arity k , challenge γ , shift ε , equality values $E_0, \dots, E_{G-1}$ and selection points $x_0, \dots, x_{Q-1}$,

$$w(\rho) = \gamma^\varepsilon \left(\sum_{g < G} \gamma^g E_g(\rho) + \gamma^G \sum_{q < Q} \gamma^q \text{Sel}(\rho, x_q) \right), \quad \text{Sel}(\rho, x) = \prod_{t < k} (\rho_{k-1-t}(x^{2^t} - 1) + 1) \quad (75)$$

[WhirGadgets.sol:618-767, 525-587], with $\text{eq}(\mathbf{p}, \mathbf{q}) = \prod_t (1 + 2p_t q_t - p_t - q_t)$ and $\text{exp}_k(v) = (v^{2^{k-1}}, \dots, v^2, v)$ [WhirGadgets.sol:119-167].

- *Initial* ($\varepsilon = 0$, $\gamma = \gamma_0$, $k = k_0$, $Q = 0$). The satellite derives the equality groups from round r of STATEMENT and the virtual points [TerminalWeight.sol:125-203, 1357-1368]. Matrices are placed as in Definition 6.3: arity $a = \max(\log_size, 4)$, required $a \leq k_0$; stable sort ascending, walked in reverse; each column takes slot index $s = \text{bitrev}_{k_0-a}(o \gg a)$ with o advancing by 2^a per column. Groups are emitted per matrix, per point z , per column; for such a group, with $c_t = z^{2^{a-1-t}}$,

$$E(\rho) = \prod_{t < a} \frac{1 - \rho_t + \rho_t c_t}{1 + c_t} \cdot \prod_{j < k_0 - a} [\text{bit}_{k_0 - a - 1 - j}(s) ? \rho_{a+j} : 1 - \rho_{a+j}], \quad (76)$$

i.e. $\text{eq}(\rho, (X(z), \text{bits}(s)))$ with the bridge point $X(z)$ of Definition 6.4 [WhirGadgets.sol:203-337]. The m_0 virtual groups follow with $E = \text{eq}(\rho, \text{exp}_{k_0}(v_i))$.

- *Round* $i < M$ ($\varepsilon = 1$, $\gamma = \gamma_{i+1}$, $k = k_{i+1}$): $E_j = \text{eq}(\cdot, \text{exp}_k(z_{i,j}))$ and $x_q = \omega_{\ell_i}^{i,q}$ lifted to $\mathbb{K}$.

Property 7.2 (Terminal identity). Let $\mathbf{r}^{\text{all}} = \mathbf{r}^{(0)} \parallel \mathbf{r}^{(1)} \parallel \dots \parallel \mathbf{r}^{(M)} \parallel \mathbf{r}^*$ and let ρ_c be its last k_c entries (reverting if $k_c > |\mathbf{r}^{\text{all}}|$). The engine requires

$$\sigma_* = \left(\sum_{c=0}^M w_c(\rho_c) \right) \cdot \text{Fold}(\phi, \mathbf{r}^*), \quad |\phi| = 2^{|\mathbf{r}^*|}, \quad (77)$$

where the weight and $\text{Fold}(\phi, \cdot)$ are computed by the satellite's TWIGHT frame and the equality is evaluated in the engine [WhirVerifier.sol:701-726] [TerminalWeight.sol:1370-1371] [WhirGadgets.sol:782-806] [KoalaBearExt4.sol:217-219].

```
uint256[] memory allR = _concat(allRandomness, closing);
(uint256 weight, uint256 value) =
    _terminalWeight(allR, constraints, finalPoly, closing);
uint256 expected = KoalaBearExt4.mul(weight, value);
if (foldedClaim != expected) {
    revert TerminalClaimMismatch(expected, foldedClaim);
}
```

The query points x_q are derived from transcript indices. The opening points z in the initial weight are read from STATEMENT and are not compared with ζ or $\zeta \omega_\ell$, nor bound by the transcript [V-02].

7.6 Constraint identity

After all R openings the engine sends the CIDNTY frame

$$[\zeta, \alpha, \alpha_L, \beta, (\tau_t)_{t < T}, (st_i)_i, \text{CONSTRAINTS}, (\mathbf{b}^{(r)})_{r < R}] \quad (78)$$

to the satellite, where $\mathbf{b}^{(r)}$ is the `boundEvals` array of opening r [WhirVerifier.sol:302-318, 1324-1381]; the satellite reverts on any mismatch [TerminalWeight.sol:998-1194].

Definition 7.9 (CONSTRAINTS section). u32 LE words: instance count n ; statement instance κ ; per instance i : widths $(w_i, w_i^{\text{pre}}, w_i^{\text{aux}})$, flags `hasMainNext`, `hasPreNext`, a skipped constraint count, a node program (word count, then 12-byte nodes $[\text{op}, x, y]$), base and extension constants, roots $(\varrho_g)_{g < K_i}$, trace ℓ_i , a skipped shift, is_i (inverse shift), hi_i (`traceHInv`), chunk count L_i and per chunk $(\ell_{i,j}, \text{skipped}, is_{i,j})$, and $\text{invD}_{i,:}$; then W (`maxMsgW`), bus ids per instance, `hasTerminal` per instance, n_r and per-round arity arrays $A^{(r)}$. The parse must end exactly at c_s words and the frame exactly after n_r bound sections [TerminalWeight.sol:1015-1097].

Definition 7.10 (Opened values). With $s_a(z) = \prod_{t < a} (1 + z^{2^t})$ [TerminalWeight.sol:907-915] and $\zeta'_i = \zeta \cdot \omega_{\ell_i}$ [TerminalWeight.sol:1100-1103], the bound sections are read by role: $r = 0$ main, $r = 1$ quotient, $r = 2$ preprocessed, $r \geq 3$ permutation [TerminalWeight.sol:794-861, 1139-1175].

- Main ($r = 0$) and preprocessed ($r = 2$): per instance one claim at ζ and, if the corresponding **Next** flag is set, one at ζ'_i , each of width w_i (resp. w_i^{pre}) and arity $A_i^{(r)}$; values $s_a(z) \cdot b$.
- Quotient ($r = 1$): L_i claims of width 4 at ζ , arity $A_j^{(1)}$ indexed by the global claim index j ; chunk value $q_{i,j} = \sum_{k < 4} X^k s_a(\zeta) b_k$.
- Permutation ($r \geq 3$): per instance claims at ζ and ζ'_i of width $4w_i^{\text{aux}}$, recombined in groups of four as for the quotient.
- Challenges: per bus id id of instance i , $(\alpha_L + (id + 1) \beta^{\max(W,1)}, \beta)$ [TerminalWeight.sol:1122-1134].
- Terminals: the t -th instance with **hasTerminal** receives τ_t ; no check requires all T terminals to be consumed [TerminalWeight.sol:1114-1121] [V-05].
- Public values of instance κ : the caller's words st_x , lifted as $st_x \ll 224$ (low 32 bits) [TerminalWeight.sol:1135-1137] [ConstraintIdentity.sol:250-254]; with (66) this admits the aliasing of [M-03].

Property 7.3 (Constraint identity). For each instance $i < n$, with $u = \zeta \cdot is_i$ and $Z = u^{2^{\ell_i}} - 1$, selectors $isFirst = Z/(u-1)$, $isLast = Z/(u-hi_i)$, $isTransition = u-hi_i$ [ConstraintIdentity.sol:119-133]; constraint fold $F_i = \sum_{g < K_i} \alpha^{K_i-1-g} C_{i,g}$ (Horner over the roots, the DAG evaluated on the opened values and selectors) [ConstraintIdentity.sol:138-272]; and

$$Q_i = \sum_{j < L_i} q_{i,j} \cdot invD_{i,j} \cdot \prod_{j' \neq j} \left((\zeta \cdot is_{i,j'})^{2^{\ell_i,j'}} - 1 \right) \quad (79)$$

[ConstraintIdentity.sol:276-310], the satellite requires

$$F_i \cdot Z^{-1} = Q_i \quad (80)$$

and reverts with **ConstraintIdentityMismatch**(i) otherwise [TerminalWeight.sol:1177-1187]. Inversion of 0 reverts [KoalaBearExt4.sol:96-97].

This is Definition 6.10 with the Lagrange factors' denominators $invD$ and all domain constants supplied by **CONSTRAINTS** (**CONFIG**, [V-01]). The bound evaluations it consumes are the values the **WHIR** openings proved at the **STATEMENT** points, while the identity scales them at ζ and ζ'_i [V-02]; the trace, quotient, preprocessed and permutation roles are assigned by round index, and the roots those rounds opened against are the per-round **batchCommitments** [V-03].

7.7 The satellite and its pin

TerminalWeight dispatches on the first calldata word: **QFOLD**, **MROOTS**, **CIDNTY**, else **TWIGHT**, which must match its magic and end exactly at **calldatasize** [TerminalWeight.sol:1200-1355]. The engine stores the satellite's address and code hash at construction (rejecting empty code) and re-checks **codehash** before every call, both in the engine and in the core's **QFOLD** path [WhirVerifier.sol:126-132, 888-916] [WhirVerifierCore.sol:652-673]. A satellite revert is bubbled. For **TWIGHT** the final equality is evaluated in the engine; for **QFOLD** the Merkle comparison is inside the satellite and its returned sum is used as σ_{i+1} ; for **CIDNTY** the satellite's return is the verdict. The engine never builds an **MROOTS** frame.

Source	Values
CONFIG (caller bundle)	seed and degree words, preDigest , PoW bits, hasRand , R , every Σ_r (query and OOD counts, depths, arities, framing words, widths, claimPerm), CONSTRAINTS (programs, domain constants, $invD$, role layout) [V-01]
STATEMENT (caller bundle)	WHIR opening points and matrix shapes of every initial weight [V-02]
PROOF	digests, public values, terminals and their count [V-05], round roots incl. batchCommitment [V-03], evaluations, OOD answers, sumcheck messages and their counts, rows, paths, finalPoly [M-05]
Derived in contract	$\alpha_L, \beta, \alpha, \zeta$; virtual and OOD points; γ_r ; sumcheck randomness; query indices; $x_q = \omega_{\ell}^t$

7.8 Spec deltas

Spec delta 7.1 (STATEMENT is consumed). The engine header calls STATEMENT an “audit surface, not consumed here” [WhirVerifier.sol:16-17]; it is the sole source of the initial-weight opening points [WhirVerifier.sol:510-513] [TerminalWeight.sol:125-203]. The header also states that every domain point is recomputed and none is read from the proof [WhirVerifier.sol:31-35]; that holds for query points only [V-02].

Spec delta 7.2 (Wire description). The header describes a “16-byte header followed by three word sections” under “v3” and the version error is documented as “not version 3” [WhirVerifier.sol:13-19,62-63]; the code accepts {5,7,8}, reads one byte of version, and places a u32 STATEMENT count between PROOF and STATEMENT [WhirVerifier.sol:242-257].

Spec delta 7.3 (CONFIG pin). “A deployment should pin keccak256(configSection)” [WhirVerifier.sol:23-25], the preprocessed digest is described as a “deployment constant” [BatchTranscript.sol:95-98], and the unchecked framing absorb is justified by the WhirVerifierV6 digest [WhirVerifierCore.sol:80-86]; the deployed engine is the unwrapped WhirVerifier [contracts/script/Deploy.s.sol:40-45] [V-01].

Spec delta 7.4 (Satellite trust statement). The engine and satellite state that a faulty satellite “can only make verification FAIL” [WhirVerifier.sol:97-100] [TerminalWeight.sol:49-55]; this describes TWIGHT only. QFOLD and CIDNTY results are accepted as returned, so the code-hash pin is what fixes them (§7.7). The _callSatellite comment names the MROOTS walk as a caller [WhirVerifier.sol:881-887]; no engine path builds that frame.

Spec delta 7.5 (Minor). TerminalWeight.MONT is commented as $2^{31} \bmod p$ while its value 0x01FFFFFFE is $2^{32} \bmod p$ [TerminalWeight.sol:67-68]. StirOpenings documents per-query paths as the verification strategy [StirOpenings.sol:59-67]; intermediate rounds of v8 bundles use the pruned walk. BatchTranscript.lookupPair has no caller outside a unit test [contracts/test/BatchTranscriptNative.t.sol:87]; the lookup challenge pairs are recomputed in CIDNTY, which uses $\beta^{\max(W,1)}$ where lookupPair uses β^W (the two differ only at $W = 0$) [BatchTranscript.sol:155-172] [TerminalWeight.sol:1122-1134].

8 The settlement contract

The L1 component that holds the rollup state is ShieldedPool [contracts/src/ShieldedPool.sol:63]. It stores a block counter, two 32-byte roots and a fee-recipient address, delegates proof verification to an injected verifier, checks root continuity itself, and records the roots that a verified block statement names. It derives no root itself, holds no nullifier set and no note data. [contracts/src/ShieldedPool.sol:7-26,57-62] This section states the state it holds, the transition applyBlock enforces (in execution order), the event it emits, the fee path, the deployment performed by Deploy.s.sol, and the calldata and transaction the node sends to reach it. The statement vector S and its decoding are those of Def. 5.8 and Def. 5.11; the verifier’s binding of S to the proof is specified in Section 7.

8.1 Contract state

Definition 8.1 (Pool state). A ShieldedPool instance at address A on a chain with identifier χ is the tuple

$$\mathcal{P} = (V, f; N, R, R^{\text{nf}}),$$

with the following components.

Symbol	Solidity name	Type	Kind	Written by
V	verifier	IWhirVerifier	immutable	constructor only [contracts/src/ShieldedPool.sol:68,131]
f	feeRecipient	address	storage	constructor only [contracts/src/ShieldedPool.sol:89,132]
N	blockNumber	uint256	storage	applyBlock (initially 0) [contracts/src/ShieldedPool.sol:71,168-170]
R	currentRoot	bytes32	storage	constructor, applyBlock [contracts/src/ShieldedPool.sol:74,133,165]
R^{nf}	currentNullifierRoot	bytes32	storage	constructor, applyBlock [contracts/src/ShieldedPool.sol:77,134-135,166]
E^{nf}	EMPTY_NULLIFIER_ROOT	bytes32	constant	— [contracts/src/ShieldedPool.sol:85-86]

All five state variables and the constant are `public`, so each has an auto-generated getter. The view selectors for N , R , R^{nf} are pinned off-chain as 0x57e871e7, 0xfdb463d, 0x222d1bed. [contracts/test/SelectorPins.t.sol:14-18]

Property 8.1 (Write surface). Besides the constructor, the contract declares exactly two external functions, `applyBlock` and `withdrawFees` [contracts/src/ShieldedPool.sol:144,182]. Only `applyBlock` writes (N, R, R^{nf}) ; nothing writes f after construction, and V is immutable. There is no owner, no pause, no upgrade path and no proxy. No function is payable, and there is no `receive` or `fallback` function. [contracts/src/ShieldedPool.sol:63-190]

8.2 Construction

Definition 8.2 (Constructor). `constructor(V, f, r0, r0nf)` enforces $V \neq 0$ and $f \neq 0$ (string reverts), then sets

$$R := r_0, \quad (81)$$

$$R^{\text{nf}} := \begin{cases} E^{\text{nf}} & \text{if } r_0^{\text{nf}} = 0^{32}, \\ r_0^{\text{nf}} & \text{otherwise,} \end{cases} \quad (82)$$

and leaves $N = 0$. [contracts/src/ShieldedPool.sol:123-136] No check is applied to r_0 or to a nonzero r_0^{nf} , and V is not checked to have code. A wrong genesis pair is not rejected at construction; it makes every block whose statement was witnessed against the true genesis fail the continuity checks of Def. 8.3. [contracts/src/ShieldedPool.sol:120-122]

Spec delta 8.1 (Empty nullifier root). The constant is

```
EMPTY_NULLIFIER_ROOT =
    0x83265b7a2c40cc6eca237b6016b764606335cd0c6ff0732470e20a484d5a7329
```

documented as the Poseidon2 empty-subtree chain at depth `NULLIFIER_TREE_DEPTH = 96` [contracts/src/ShieldedPool.sol:79-86] [crates/shielded/src/nullifier_tree.rs:64], and it equals the `nullifier_before_hex` of the settled-block vector [contracts/test/vectors/block_vectors.json:4] (see Section 3.6). The constructor documentation instead states that a zero argument makes the contract *compute* a depth-256 Keccak empty root at deploy time [contracts/src/ShieldedPool.sol:114-118]; the code performs no hashing and selects the pinned constant [contracts/src/ShieldedPool.sol:134-135] [I-02].

8.3 applyBlock: the state transition

Definition 8.3 (`applyBlock`). `applyBlock(uint256[] calldata S, bytes calldata  $\pi$ )` is external, non-payable, and has no caller restriction. [contracts/src/ShieldedPool.sol:144] It executes the following steps in order; any failing step reverts the whole call, so no step's effect persists unless all steps succeed.

#	Step	Source
1	$b := V.\text{verify}(S, \pi)$. The interface declares <code>verify</code> view, so the call is a static call; a revert inside V propagates. If $b = \text{false}$, revert <code>NotVerified()</code> .	[ShieldedPool.sol:145] [IWhirVerifier.sol:21-23]
2	$B := \text{decode}(S)$ after copying S from calldata to memory; all checks of Def. 5.11 apply (string reverts).	[ShieldedPool.sol:147-149]
3	require $B.\text{rootBefore} = R$, else revert <code>RootMismatch(R, B.rootBefore)</code> .	[ShieldedPool.sol:152-154]
4	require $B.\text{nullifierBefore} = R^{\text{nf}}$, else revert <code>NullifierRootMismatch(R^{nf}, B.nullifierBefore)</code> .	[ShieldedPool.sol:155-157]
5	$R := B.\text{rootAfter}$; $R^{\text{nf}} := B.\text{nullifierAfter}$.	[ShieldedPool.sol:165-166]
6	$N := N + 1$ (in an unchecked block).	[ShieldedPool.sol:168-170]
7	emit <code>BlockApplied(N, B.rootBefore, B.rootAfter, B.nullifierBefore, B.nullifierAfter, B.totalFee)</code> with the incremented N .	[ShieldedPool.sol:171-178]

The guard portion of the function (comments and blank lines omitted):

```
if (!verifier.verify(statement, proof)) revert NotVerified();
BlockStatement.Block memory block_ = statement.decode();
if (block_.rootBefore != currentRoot) {
```

```

    revert RootMismatch(currentRoot, block_.rootBefore);
}
if (block_.nullifierBefore != currentNullifierRoot) {
    revert NullifierRootMismatch(currentNullifierRoot, block_.nullifierBefore);
}

```

[contracts/src/ShieldedPool.sol:145-157]

Writing $D_j(S) = \text{digestFromLimbs}(S, 1 + 2n + 16j)$ for $j = 0, \dots, 4$ (Def. 5.9), the enforced relation is:

Definition 8.4 (Settlement relation). For a pool state $(V, f; N, R, R^{\text{nf}})$, $\text{applyBlock}(S, \pi)$ succeeds if and only if

$$V.\text{verify}(S, \pi) = \text{true} \text{ (without reverting)}, \quad (83)$$

$$\text{decode}(S) \text{ succeeds, i.e. } 1 \leq S[0] = n \leq 2^{16} - 1, |S| = 81 + 6n, \text{ all decoded limbs } \leq 0\text{xffff}, \quad (84)$$

$$D_1(S) = R \text{ and } D_3(S) = R^{\text{nf}}, \quad (85)$$

and on success the post-state is

$$(N, R, R^{\text{nf}}) \mapsto (N + 1, D_2(S), D_4(S)). \quad (86)$$

[contracts/src/ShieldedPool.sol:144-179] [contracts/src/BlockStatement.sol:79-112] Apart from whatever V reads internally, no other input, state variable or environment value (`msg.sender`, `tx.origin`, `block.chainid`, `address(this)`, `block.number`, `msg.value`) enters the relation.

Property 8.2 (Decoded but unused fields). Of the decoded `Block`, `numTransfers` and `statementRoot` = $D_0(S)$ are not read by `applyBlock`, and `totalFee` is used only as an event argument. [contracts/src/ShieldedPool.sol:149-178] The header limbs $S[1], \dots, S[2n]$ are not read by `decode` or `applyBlock` (Def. 5.11); on L1 they, like the fold root, are compared only by the verifier’s statement check inside step 1 [contracts/src/verifier/WhirVerifier.sol:268,426-440]. The fold root is therefore constrained on L1 only through (83) [L-04].

Property 8.3 (Caller). `applyBlock` contains no `msg.sender` comparison, modifier or allow-list; every account may call it, and the only gate is (83)–(85). [contracts/src/ShieldedPool.sol:144-179] The adequacy of this gate is examined in [H-01].

Property 8.4 (Statement context). The statement layout (Def. 5.8) carries the transfer count, the shape header, the fold root, the four chain endpoints and the fees, and no chain identifier, pool address, block height or other per-deployment value. [contracts/src/BlockStatement.sol:11-15,79-112] The only binding between a pair (S, π) and a particular pool instance is (85), i.e. equality of $D_1(S), D_3(S)$ with that instance’s current roots [M-01].

Property 8.5 (Re-submission on the same instance). After (S, π) has been applied, $R = D_2(S)$ and $R^{\text{nf}} = D_4(S)$. A second `applyBlock`(S, π) on the same instance passes (85) only if $D_2(S) = D_1(S)$ and $D_4(S) = D_3(S)$, i.e. only for a block that leaves both roots unchanged. [contracts/src/ShieldedPool.sol:152-157, 165-166]

Property 8.6 (Check order and cost). Steps 2–4, which read only S and two storage slots, run after step 1. A statement that is malformed (fails (84)) or stale (fails (85)) is therefore rejected only after the verifier has executed. The ordering is documented as deliberate. [contracts/src/ShieldedPool.sol:137-145] The node submits each settlement with a gas ceiling of $2 \cdot 10^{10}$, documented as matching the dev-chain runs, with the note that the verifier’s cost is data-dependent. [crates/node/src/settlement.rs:28-31]

Property 8.7 (Fee handling in the transition). $\text{totalFee} = \sum_{c=1}^n \text{u64FromLimbs}(S, 81 + 2n + 4(c - 1))$ is accumulated in checked `uint256` arithmetic [contracts/src/BlockStatement.sol:104-107]; `applyBlock` moves no value and credits no account with it, it only emits it. [contracts/src/ShieldedPool.sol:171-178] The contract does not check fee correctness; it relies on the transfer relation’s value conservation. [contracts/src/ShieldedPool.sol:59-62]

8.4 The BlockApplied event

Definition 8.5 (Event).

```

event BlockApplied(
    uint256 indexed blockNumber,
    bytes32 rootBefore, bytes32 rootAfter,
    bytes32 nullifierRootBefore, bytes32 nullifierRootAfter,
    uint256 fee);

```

[contracts/src/ShieldedPool.sol:91-99] The log has topic_0 = the event signature hash, $\text{topic}_1 = N$ (the post-increment value, so the first applied block logs 1), and five data words $D_1(S), D_2(S), D_3(S), D_4(S), \text{totalFee}$. [contracts/src/ShieldedPool.sol:168-178]

Property 8.8 (Event content). The event omits the fold root $D_0(S)$, the transfer count n , the shape header, the per-transfer fees, and the submitting account. Neither the event nor S carries per-transfer nullifiers or output commitments: on L1 these appear only as absorbed into the fold root (Def. 5.7) [L-04]. [contracts/src/ShieldedPool.sol:91-99, 171-178] [contracts/src/BlockStatement.sol:17-24]

8.5 withdrawFees

Definition 8.6 (`withdrawFees`). `withdrawFees()` requires `msg.sender = f`, else reverts `NotFeeRecipient()`; with $a := \text{address(this).balance}$, if $a > 0$ it performs `f.call{value: a}("")` and requires success. [contracts/src/ShieldedPool.sol:181-189]

Spec delta 8.2 (Fee accrual). f is documented as the recipient of “collected fees” [contracts/src/ShieldedPool.sol:88-89]. By Property 8.1 and Property 8.7, no function of the contract accepts value and `applyBlock` credits nothing, so no code path of the system funds the balance that `withdrawFees` pays out (a nonzero balance can arise only from value delivered without a call, e.g. to a `SELFDESTRUCT` beneficiary or to an address funded before deployment); the shielded fee φ of each transfer has no on-chain counterpart in this contract [I-04].

8.6 Deployment

Definition 8.7 (Deployment script). `Deploy.run()` performs, in order:

#	Action	Source
1	Read the genesis file named by <code>GENESIS_FILE</code> , defaulting to <code>test/vectors/block-genesis.json</code> ; $r_0 := \text{its .genesis_root_hex as bytes32}$.	[Deploy.s.sol:28-32]
2	$\text{dep} := \text{msg.sender}$ of the script (the broadcast account).	[Deploy.s.sol:34-36]
3	Broadcast <code>new TerminalWeight() = T</code> .	[Deploy.s.sol:37-40]
4	Broadcast <code>new WhirVerifier(T) = W</code> .	[Deploy.s.sol:41]
5	Broadcast <code>new ShieldedPool(W, dep, r₀, 0³²)</code> , so $f = \text{dep}$ and $R^{\text{nf}} = E^{\text{nf}}$.	[Deploy.s.sol:42-46]
6	Write <code>deployments/local.json</code> with keys <code>verifier</code> , <code>terminalWeight</code> , <code>pool</code> , <code>deployer</code> , <code>chainId</code> .	[Deploy.s.sol:48-57]

The documented invocation targets an `anvil` chain with chain id 31337 and the dev chain’s default funded key. [contracts/script/Deploy.s.sol:4-6]

Property 8.9 (Verifier instance). The constructor of `WhirVerifier` rejects a zero or code-less `terminalWeight` and pins its address (immutable) and code hash; the code hash is re-checked before every satellite call. [contracts/src/verifier/WhirVerifier.sol:116-132, 895] It takes no configuration argument; `verify` decodes the CONFIG section from the caller-supplied bundle π . [contracts/src/verifier/WhirVerifier.sol:209-213, 259-265] The script does not deploy `WhirVerifierV6`, whose constructor pins a CONFIG digest and length [contracts/src/verifier/WhirVerifierV6.sol:41-47, 61-75]; consequently the pool’s immutable V is the bare engine. See Section 7 and [V-01].

Spec delta 8.3 (Explicit genesis nullifier root). The script comments that “the genesis file may pin one explicitly”; the code passes the literal 0^{32} and reads no nullifier root from the genesis file. [contracts/script/Deploy.s.sol:42-45]

8.7 Settlement calldata

The node constructs the call by hand. [crates/node/src/settlement.rs:1-8]

Definition 8.8 (Word encodings). For $x \in [0, 2^{64}]$, $w(x) \in \{0, 1\}^{256}$ is the 32-byte big-endian word with the top 24 bytes zero [crates/node/src/settlement.rs:93-98]. For a sequence $s = (s_0, \dots, s_{L-1})$ of u64 and a byte string b :

$$\text{arr}(s) = w(L) \parallel w(s_0) \parallel \dots \parallel w(s_{L-1}), \quad (87)$$

$$\text{byt}(b) = w(|b|) \parallel b \parallel 0^{8\rho}, \quad \rho = (32 - |b| \bmod 32) \bmod 32. \quad (88)$$

[crates/node/src/settlement.rs:100-116]

Definition 8.9 (applyBlock calldata).

$$\text{cd}(S, \pi) = 0x0cb000b5 \parallel w(0x40) \parallel w(0x40 + 32(L + 1)) \parallel \text{arr}(S) \parallel \text{byt}(\pi), \quad L = |S|, \quad (89)$$

of length $4 + 64 + 32(L + 1) + 32 + |\pi| + \rho$. [crates/node/src/settlement.rs:118-133] The two head words are the ABI offsets of the two dynamic tails measured from the end of the selector; this is the standard ABI encoding of `(uint256[], bytes)`. The selector is `bytes4(keccak256("applyBlock(uint256[], bytes)"))`, pinned by a test that compares it with the EVM’s own `keccak256`. [crates/node/src/settlement.rs:24-26] [contracts/test/SelectorPins.t.sol:10-12] For a block statement, $L = 81 + 6n$ (Def. 5.8).

Property 8.10 (Statement words). The node takes S as the canonical representatives of the block statement’s field elements, widened to `u64`, and π as the settlement bundle encoded from the same recursion artifact and statement. [crates/node/src/main.rs:444-456] Every word of $\text{arr}(S)$ is thus $< p < 2^{32}$; the encoder cannot express `uint256` values $\geq 2^{64}$, and none are needed.

8.8 Submission

Definition 8.10 (Settlement transaction). `SettlementSender::settle( $S, \pi$ )` issues one JSON-RPC `eth_sendTransaction` with the object

Field	Value	Source
<code>from</code>	<code>manifest deployer</code>	[settlement.rs:201]
<code>to</code>	<code>manifest pool</code>	[settlement.rs:202]
<code>data</code>	<code>0x $\parallel$ hex($\text{cd}(S, \pi)$)</code>	[settlement.rs:199, 203]
<code>gas</code>	<code>0x4a817c800 = $2 \cdot 10^{10}$</code>	[settlement.rs:31, 204]

and no `value`, `nonce`, `fee` or `chainId` field; signing and the remaining fields are left to the RPC endpoint, and the node holds no key. [crates/node/src/settlement.rs:10-17, 198-216] On a dev chain the `from` account is an account unlocked on the RPC endpoint (`anvil`). [crates/node/src/settlement.rs:12-17]

Definition 8.11 (Receipt wait). After the send, the sender polls `eth_getTransactionReceipt` with a 2s sleep between polls. A non-null receipt with `status = "0x1"` returns the hash; any other status (absent status read as `"0x0"`) returns `Reverted`. The 600s deadline is evaluated only after a poll that returned a null receipt; a transport or JSON-RPC error on any poll returns that error immediately. [crates/node/src/settlement.rs:148-156, 218-250] The HTTP client is `request::Client::new()` with no request timeout configured, and `settle` is awaited inline by the node’s single command actor, which processes commands sequentially. [crates/node/src/settlement.rs:150] [crates/node/src/main.rs:464-476, 507-509, 780-781] See [I-05].

Property 8.11 (Manifest and retention). The node reads the manifest from `NODE_MANIFEST` (default `contracts/deployments/local.json`) and the RPC URL from `NODE_RPC_URL` (default `http://127.0.0.1:8545`) [crates/node/src/main.rs:125-126]; the `Manifest` type deserializes only `pool` and `deployer`, so the `chainId` the script writes is not consulted [crates/node/src/settlement.rs:70-77]. The node keeps a single pending (S, π) : `settle` clears it only on success and leaves it in place on any error, and the next produced block overwrites it whether or not it was settled. [crates/node/src/main.rs:268, 444-459, 464-476]

Spec delta 8.4 (Settlement-side documentation). (i) `Manifest.pool` is documented as “the `ShieldedPool` proxy address” and the manifest as written by `contracts/scripts/deploy_local.mjs` [crates/node/src/settlement.rs:72, 80-81]; the deployment deploys no proxy and is `Deploy.s.sol`, and no `deploy_local.mjs` exists under `contracts/scripts`. [contracts/script/Deploy.s.sol:13, 45] (ii) The `settle` documentation states that re-sending a block whose first attempt landed “would double-apply nullifiers” [crates/node/src/settlement.rs:190-193]; by Property 8.5, the contract rejects such a re-send at step 3 or 4 unless the block leaves both roots unchanged.

9 The application layer: node, spend authorization, custody

This section specifies what the off-chain components enforce around the transfer relation of Section 4: the byte string a spend authorization signs (Section 9.1), the signature scheme that signs it (Section 9.2), the node’s HTTP admission path and the in-process mempool admission (Sections 9.3–9.4), the bearer-token access control in front of every route (Section 9.5), and the password vault that holds a wallet’s secrets

(Section 9.6). None of these checks is re-executed by the transfer circuit, the block circuit or the settlement contract of Section 8; they hold only for traffic that passes through this node.

Notation. A transfer’s public statement is the tuple

$$x = (N_1, \dots, N_k; C_1, \dots, C_m; rt, rt', R_{\text{before}}, R_{\text{after}}; fee)$$

of `TransferPublic` [crates/shielded/src/transfer.rs:151-176], with each $N_i, C_j, rt, rt', R_{\text{before}}, R_{\text{after}}$ a raw 32-byte digest and $fee \in [0, 2^{64})$. It is the same tuple whose limb flattening is the statement of Section 4.5. $\text{LE}_w(v)$ is the w -bit little-endian encoding of v and $\parallel$ is byte concatenation.

9.1 The spend-authorization message

Definition 9.1 (Signed message). Let `DOM` be the 23 ASCII bytes `pq-rollup/spend-auth/v1 (DOMAIN_TX)` [crates/shielded/src/signing.rs:35] and `MAXC = 1024 (MAX_COUNT_PER_FIELD)` [crates/shielded/src/signing.rs:42]. `encode_statement` is defined iff $k \leq \text{MAXC}$ and $m \leq \text{MAXC}$, otherwise it returns `EncodeError::TooLarge` [crates/shielded/src/signing.rs:78-80]. When defined,

$$M(x) = \text{DOM} \parallel \text{LE}_{32}(k) \parallel N_1 \parallel \dots \parallel N_k \parallel \text{LE}_{32}(m) \parallel C_1 \parallel \dots \parallel C_m \parallel rt \parallel rt' \parallel R_{\text{before}} \parallel R_{\text{after}} \parallel \text{LE}_{64}(fee), \quad (90)$$

emitted in exactly this order [crates/shielded/src/signing.rs:84-97]. The counts are written by `push_count` as `(len as u32).to_le_bytes()` [crates/shielded/src/signing.rs:112-114]; the bound above makes the cast lossless.

Field	Byte offset	Length	Source
DOM	0	23	[crates/shielded/src/signing.rs:84]
$\text{LE}_{32}(k)$	23	4	[crates/shielded/src/signing.rs:85]
$N_i, i = 1..k$	$27 + 32(i - 1)$	32	[crates/shielded/src/signing.rs:86-88]
$\text{LE}_{32}(m)$	$27 + 32k$	4	[crates/shielded/src/signing.rs:89]
$C_j, j = 1..m$	$31 + 32k + 32(j - 1)$	32	[crates/shielded/src/signing.rs:90-92]
rt	$31 + 32(k + m)$	32	[crates/shielded/src/signing.rs:93]
rt'	$63 + 32(k + m)$	32	[crates/shielded/src/signing.rs:94]
R_{before}	$95 + 32(k + m)$	32	[crates/shielded/src/signing.rs:95]
R_{after}	$127 + 32(k + m)$	32	[crates/shielded/src/signing.rs:96]
$\text{LE}_{64}(fee)$	$159 + 32(k + m)$	8	[crates/shielded/src/signing.rs:97]

Hence $|M(x)| = 167 + 32(k + m)$ bytes; 231 for one input and one output. (The inline test comment “domain(24)” is off by one; the assertion itself uses `DOMAIN_TX.len()` [crates/shielded/src/signing.rs:159-160].)

Property 9.1 (Injectivity). On its domain $(k, m \leq 1024)$, $x \mapsto M(x)$ is injective: `DOM` has fixed length, each variable-length list is preceded by its exact count, every list element is exactly 32 bytes, and the trailing $4 \cdot 32 + 8 = 136$ bytes are fixed-width, so a parser recovers $(k, N_*, m, C_*, rt, rt', R_{\text{before}}, R_{\text{after}}, fee)$ uniquely and $|M(x)|$ determines $k + m$. [crates/shielded/src/signing.rs:77-99]

What M does not contain. $M(x)$ carries no chain identifier, pool address or other deployment context [M-01]; it does not contain the verifying key that signs it, the note owner key pk_d , the proof, or the limb statement of Section 4.5. The node module states that it cannot relate a revealed verifying key to a note’s pk_d [crates/node/src/tx.rs:21-28] [M-02]. M is the only input to the signature; the envelope type pairs it with a key and a signature and nothing else [crates/node/src/tx.rs:94-101, 117-131].

9.2 SLH-DSA instantiation

Definition 9.2 (Spend-authorization scheme). `SphincsPlusAuth` is the `SpendAuth` implementation with `PublicKey = VerifyingKey<Sha2_128f>`, `SecretKey = SigningKey<Sha2_128f>` and `Signature = Signature<Sha2_128f>` [crates/pq-sign/src/spend_auth.rs:89-96], from the pre-release crate `slh-dsa 0.2.0-rc.5` [Cargo.toml:42]. Its operations are:

- **KeyGen**(ρ): `SigningKey::new(rng)` and its verifying key [crates/pq-sign/src/spend_auth.rs:98-104];
- **Sign**(sk, M): the `Signer` trait [crates/pq-sign/src/spend_auth.rs:106-110], which in the crate calls the context variant with $ctx = \varepsilon$ and no randomizer [slh-dsa-0.2.0-rc.5/src/signing_key.rs:244-253];
- **Verify**(vk, M, σ): the `Verifier` trait, mapped to `InvalidSignature` on failure [crates/pq-sign/src/spend_auth.rs:112-119], which in the crate verifies with $ctx = \varepsilon$ [slh-dsa-0.2.0-rc.5/src/verifying_key.rs:178-188];
- parsing: `VerifyingKey::try_from` and `Signature::try_from`, each requiring the exact length, mapped to `Malformed` [crates/pq-sign/src/spend_auth.rs:125-135] [slh-dsa-0.2.0-rc.5/src/verifying_key.rs:164-175] [slh-dsa-0.2.0-rc.5/src/signature_encoding.rs:77-80].

The message actually processed by the internal FIPS 205 routines is the pure-mode framing

$$M' = 0x00 \parallel 0x00 \parallel M(x) \quad (\text{domain byte } 0, |ctx| = 0, ctx = \varepsilon) \quad (91)$$

on both sides [slh-dsa-0.2.0-rc.5/src/signing_key.rs:190-200] [slh-dsa-0.2.0-rc.5/src/verifying_key.rs:108-119]. Signing is deterministic: with no randomizer supplied, `opt_rand` defaults to `PK.seed` [slh-dsa-0.2.0-rc.5/src/signing_key.rs:150-153], so σ is a function of (sk, M) only.

Parameter	Sha2_128f	Source
name	SLH-DSA-SHA2-128f	
n, m (bytes)	16, 34	[slh-dsa-0.2.0-rc.5/src/ hashes/sha2.rs:195-198]
h, d, h'	66, 22, 3	[slh-dsa-0.2.0-rc.5/src/ hashes/sha2.rs:178]
k, a	33, 6	[slh-dsa-0.2.0-rc.5/src/ hashes/sha2.rs:183-189]
$lg w, len$	4, 35	[slh-dsa-0.2.0-rc.5/src/ hashes/sha2.rs:190-194]
$ vk $	32 (PK.seed PK.root)	[slh-dsa-0.2.0-rc.5/src/ wots.rs:9-10] [slh-dsa-0.2.0-rc.5/src/ hashes/sha2.rs:179-182]
$ sk $	64 (SK.seed SK.prf vk)	[slh-dsa-0.2.0-rc.5/src/ verifying_key.rs:224-226]
$ \sigma $	17088	[slh-dsa-0.2.0-rc.5/src/ signing_key.rs:330-332]
		[slh-dsa-0.2.0-rc.5/src/ signature_encoding.rs:175-177]

The signature length is consistent with $|\sigma| = n + k(1 + a)n + (h + d \cdot len)n = 16 + 3696 + 13376$.

Spec delta 9.1 (Size and cost statements). Comments state an “8KB” signature [crates/pq-sign/src/lib.rs:21-24] [crates/pq-sign/src/spend_auth.rs:214] [crates/node/src/main.rs:87-89], a “64 bytes” verifying key [crates/node/src/wire.rs:42], and that **f** “verifies $\sim 16\times$ faster than **s**” [crates/pq-sign/src/lib.rs:21]. The enforced sizes are $|\sigma| = 17088$ and $|vk| = 32$. The 128s set in the same crate has $d = 7$, $k = 14$, $a = 12$ [slh-dsa-0.2.0-rc.5/src/ hashes/sha2.rs:155-175], i.e. fewer hypertree layers to recompute at verification than the $d = 22$ of 128f; the stated rationale does not follow from the parameters [L-08]. The size test only pins $7000 < |\sigma| < 20000$ [crates/pq-sign/src/spend_auth.rs:218-219].

Where the scheme is verified. Verify is called only natively, from `ShieldedTransfer::verify_signature` [crates/node/src/tx.rs:127-131] (and the wallet’s local pre-check). No circuit of Sections 4–5 and no contract of Section 8 evaluates it, and the verifying key is whatever the envelope carries [M-02].

Key lifetime in the wallet. A wallet vault holds exactly one SLH-DSA secret key, generated at vault creation [crates/wallet-wasm/src/lib.rs:298-310]; a password change re-seals the same keys [crates/wallet-wasm/src/lib.rs:320-332]; and every `TransferWire` the wallet emits carries $vk = \text{PK}(sk)$ of that key, hex-encoded, beside $M(x)$ ’s fields [crates/wallet-wasm/src/lib.rs:424-430, 464-476]. The 32-byte vk is therefore constant across all submissions from one vault.

9.3 The transfer wire format and /v1/transfer admission

Definition 9.3 (Wire DTO). `TransferWire` is a JSON object with fields `nullifiers`, `outputs` (arrays of hex strings), `root`, `root_after`, `nullifier_root_before`, `nullifier_root_after` (hex strings), `fee` (JSON integer, u64), `verifying_key` and `signature` (hex strings) [crates/node/src/wire.rs:26-46]. It derives `Deserialize` without `deny_unknown_fields`, so extra members are ignored. It carries no proof and no limb statement [crates/node/src/wire.rs:8-12].

Definition 9.4 (/v1/transfer admission predicate). A POST /v1/transfer request with body w is evaluated in this order; the first failing step determines the response.

- (W0) Guard: a valid bearer token with role $\geq$ **Submitter** (Def. 9.7), then the rate limiter [crates/node/src/auth.rs:283] [crates/node/src/main.rs:543-581]. Independently, an outer layer caps the body at 2^{20} bytes [crates/node/src/main.rs:90,737-739], and the handler's axum **Json** extractor must deserialize it as **TransferWire** [crates/node/src/main.rs:619].
- (W1) $k \geq 1$ and $m \geq 1$ [crates/node/src/wire.rs:71-73].
- (W2) Each of N_i , C_j , rt , rt' , R_{before} , R_{after} is a string of exactly 64 ASCII hex digits (**is_ascii_hexdigit**, either case), decoded to 32 bytes [crates/node/src/wire.rs:49-61,74-94].
- (W3) **verifying_key** hex-decodes and the result has length exactly 32 [crates/node/src/wire.rs:106-109].
- (W4) **signature** hex-decodes and the result has length exactly 17088 [crates/node/src/wire.rs:110-113].
- (W5) $M(x)$ is defined, i.e. $k, m \leq 1024$ [crates/node/src/tx.rs:117-119,127-128].
- (W6) **Verify**($vk, M(x), \sigma$) = 1 with vk and σ from the same request [crates/node/src/wire.rs:114-120] [crates/node/src/tx.rs:129-130].
- (W7) Committed-state check, executed on the actor [crates/node/src/main.rs:631-641,490-502]: **check_admit** against the committed roots [crates/node/src/state.rs:197-199], i.e. in order

$$rt = \text{root}(\mathcal{T}) \quad (92)$$

$$R_{\text{before}} = \text{root}(\mathcal{N}) \quad (93)$$

$$\forall i \leq k : N_i \notin \mathcal{N} \quad (94)$$

where $\mathcal{T}$ is the committed commitment tree and $\mathcal{N}$ the committed nullifier map [crates/node/src/state.rs:224-248].

If (W0)–(W7) all hold the response is 501 **Not Implemented** [crates/node/src/main.rs:643-648]; the transfer is not enqueued and no state changes.

Failing step	Status	Body / source
(W0) path routed but token missing, malformed, bad MAC, unknown role	401	token rejected [crates/node/src/main.rs:548-573]
(W0) token expired	401	token rejected [crates/node/src/main.rs:561]
(W0) role below Submitter	403	token rejected [crates/node/src/main.rs:562]
(W0) rate limit	429	rate limited [crates/node/src/main.rs:575-579]
(W1)–(W6)	422	envelope rejected: < TxError > [crates/node/src/main.rs:620-630]
(W7)	422	state rejected: < StateError > [crates/node/src/main.rs:649-656]
all pass	501	fixed text [crates/node/src/main.rs:643-648]

In (W1)–(W5) the error variant is not descriptive of the cause: any **to_public** failure and any hex-decoding failure of the key or signature is reported as **TxError::TooLarge**, and a key or signature of the wrong length as **TxError::BadSignature** [crates/node/src/wire.rs:105-113]. In (W7) the **StateError** text distinguishes a stale tree root, a stale nullifier root, and an already-spent nullifier [crates/node/src/state.rs:78-101].

Property 9.2 (What /v1/transfer does not check). The predicate of Def. 9.4 does not constrain rt' or R_{after} (they enter only $M(x)$), does not require the N_i to be pairwise distinct, does not check value conservation, verifies no proof, and takes vk from the request without relating it to any note, pk_d or registered identity [**M-02**]. Because (W6) accepts any self-generated key and (W7) evaluates (94) only after (92)–(93) hold against roots that /v1/roots publishes [crates/node/src/main.rs:591-600], the response to a **Submitter** reveals membership of chosen nullifiers in $\mathcal{N}$ [**I-06**].

Spec delta 9.2 (Wire-format comments). The module comment says hex is lowercase and that a malformed field is a 400 [crates/node/src/wire.rs:4-6]; the enforced check accepts either case [crates/node/src/wire.rs:50], and the handler answers 422 [crates/node/src/main.rs:625].

9.4 In-process mempool admission

The only path that enqueues a transfer is `Sequencer::submit`, called from the Admin-only demo endpoint (`/v1/demo/transfer`) after an in-process proof [crates/node/src/main.rs:403-412] [crates/node/src/auth.rs:290]. Its argument `ClientTransferProof` carries six independently supplied components: a verifier object, a proof, a limb statement, a shape, a `TransferPublic`, and an envelope [crates/node/src/sequencer.rs:62-75].

Definition 9.5 (Mempool admission). `submit( $t$ )` succeeds iff, in order,

- (S1) `Verify( $t.envelope.vk$ ,  $M(t.envelope.public)$ ,  $t.envelope.\sigma$ ) = 1` [crates/node/src/sequencer.rs:332];
- (S2) `check_admit_against( $t.public$ ,  $\text{root}(\mathcal{T}^{\text{pend}})$ ,  $\text{root}(\mathcal{N}^{\text{pend}})$ )`, i.e. (92)–(93) with the pending projections' roots in place of the committed ones, while (94) is still evaluated against the committed map $\mathcal{N}$ [crates/node/src/state.rs:242-246] [crates/node/src/sequencer.rs:337-341];
- (S3) `$t.verifier.verify(t.proof, t.statement)$` [crates/node/src/sequencer.rs:342-345];
- (S4) each N_i of $t.public$ inserts freshly into $\mathcal{N}^{\text{pend}}$ [crates/node/src/sequencer.rs:346-356];

after which the C_j are appended to $\mathcal{T}^{\text{pend}}$ and t is queued [crates/node/src/sequencer.rs:361-365].

Property 9.3 (Bindings not enforced at admission). `submit` compares none of $t.envelope.public$, $t.public$ and $t.statement$ with one another, and does not check $t.shape$ against $t.statement$ [crates/node/src/sequencer.rs:331-366]. The verifier object in (S3) is supplied with the transfer [crates/node/src/sequencer.rs:63-64] [V-06]. The shape/length check `| $t.statement$ | = shape.statement_len()` is first applied in `block_statement` [crates/prover/src/block.rs:334-342], during `produce_block`, after the batch has been drained from the mempool [crates/node/src/sequencer.rs:387-390] [M-06]. In the shipped binary the three components are produced together from one proving run [crates/node/src/main.rs:391-411], using fixture spend keys [crates/node/src/main.rs:281-289] [I-03].

The demo endpoint additionally requires `out_value + fee ≤ val(input)` (saturating) [crates/node/src/main.rs:361-366]. When a block is applied, `apply` re-checks per transfer that the N_i are unspent and pairwise distinct and that R_{after} and rt' equal the roots the insertions and appends produce, before mutating [crates/node/src/state.rs:271-322].

9.5 Bearer tokens and access control

Definition 9.6 (Token). Roles are bytes `ReadOnly = 0 < Submitter = 1 < Admin = 2` [crates/node/src/auth.rs:56-65]. For signing key K , role r and expiry $e \in [0, 2^{64})$ (Unix seconds),

$$P = r \parallel \text{LE}_{64}(e) \quad (|P| = 9) \quad (95)$$

$$\tau = \text{HMAC-SHA256}(K, P) \quad (|\tau| = 32) \quad (96)$$

$$\text{tok} = \text{b64u}(P) \parallel \text{"."} \parallel \text{b64u}(\tau) \quad (97)$$

[crates/node/src/auth.rs:175-179, 236-253], where `b64u` is the RFC 4648, Section 5 alphabet without padding [crates/node/src/auth.rs:306-324]. A token is therefore $12 + 1 + 43 = 56$ characters. Issuance is deterministic in (K, r, e) .

Definition 9.7 (Token verification). `Verify $_K$ (tok,  $t_{\text{now}}$ )` returns a role iff, in order [crates/node/src/auth.rs:190-211]:

- (T1) `tok` splits at its first `"."` into (s_P, s_τ) ;
- (T2) both halves decode under `b64u-1` (Def. 9.8);
- (T3) `ct_eq( $\tau'$ ,  $\text{HMAC}(K, P')$ )`, with `subtle`'s slice comparison (which returns false on a length mismatch before comparing contents);
- (T4) $|P'| = 9$;

(T5) $P'_0 \in \{0, 1, 2\}$;

(T6) $t_{\text{now}} \leq \text{LE}_{64}^{-1}(P'_{1..8})$.

`verify_for` additionally requires $r \geq r_{\text{req}}$ [crates/node/src/auth.rs:219-234].

Definition 9.8 (Decoder). $\text{b64u}^{-1}(s)$ rejects $|s| \equiv 1 \pmod{4}$ and any symbol outside the alphabet; it decodes groups of 4 symbols to 3 bytes, a final group of 3 symbols to 2 bytes, and a final group of 2 symbols to 1 byte [crates/node/src/auth.rs:332-373]. The low-order bits of the last symbol of a partial final group (2 bits for 3 symbols, 4 bits for 2 symbols) are discarded without being required to be zero [crates/node/src/auth.rs:351-359], so b64u^{-1} is not injective; the 43-symbol MAC field ends in a 3-symbol group, giving four accepted spellings of each token [L-03].

Path	Method	Required role	Handler / source
/health	GET	ReadOnly	snapshot JSON [crates/node/src/main.rs:727]
/metrics	GET	ReadOnly	Prometheus text [crates/node/src/main.rs:728]
/v1/state	GET	ReadOnly	same handler as /health [crates/node/src/main.rs:729]
/v1/roots	GET	ReadOnly	committed $\text{root}(\mathcal{T})$, $\text{root}(\mathcal{N})$ [crates/node/src/main.rs:730]
/v1/transfer	POST	Submitter	Def. 9.4 [crates/node/src/main.rs:731]
/v1/demo/transfer	POST	Admin	in-process prove and submit [crates/node/src/main.rs:732]
/v1/block/produce	POST	Admin	drain mempool, prove block [crates/node/src/main.rs:733]
/v1/block/settle	POST	Admin	send L1 transaction [crates/node/src/main.rs:734]
/v1/admin/rotate-key	—	Admin	listed in the ACL, no route [crates/node/src/auth.rs:292]

Roles per path are from `Acl::default_policy`, matched by exact string equality on the URI path [crates/node/src/auth.rs:270-302].

Definition 9.9 (Guard). The guard is installed with `route_layer` [crates/node/src/main.rs:735], so it runs only for requests that match a route; others receive the router’s default 404 without authentication. For a matched route with path π it (G1) looks up $r_{\text{req}} = \text{ACL}(\pi)$, answering 404 if absent; (G2) requires an `Authorization` header that is visible ASCII and begins with the exact prefix `Bearer` ; (G3) runs `verify_for` with the current Unix time; (G4) charges one cell to the keyed rate limiter under the key formed by the role’s Debug name, one space, and the path (e.g. `Submitter /v1/transfer`), answering 429 when exhausted [crates/node/src/main.rs:543-581]. The limiter quota is 20 per second with burst 40 per key [crates/node/src/main.rs:765-767]. Requests rejected in (G1)–(G3) are not charged. The key is shared by every token holder of the same role on the same path [M-11]. CORS allows the configured origins, methods GET and POST, and headers `Authorization` and `Content-Type` [crates/node/src/main.rs:717-725].

Key and issuance. K is read from `NODE_TOKEN_KEY_FILE` (regular file, mode without group/other bits, non-empty) or else from hex in `NODE_TOKEN_KEY` (at least 16 bytes) [crates/node/src/main.rs:138-157] [crates/node/src/keystore.rs:120-133]; `TokenSigner::new` accepts any length [crates/node/src/auth.rs:165-168]. The `token` subcommand issues $e = t_{\text{now}} + \text{ttl}$ with default role `admin` and default $\text{ttl} = 86400$, with no upper bound on ttl [crates/node/src/main.rs:837-853] [L-03].

Spec delta 9.3 (Token lifetime and ACL). The module comment gives a default lifetime of one hour [crates/node/src/auth.rs:41-43]; the shipped issuer defaults to 24 hours [crates/node/src/main.rs:839-840]. The ACL comment presents `/v1/admin/rotate-key` as key management [crates/node/src/auth.rs:291-292]; no handler exists, and revocation is only by restarting with a new K .

9.6 Custody: the password vault

Definition 9.10 (Seal). Constants: $|key| = 32$, $|salt| = 16$, $|nonce| = 12$, `VAULT_VERSION` = 1 [crates/vault/src/lib.rs:69-73,166]. The KDF is Argon2id, version 0x13, $m = 65536$ KiB, $t = 3$, $p = 1$, output 32 bytes, validated at compile time [crates/vault/src/lib.rs:82-112]. `Seal(pw, pt, max)` requires $|pt| \leq \text{max}$, draws salt then nonce from the caller’s RNG, and outputs

$$\text{key} = \text{Argon2id}(\text{pw}, \text{salt}) \quad (98)$$

$$\text{ct} = \text{AES-256-GCM.Enc}(\text{key}, \text{nonce}, \text{pt}; \text{ad} = \varepsilon) = c \parallel T, \quad |T| = 16 \quad (99)$$

as $(\text{ver} = 1, \text{salt}, \text{nonce}, \text{ct})$ [crates/vault/src/lib.rs:211-255]. No associated data is passed, so the version byte and length fields are not authenticated by the tag.

Definition 9.11 (Open). `Open(pw)` requires, in order, $ver = 1$, $|salt| \geq 16$, $|nonce| \geq 12$; derives *key*; requires $|nonce| = 12$ (else a `Kdf` error); and returns the plaintext iff GCM decryption succeeds, else `WrongPassword` [crates/vault/src/lib.rs:269–288].

Definition 9.12 (Serialized vault). `to_bytes` emits

$$ver \parallel \text{LE}_8(|salt|) \parallel salt \parallel \text{LE}_8(|nonce|) \parallel nonce \parallel \text{LE}_{32}(|ct|) \parallel ct, \quad (100)$$

refusing lengths that do not fit their prefix [crates/vault/src/lib.rs:308–326]; for a sealed vault this is $35 + |pt| + 16$ bytes. `from_bytes` requires total length ≥ 35 , then reads the version, the salt-length byte, the salt (via a bounds-checked `read_slice`), the nonce-length byte at the current offset, the nonce, four ciphertext-length bytes at the current offset, and the ciphertext [crates/vault/src/lib.rs:334–358, 362–370]. It does not check the version (deferred to `Open`) and accepts trailing bytes after *ct*. The nonce-length byte and the four length bytes are read by direct indexing, which is not bounds-checked when a declared length consumes the remaining input [crates/vault/src/lib.rs:344, 347–349] [L-01].

Consumer	Plaintext	Entropy for <i>salt</i> , <i>nonce</i>
node keystore	caller bytes	OS CSPRNG (<code>SysRng</code>) [crates/node/src/keystore.rs:57–65]
wallet (wasm)	$0x01 \parallel sk_d(32) \parallel sk_{SLH}(64)$, 97 bytes, cap 4096	<code>StdRng</code> seeded from exactly 32 caller-supplied bytes, the same stream that generated sk_d and sk_{SLH} [crates/wallet-wasm/src/lib.rs:217–228, 261–267, 298–318]

On unlock the wallet requires the opened payload to be exactly 97 bytes with leading version byte $0x01$ [crates/wallet-wasm/src/lib.rs:278–292], and parses sk_{SLH} with `SigningKey::try_from`, which parses the embedded vk [crates/wallet-wasm/src/lib.rs:427–428] [slh-dsa-0.2.0-rc.5/src/signing_key.rs:226–240]. The wallet signs $M(x)$ from `shielded::encode_statement`, the same function the node uses (Def. 9.1) [crates/wallet-wasm/src/lib.rs:424–430].

10 Specification deltas (documentation vs enforced code)

This section collects the places where the repository’s own prose (the top-level README, module and contract doc comments, deployment-script comments, and the vendored-patch record) states a property that differs from the one the code enforces. Each item gives (i) the documentation location, (ii) the code location, and (iii) which of the two the client intends to be authoritative, judged only from evidence inside the repository (the decision log and migration notes under `.scratch/pq-shielded-rollup/`, in-code rationale, and compile-time guards). Where the repository records no intent, the item says so. Everywhere else in this document the *code* is the specification; the items below mark where a reader of the documentation alone would form a different model. Several deltas were already noted locally in earlier sections; this section consolidates them. The umbrella informational finding is [I-02].

10.1 Summary

#	Topic	Documentation states	Code enforces	Intended
1	Shielded and commitment hashes	SHA3-256 for pk_d and nf ; Keccak-256 for cm and for the nullifier tree	Poseidon2 (H_{P2} , C_{P2}) for pk_d , nf , cm , the note tree and the nullifier tree; documented ~ 96 -bit sponge margin	Code
2	Caller of <code>applyBlock</code>	an “operator set allowed to call <code>applyBlock</code> ”	no caller restriction of any kind	Not recorded
3	Empty nullifier root	computed at deploy: Keccak chain, depth 256	pinned Poseidon2 constant, depth 96; no hashing at deploy	Code
4	Vendored <code>p3-recursion</code> patch record	one file (<code>pcs.rs</code>); “ZK false $\rightarrow$ true”; “evaluation truncation”; a pristine upstream copy in-tree	five changed code files; ZK is a const generic defaulting to <code>true</code> ; no truncation; no pristine copy	Code

#	Topic	Documentation states	Code enforces	Intended
5	Security level	“~128-bit”; “the 128-bit target”	SECURITY_LEVEL = 96; level, regime, folding, rate, grinding and LDE are environment-overridable in library code	Code (96)
6	WhirVerifier wire-format header	STATEMENT “not consumed here”; eq-points carried in PROOF	STATEMENT supplies the initial constraint’s opening points; the eq-points blob is gone from the wire	Code
7	Fees on settlement	“roots + fees move”; the chain verifies “fee accounting”	fee total decoded and emitted only; no payable entry point	Not recorded

10.2 Hash instantiation of the shielded and commitment layers

Spec delta 10.1 (SHA3-256 / Keccak-256 in prose, Poseidon2 in code). **Documentation.** The README states that “nullifiers are SHA3-256” [README.md:11-12], labels the wallet column “SHA3-256 nullifiers” [README.md:23], defines the note commitment as “Keccak-256(DOMAIN || value || rho || psi || pk_d)”, calls the nullifier map “a Keccak-256 map” and the nullifiers “SHA3-256 over (DOMAIN || sk_d || rho)” [README.md:31-38], and describes crates/pq-hash as “SHA3-256 note/nullifier hashing” [README.md:60]. The pq-hash layering table assigns SHA3-256 to the shielded layer and Keccak-256 to the nullifier tree [crates/pq-hash/src/lib.rs:16-21], and the crate preamble names Sha3_256Shielded as the shielded hasher [crates/pq-hash/src/lib.rs:11-12]. The note methods describe the commitment as Keccak-256 and the nullifier as SHA3-256 [crates/shielded/src/note.rs:104-106,118-120]. The prover’s layering diagrams describe the shielded layer as SHA3-256 computed in-circuit on Keccak-f[1600] rows with the 0x06 pad byte [crates/prover/src/config.rs:16-18] [crates/prover/src/whir.rs:30-31].

Code. With H_{P2} the Poseidon2 byte hash and C_{P2} the Poseidon2 two-to-one compression of Section 2, the enforced relations are

$$pk_d = H_{P2}(\text{DOMAIN_PK} \parallel sk_d) \quad [\text{crates/shielded/src/keys.rs:40-42}] \quad (101)$$

$$nf = H_{P2}(\text{DOMAIN_NULLIFIER} \parallel sk_d \parallel \rho) \quad [\text{crates/shielded/src/note.rs:125-126}] \quad (102)$$

$$cm = H_{P2}(\text{DOMAIN_NOTE} \parallel \text{LE64}(v) \parallel \rho \parallel \psi \parallel pk_d) \quad [\text{crates/shielded/src/note.rs:108-111}] \quad (103)$$

$$E_0 = 0^{32}, \quad E_h = C_{P2}(E_{h-1}, E_{h-1}) \quad (1 \leq h \leq 96) \quad [\text{crates/shielded/src/nullifier_tree.rs:129-134}] \quad (104)$$

where the shielded hasher is Poseidon2Shielded, whose hash_to_digest(dom, parts) is the commitment byte hash of dom || parts with no length prefixes [crates/pq-hash/src/shielded.rs:98-104], and hash_pair of Poseidon2Commitment is C_{P2} [crates/pq-hash/src/poseidon.rs:240-242]. These hashers are the ones instantiated at every production site: the node’s committed tree and nullifier map [crates/node/src/state.rs:48, 50, 110-111], the wallet wasm [crates/wallet-wasm/src/lib.rs:347, 520, 533, 536], and the native client prover [crates/prover/src/client.rs:166, 204]. In-circuit, nf and cm are Poseidon2 sponge evaluations [crates/prover/src/transfer.rs:456, 580], and the transfer module states that SHA3-256 and Keccak-f[1600] are absent from the circuit [crates/prover/src/transfer.rs:30-43]. Sha3_256Shielded and Keccak256Commitment remain exported [crates/pq-hash/src/lib.rs:55, 62] but are referenced only from test code (e.g. [crates/shielded/src/nullifier_tree.rs:361-364]); the prover::sha3_block module is declared [crates/prover/src/lib.rs:56] and referenced by no other module. Keccak-256 remains the hash of the settlement layer’s Merkle tree and Fiat–Shamir transcript only [crates/prover/src/config.rs:66-90] [crates/prover/src/whir.rs:77].

Margin. The shielded hasher’s documentation records the collision bound as the Poseidon2 sponge’s “~96-bit classical margin at this width”, not SHA3’s [crates/pq-hash/src/shielded.rs:82-85], and the nullifier-tree address is the low 96 bits of nf [crates/shielded/src/nullifier_tree.rs:56-64]. The crate-level statement of “the same 128-bit collision security” [crates/pq-hash/src/lib.rs:23-28] compares Keccak-256 with SHA3-256 and no longer describes the shielded layer.

Intended: code. The migration record cites the directive “no sha3, only poseidon2” and lists the SHA3/Keccak removals as consensus-critical [.scratch/pq-shielded-rollup/verifier-redesign.md:3456-3478]; the code records operator acceptance of the reduced margin [crates/pq-hash/src/shielded.rs:85] [crates/prover/src/nullifier_gadget.rs:17-18] [crates/prover/src/transfer.rs:41-43]. Related enforcement gaps are catalogued as [V-07] (in-circuit sponge) and [M-09] (nullifier address width).

10.3 Who may settle

Spec delta 10.2 (No operator set on `applyBlock`). **Documentation.** The README’s trust statement lists “the operator set allowed to call `applyBlock`” among the things the chain trusts [README.md:160-162].

Code. `applyBlock` is declared `external` with no modifier [contracts/src/ShieldedPool.sol:144]; the contract stores no operator, role or allow-list (its state is `verifier`, `blockNumber`, `currentRoot`, `currentNullifierRoot`, `feeRecipient`) [contracts/src/ShieldedPool.sol:68-89], and the only `msg.sender` comparison in the contract guards `withdrawFees` [contracts/src/ShieldedPool.sol:183]. The enforced relation is, for any caller,

$$\begin{aligned} \text{applyBlock}(s, \pi) \text{ succeeds} &\iff \text{verifier.verify}(s, \pi) = \text{true} \\ &\quad \wedge \text{rootBefore}(s) = \text{currentRoot} \\ &\quad \wedge \text{nullifierBefore}(s) = \text{currentNullifierRoot} \end{aligned} \quad (105)$$

[contracts/src/ShieldedPool.sol:145-157] after which `(currentRoot, currentNullifierRoot)` $\leftarrow$ `(rootAfter(s), nullifierAfter(s))` and `blockNumber` is incremented [contracts/src/ShieldedPool.sol:165-170]. The contract’s own trust-model text treats the prover and sequencer as untrusted and gates every transition solely on a verified proof [contracts/src/ShieldedPool.sol:51-55], which matches the code; the node’s settlement sender submits from a configured unlocked account and registers no operator [.scratch/pq-shielded-rollup/decisions.md:1342-1355].

Intended: not recorded. No decision-log entry or code path introduces an operator set; the README sentence is the only occurrence. This specification takes the enforced relation above (permissionless settlement) as normative. Consequences are catalogued in [V-01], [H-01] and [M-01].

10.4 Genesis nullifier root

Spec delta 10.3 (Keccak depth-256 computed root in prose, Poseidon2 depth-96 constant in code). **Documentation.** The constructor documentation states that passing `bytes32(0)` makes the contract compute “the prover’s empty-map root itself (depth 256, empty leaf zero, `empty[h] = keccak(empty[h-1] || empty[h-1])` ... ~256 keccak at deploy)” [contracts/src/ShieldedPool.sol:114-118]. The deployment script says `bytes32(0)` “makes ShieldedPool compute the empty sparse-tree root itself; the genesis file may pin one explicitly” [contracts/script/Deploy.s.sol:42-43]. The nullifier-tree module documentation describes full 256-bit addressing, 256 levels, and a path through bits 0, ..., 255 [crates/shielded/src/nullifier_tree.rs:14-22,45-47].

Code. The contract pins

```
EMPTY_NULLIFIER_ROOT = 0x83265b7a2c40cc6eca237b6016b76460
                        6335cd0c6ff0732470e20a484d5a7329
```

(one 32-byte word, printed in two halves) [contracts/src/ShieldedPool.sol:85-86], documented in the same file as the Poseidon2 empty-subtree chain at `NULLIFIER_TREE_DEPTH = 96` [contracts/src/ShieldedPool.sol:79-84]. The constructor performs no hashing:

$$\text{currentNullifierRoot} = \begin{cases} \text{EMPTY_NULLIFIER_ROOT} & \text{genesisNullifierRoot_} = 0^{256} \\ \text{genesisNullifierRoot_} & \text{otherwise} \end{cases} \quad (106)$$

[contracts/src/ShieldedPool.sol:134-135] The deployment script passes `bytes32(0)` unconditionally [contracts/script/Deploy.s.sol:44] and reads only `.genesis_root_hex` from the genesis file [contracts/script/Deploy.s.sol:32]; no nullifier-root field is consulted. On the Rust side `NULLIFIER_TREE_DEPTH = 96` [crates/shielded/src/nullifier_tree.rs:64], the empty-map root is E_{96} of Delta 10.1, and two nullifiers agreeing on the low 96 address bits are treated as the same address [crates/shielded/src/nullifier_tree.rs:56-63,79-83]. The pinned constant equals the `nullifier_before_hex` of the committed block vector [contracts/test/vectors/block_vectors.json:4].

Intended: code. The migration record states “nullifier tree: Keccak $\rightarrow$ Poseidon2, depth 256 $\rightarrow$ 96” and “`_emptyNullifierRoot()` keccak loop deleted; `EMPTY_NULLIFIER_ROOT` constant ... pinned” [.scratch/pq-shielded-rollup/verifier-redesign.md:3469-3477]. The constructor comment and the deployment-script comment predate that change. The 96-bit address width is the subject of [M-09].

10.5 Vendored recursion tree

Spec delta 10.4 (PATCHES.md under-reports and mis-describes the vendored diff). **Documentation.** `vendor/p3-recursion/PATCHES.md` records the upstream revision `8f9876efb26cff0f4b837555b478f043c4314240`

[vendor/p3-recursion/PATCHES.md:3-4], states that a pristine copy is kept at `.scratch/vendor/p3-recursion-pristine` from which the complete diff is reproducible [vendor/p3-recursion/PATCHES.md:17-22], and lists a single patch: `recursion/src/pcs/whir/uni/pcs.rs`, “`const ZK: bool = false → true`; hiding-aware `commit`, `commit_preprocessing`, `get_opt_randomization_poly_commitment`, and evaluation truncation” [vendor/p3-recursion/PATCHES.md:35-37]. It cites decision-log entries D-046 and D-051 for the rationale [vendor/p3-recursion/PATCHES.md:13,59].

Code. A recursive comparison of the vendored tree with a clean checkout of the upstream repository at the recorded revision (excluding `.git` and `target`) yields exactly the following differences besides `PATCHES.md` and the `.cargo-ok` marker:

File	Change
<code>recursion/Cargo.toml</code>	<code>rand</code> gains the <code>sys_rng</code> feature and a <code>spin</code> dependency is added [vendor/p3-recursion/recursion/Cargo.toml:45-54]; the masking RNG is seeded from OS entropy [vendor/p3-recursion/recursion/src/pcs/whir/uni/pcs.rs:419-422].
<code>recursion/src/backend/whir.rs</code>	upstream rejects inputs with <code>config.is_zk() != 0</code> (“supports only non-ZK STARK inputs”) at two sites; both checks are removed, the surrounding code following directly at [vendor/p3-recursion/recursion/src/backend/whir.rs:562-565,643-647].
<code>recursion/src/input_contract/whir.rs</code>	test only: <code>whir_config(8)</code> becomes <code>whir_config(9)</code> to match the masked stacked arity [vendor/p3-recursion/recursion/src/input_contract/whir.rs:1257-1260].
<code>recursion/src/pcs/whir/uni/pcs.rs</code>	ZK becomes a <code>const</code> generic of <code>WhirUniPcs</code> with default <code>true</code> [vendor/p3-recursion/recursion/src/pcs/whir/uni/pcs.rs:322], forwarded by <code>const ZK: bool = ZK</code> [vendor/p3-recursion/recursion/src/pcs/whir/uni/pcs.rs:967] (upstream: <code>const ZK: bool = false</code>); hiding-aware <code>commit/open</code> paths are gated on it.
<code>recursion/src/pcs/whir/uni/recursive_pcs.rs</code>	the <code>RecursivePcs</code> impl is generalised over <code>const ZK: bool</code> [vendor/p3-recursion/recursion/src/pcs/whir/uni/recursive_pcs.rs:361-368].

No evaluation truncation exists: `evaluations_on_domain` asserts $|domain| \geq$ the committed height and zero-pads the coefficients before the coset DFT [vendor/p3-recursion/recursion/src/pcs/whir/uni/pcs.rs:538-554]. The directory `.scratch/vendor/` does not exist in the working tree. The decision log has no D-046 or D-051 entry; D-046 is only mentioned in passing [scratch/pq-shielded-rollup/decisions.md:170,369]. The “`false→true`” description matches only the *default*: the workspace instantiates the settlement PCS with `ZK = false` [crates/prover/src/whir.rs:86] [crates/prover/src/settlement_replay.rs:87] and the inner (client-proof) PCS with the default [crates/prover/src/whir_recursion.rs:106-113], and pins both choices with compile-time assertions [crates/prover/src/lib.rs:102-113]. The item-2 analysis in `PATCHES.md` cites line numbers that still hold [vendor/p3-recursion/recursion/src/pcs/whir/uni/recursive_pcs.rs:384] [vendor/p3-recursion/recursion/src/verifier/batch_stark.rs:1528].

Intended: code. The tree that builds is the vendored one, and the non-ZK settlement layer is a recorded, deliberate choice [crates/prover/src/lib.rs:92-101]; `PATCHES.md` was not updated with the later changes. The removed upstream ZK guard and the hiding claims are the subject of [L-07] and [M-07]; the non-ZK settlement witness is the subject of [M-10].

10.6 Security level and configuration knobs

Spec delta 10.5 (“128-bit” in prose, 96-bit target in code, overridable at run time). **Documentation.** `config.rs` states that the degree-5 challenge field gives “~128-bit conjecturable security” [crates/prover/src/config.rs:28-30,46-50]. The `whir.rs` module header states that “`SECURITY_LEVEL` is set directly to the 128-bit target” [crates/prover/src/whir.rs:43-46].

Code. The degree-5 FRI Config of `config.rs` is used only by the test-gated `spike` module [crates/prover/src/lib.rs:64-65] [crates/prover/src/spike.rs:18]; the deployed path takes only $\mathbb{F}_p$, the Keccak Merkle MMCS and the Keccak challenger from it [crates/prover/src/whir.rs:60,346-350]. The settlement challenge field is the degree-4 binomial extension $\mathbb{K}$ [crates/prover/src/whir.rs:70], and

$$\text{SECURITY_LEVEL} = 96 \quad [\text{crates/prover/src/whir.rs:120}] \quad (107)$$

with an in-code derivation that 128 bits exceeds the structural ceiling on this field [crates/prover/src/whir.rs:97-119]; the recursion layer is tied to the same constant [crates/prover/src/whir_recursion.rs:118-123].

The following environment variables are read by library code (not under `cfg(test)` or a feature) whenever a configuration is built:

Variable	Effect	Source
WHIR_SECURITY_LEVEL	replaces the 96-bit target, settlement and inner; parsed as <code>usize</code> , no bound	[crates/prover/src/whir.rs:126-131,211] [crates/prover/src/whir_recursion.rs:212]
WHIR_SOUNDNESS_REGIME	<code>udr/capacity/Johnson</code> (default)	[crates/prover/src/whir.rs:182-188]
WHIR_INNER_SOUNDNESS_REGIME	same, inner layer	[crates/prover/src/whir_recursion.rs:186-193]
WHIR_FOLDING_FINAL	settlement folding factor $\in \{4, 8\}$	[crates/prover/src/whir.rs:147-153]
WHIR_INNER_RATE	inner starting rate $\in \{1, 2, 3\}$	[crates/prover/src/whir_recursion.rs:314-318]
WHIR_POW_FLOOR, WHIR_INNER_POW_FLOOR	grinding floor	[crates/prover/src/settlement_replay.rs:151-154] [crates/prover/src/whir_recursion.rs:345-348]
WHIR_FINAL_LDE	settlement LDE height	[crates/prover/src/composed_export.rs:1255-1258]

The node reaches these on its settlement path (`settlement_bundle` [crates/node/src/main.rs:453] $\rightarrow$ [crates/prover/src/composed_export.rs:1237,1259] $\rightarrow$ [crates/prover/src/settlement_replay.rs:180] $\rightarrow$ [crates/prover/src/whir.rs:209-211]). The grinding-budget search bound uses the constant rather than the overridden value [crates/prover/src/whir.rs:276]. On-chain, the WHIR schedule is read from the bundle’s CONFIG section rather than fixed by the deployed contract (Delta 10.6), so the deployed verifier does not itself fix the level a settled proof attains [V-01].

Intended: code (96 bits, fixed). The 96-bit rationale is explicit [crates/prover/src/whir.rs:97-120], and the overrides are documented as experiment-only (“shipping below 96 bits is a protocol decision, never a default”) [crates/prover/src/whir.rs:122-125,178-181], yet they are compiled into the production path; see [I-01].

10.7 Settlement-verifier header

Spec delta 10.6 (STATEMENT section described as unconsumed). **Documentation.** The `WhirVerifier` header describes the bundle as CONFIG, PROOF and “STATEMENT (audit surface, not consumed here)” [contracts/src/verifier/WhirVerifier.sol:15-17]; lists among PROOF contents “the zeta-derived eq-points (D-072 phase 1)” [contracts/src/verifier/WhirVerifier.sol:26-29]; states that every domain point is “re-computed here, never read from the proof” [contracts/src/verifier/WhirVerifier.sol:31-35]; and states that CONFIG is deploy-time data that “a deployment should pin” [contracts/src/verifier/WhirVerifier.sol:22-25]. An inline comment repeats “zeta-derived, proof-supplied, D-072 phase 1” [contracts/src/verifier/WhirVerifier.sol:494-498].

Code. `verify` locates the STATEMENT section from its u32 little-endian word count after PROOF and bounds-checks it [contracts/src/verifier/WhirVerifier.sol:244-257]; every opening round receives it [contracts/src/verifier/WhirVerifier.sol:465-470] and sets the initial WHIR constraint to mode 2 over that slice [contracts/src/verifier/WhirVerifier.sol:503-513]; the raw slice is copied into the `TerminalWeight` call frame [contracts/src/verifier/WhirVerifier.sol:826-840], which derives the initial constraint’s equality groups from the opening points it contains [contracts/src/verifier/TerminalWeight.sol:38-40,108-125]. The per-round eq-points blob is no longer on the wire [contracts/src/verifier/WhirVerifier.sol:1101-1102]. Hence STATEMENT is consumed: it supplies the opening points of each round’s initial constraint. The deployed verifier holds no CONFIG digest (its only pin is the satellite code hash [contracts/src/verifier/WhirVerifier.sol:122,131,895]); a CONFIG-pinning wrapper exists as `WhirVerifierV6` [contracts/src/verifier/WhirVerifierV6.sol:44,63], but the deployment script constructs `WhirVerifier` and passes it to the pool [contracts/script/Deploy.s.sol:41,45].

Intended: code. The D-086 step C comments at the consuming sites describe the current behaviour [contracts/src/verifier/WhirVerifier.sol:503-506,827-830,1101-1102]; the header predates them. The opening points so supplied are not related to the transcript’s out-of-domain challenge, which is [V-02]; the unpinned CONFIG is [V-01].

10.8 Fees

Spec delta 10.7 (Fees described as moving on settlement). **Documentation.** The README diagram says “roots + fees move” on `applyBlock` [README.md:23-26], and the security model lists “fee accounting” among what the chain verifies [README.md:157-159].

Code. `ShieldedPool` declares no `payable` function, `receive` or `fallback` [contracts/src/ShieldedPool.sol:63-190]. The block fee is decoded as $\sum_i \text{fee}_i$ over the per-transfer fee limbs [contracts/src/BlockStatement.sol:104-107] and is only emitted in `BlockApplied` [contracts/src/ShieldedPool.sol:171-178]; `withdrawFees` forwards whatever balance the contract holds [contracts/src/ShieldedPool.sol:182-189]. The contract’s own documentation says it does not check fees [contracts/src/ShieldedPool.sol:59-62].

Intended: not recorded. See [I-04].

10.9 Pointer to the findings

The deltas above are documentation defects in their own right, but several of them mark exactly the places where the enforced relation is weaker than the documented one. The security consequences of these deltas, and of the enforcement gaps noted throughout Sections 1–9, are catalogued in the audit report under `zk-findings/reports/`: findings [V-01]–[V-08] (Critical), [H-01]–[H-03] (High), and [M-01]–[M-11] (Medium), with the low-severity and informational items (including [I-01], [I-02] and [I-04] cited here) listed alongside them. This specification states what the code enforces; the report states what that permits.